

INTELLIGENT MOBILE APPS WITH .NET MAUI

SUCCINCTLY®

BY **ALESSANDRO
DEL SOLE**

Intelligent Mobile Apps with .NET MAUI Succinctly[®]

Alessandro Del Sole

Foreword by Daniel Jebaraj



Copyright © 2026 by Syncfusion, Inc.

2501 Aerial Center Parkway

Suite 111

Morrisville, NC 27560

USA

All rights reserved.

ISBN: 978-1-64200-251-5

Important licensing information. Please read.

This book is available for free download from www.syncfusion.com on completion of a registration form.

If you obtained this book from any other source, please register and download a free copy from www.syncfusion.com.

This book is licensed for reading only if obtained from www.syncfusion.com.

This book is licensed strictly for personal or educational use.

Redistribution in any form is prohibited.

The authors and copyright holders provide absolutely no warranty for any information provided.

The authors and copyright holders shall not be liable for any claim, damages, or any other liability arising from, out of, or in connection with the information in this book.

Please do not use this book if the listed terms are unacceptable.

Use shall constitute acceptance of the terms listed.

BOLDDesk, BOLDSIGN, BOLD BI, BOLD REPORTS, SYNCFUSION, ESSENTIAL, ESSENTIAL STUDIO, SUCCINCTLY, and the 'Cody' mascot logo are the registered trademarks of Syncfusion, Inc.

Technical Reviewer: James McCaffrey

Copy Editor: Graham High, content team lead, Syncfusion, Inc.

Acquisitions Coordinator: Graham High, content team lead, Syncfusion, Inc.

Proofreader: Jacqueline Bieringer, content producer, Syncfusion, Inc.

Table of Contents

The <i>Succinctly</i> Series of Books	8
About the Author	9
Introduction.....	11
Target audience	11
What you will learn	11
How this ebook is structured.....	12
Chapter 1 Introducing Azure AI Services	13
The role of AI in mobile applications	13
Overview of Azure AI services	13
The Microsoft Responsible AI Standard and principles	14
Azure AI Vision	14
Azure AI Language	16
Azure AI Speech.....	17
Azure OpenAI Service	18
Azure AI Search.....	19
Azure AI Document Intelligence.....	20
Programming with Azure AI services	21
The AI-powered travel companion app	22
Chapter summary.....	22
Chapter 2 Setting Up the Development Environment.....	23
Registering for an Azure subscription	23
Locating Azure AI services	23
Creating a resource group	24
Installing .NET workloads	26

Creating a .NET MAUI solution.....	28
Chapter summary.....	30
Chapter 3 Implementing Image Analysis.....	31
Setting up Azure AI Vision.....	31
Implementing image analysis in .NET MAUI.....	33
Installing the AI Vision NuGet packages	34
Specifying permissions	34
Defining the user interface	36
Exception handling	38
Performing image analysis in C#	38
Performing OCR.....	41
Adding the page to the shell	43
Running the application	44
Chapter summary.....	46
Chapter 4 Speech Recognition and Text-to-Speech.....	47
Setting up Speech Services	47
Specifying permissions	48
Implementing speech recognition	49
Defining the user interface	49
Performing speech recognition in C#	50
Implementing text-to-speech	54
Running the application	55
Chapter summary.....	56
Chapter 5 Natural Language Processing and Chatbots.....	57
Setting up the Azure AI services.....	57
Setting up the Azure AI Language service	57

Setting up the Azure Translator service	59
Setting up the Azure OpenAI service	60
Installing NuGet packages.....	64
Analyzing text sentiment.....	65
Hints about more advanced text processing	67
Running the example	68
Executing AI-powered translations	69
Running the example	72
Implementing a basic ChatGPT user interface	74
Understanding completion responses	76
Running the example	77
Chapter summary.....	78
Chapter 6 Implementing AI-Driven Recommendations	79
Setting up the Search service.....	79
Defining a search schema	80
Retrieving the admin key	82
Implementing AI Search	83
Defining the user interface	84
Executing intelligent search in C#	87
Running the application	91
Implementing recommendations	92
Chapter summary.....	93
Chapter 7 Extracting Data from Documents with AI.....	94
Setting up the Azure AI Document Intelligence service.....	94
Creating a sample invoice	95
Installing NuGet packages.....	96

Defining the user interface.....	97
Extracting document data in C#.....	97
Running the application	100
Chapter summary.....	101
Chapter 8 AI-Optimized Routes with Azure Maps Accounts.....	102
Setting up the Azure Maps service	102
Setting up the Google Maps SDK.....	103
Implementing Maps	105
Specifying permissions	106
Defining the user interface	107
Calculating routes	108
Adding the page to the navigation stack	112
Running the application	112
Chapter summary.....	114
Conclusion	115

The *Succinctly* Series of Books

Daniel Jebaraj
CEO of Syncfusion, Inc.

When we published our first *Succinctly* series book in 2012, *jQuery Succinctly*, our goal was to produce a series of concise technical books targeted at software developers working primarily on the Microsoft platform. We firmly believed then, as we do now, that most topics of interest can be translated into books that are about 100 pages in length.

We have since published over 200 books that have been downloaded millions of times. Reaching more than 2.7 million readers around the world, we have more than 70 authors who now cover a wider range of topics, such as Blazor, machine learning, and big data.

Each author is carefully chosen from a pool of talented experts who share our vision. The book before you and the others in this series are the result of our authors' tireless work. Within these pages, you will find original content that is guaranteed to get you up and running in about the time it takes to drink a few cups of coffee.

We are absolutely thrilled with the enthusiastic reception of our books. We believe the *Succinctly* series is the largest library of free technical books being actively published today. Truly exciting!

Our goal is to keep the information free and easily available so that anyone with a computing device and internet access can obtain concise information and benefit from it. The books will always be free. Any updates we publish will also be free.

Let us know what you think

If you have any topics of interest, thoughts, or feedback, please feel free to send them to us at succinctlyseries@syncfusion.com.

We sincerely hope you enjoy reading this book and that it helps you better understand the topic of study. Thank you for reading.

Please follow us on social media and help us spread the word about the *Succinctly* series!



About the Author

Alessandro Del Sole is a Xamarin Certified Mobile Developer and a former Microsoft MVP. Awarded MVP of the Year in 2009, 2010, 2011, 2012, and 2014, he is internationally considered a Visual Studio expert and a .NET authority. Alessandro has authored many printed books and ebooks on programming with Visual Studio, including [.NET MAUI Succinctly](#), *Visual Basic 2015 Unleashed*, and [Xamarin.Forms Succinctly](#). He has written tons of technical articles about .NET, Visual Studio, and other Microsoft technologies in Italian and English for many developer portals. He has been a frequent speaker at Italian conferences, and he has also produced many instructional videos in both English and Italian. Alessandro works as a senior software engineer for Fresenius Medical Care, focusing on building mobile apps with .NET MAUI in the healthcare market. You can follow him on [LinkedIn](#), and support him with a [coffee](#).

To Luca Daniele, who is now coding for the angels.

Introduction

The rapid evolution of artificial intelligence (AI) has transformed the way we interact with mobile applications. From voice assistants to intelligent image recognition, AI is becoming an integral part of modern mobile experiences. Developers today have the opportunity to build smarter, more engaging applications that can understand and respond to users in natural and intuitive ways. This ebook will guide you through the process of integrating Microsoft AI services into mobile applications using .NET MAUI. By leveraging Azure AI services, developers can enhance their apps with features such as speech recognition, text analytics, image processing, and AI-powered recommendations. The goal of this ebook is to provide a practical, hands-on approach to implementing these AI capabilities efficiently and effectively.

Target audience

This ebook is intended for mobile developers who have intermediate or better knowledge of .NET MAUI and are looking to enhance their applications with AI capabilities. This ebook will provide clear explanations and practical examples to help you get started. If you need guidance about .NET MAUI, it is recommended that you first read [.NET MAUI Succinctly](#) from the author of this ebook. The assumption is that you are familiar with creating Visual Studio solutions, that you know how to add pages and debug applications, and that you have at least a basic knowledge of development platforms that use XAML (eXtensible Application Markup Language) to define the user interface, such as .NET MAUI, Xamarin.Forms, or Universal Windows Platform.

What you will learn

This ebook will cover key AI services provided by Microsoft, demonstrating how to integrate them into .NET MAUI applications. The topics include:

- Computer Vision: Analyzing images, extracting text, and detecting objects.
- Speech Services: Implementing speech-to-text and text-to-speech functionalities for natural voice interactions.
- Text Analytics: Analyzing language to determine sentiment, extract key phrases, and translate text.
- AI-powered recommendations: Delivering personalized content to users.
- Document Intelligence: Extracting structured data from documents such as invoices and receipts.
- ChatGPT capabilities: Implementing natural language conversations with chatbots based on the popular ChatGPT models.

You will combine and integrate all these services into a travel companion sample application to provide an enhanced user experience.

How this ebook is structured

Each chapter of this ebook focuses on a specific AI capability, providing step-by-step instructions and code examples to help you integrate AI services into your mobile applications. Starting with setting up the development environment and configuring Azure AI, you will gradually explore different AI functionalities, ensuring that you gain practical insights into their implementation. By the end of this ebook, you will have a strong foundation in integrating Microsoft AI services with .NET MAUI, enabling you to build intelligent mobile applications that offer enhanced interactivity, automation, and personalization. The companion source code is available on [GitHub](#).

Chapter 1 Introducing Azure AI Services

Microsoft [Azure AI services](#), formerly Azure Cognitive Services, provide a set of APIs and tools that allow developers to add intelligent features to their applications without needing in-depth knowledge of AI or data science. These services cover various aspects of artificial intelligence, such as vision, speech, language, and decision-making. By abstracting the complexity of AI development, Azure AI services make it easier to integrate AI capabilities into applications, offering services that are reliable, scalable, and easy to use. This chapter provides an overview of the services that will be integrated in a .NET MAUI sample solution in the next chapters.

The role of AI in mobile applications

AI is no longer a futuristic concept; it has become an essential component of modern mobile applications. Businesses and developers are leveraging AI to:

- Improve user interaction through natural language processing (NLP) and speech recognition.
- Automate repetitive tasks, such as extracting information from documents.
- Enhance user engagement with personalized recommendations.
- Enable real-time image and text analysis for various use cases.

By integrating AI into mobile applications, developers can create more dynamic, context-aware apps that understand user needs and respond intelligently.

Overview of Azure AI services

Azure AI services include a wide range of AI capabilities integrated into the Microsoft Azure cloud platform, as well as Microsoft 365 and other products. The goal of these services is to democratize AI, making advanced tools available to enterprises, developers, and users regardless of their technical expertise. Microsoft's AI solutions are built on Azure, its cloud computing platform, leveraging deep learning, machine learning, and cognitive services to drive automation, insights, and enhanced productivity. These services are designed to address diverse needs, from NLP and computer vision to conversational AI, data analytics, and custom AI model development. They target different industries, such as healthcare, finance, retail, and manufacturing, offering solutions that can integrate with existing business processes. The Azure AI offering is very large, and not all of the services can be integrated with the .NET MAUI mobile app built in this ebook. For this reason, the next sections provide an introduction to the Azure AI services that will be integrated with the sample app built in the later chapters. The full list of AI services is available on the Microsoft Azure [website](#). Also, you can refer to the [Azure AI Services Succinctly](#) ebook for a more thorough discussion.

The Microsoft Responsible AI Standard and principles

Artificial intelligence is extremely powerful, and it can help solve business problems in a much faster way. The applications of AI can target different critical sectors, not only in people's daily lives, but also in most countries' public services. In fact, AI is already present in many processes in healthcare, military and defense, transportation, politics, and so on. On the other hand, democratized AI is also available to people who do not have good intentions. A common example is the use of generative AI to create completely fake content and news to influence the masses. For this reason, Microsoft has developed the Responsible AI Standard and a set of AI principles for companies and individuals using AI in their applications to follow. You can find the details of these principles on the official [webpage](#), but the summary is:

- Use AI responsibly in your applications.
- Do not use AI in a way that can violate laws and people's privacy.

When setting up certain Azure AI services in the Azure Portal, you will be asked to review and acknowledge a Responsible AI Notice so that you take responsibility for your use of Microsoft AI services.

Azure AI Vision

[Azure AI Vision](#) allows developers to integrate advanced computer vision capabilities into their applications. Leveraging state-of-the-art machine learning models, it enables the analysis and understanding of visual data, such as images, videos, and spatial information. With Azure AI Vision, businesses can automate tasks that require visual data processing, such as extracting text from images, identifying objects, or analyzing movement in physical spaces. Azure AI Vision is cloud-based, which means that businesses can scale and manage vision-related workloads without the need for on-premises infrastructure. The service supports a wide range of industries, from retail and manufacturing to healthcare, providing AI-powered visual analytics solutions that help automate and enhance business operations. Some of the primary features include:

- **Image analysis:** The image analysis service provides a set of functionalities to analyze images and extract valuable insights. Using advanced algorithms, it detects objects, people, and scenes, categorizes images into thousands of predefined tags, and provides a summary of the image content in the form of a description. It also supports image moderation to detect explicit or unwanted content. This service can handle various image formats, and it provides analysis results in JSON format, making it easy to integrate with different types of applications.
- **Facial recognition:** The facial recognition service identifies and analyzes human faces in images or video feeds. It allows developers to detect and compare faces, determine facial attributes such as age and gender, and even recognize emotions. It is commonly used in security and identity verification systems. Additionally, it supports creating face databases to recognize individuals and track their interactions over time. Privacy and security are key concerns, and Azure AI Vision offers encryption and compliance features to meet legal requirements.

- **Spatial analysis:** This service enables developers to derive insights from real-time video feeds. It uses AI models to detect and track movement in predefined spaces, identify patterns, and generate actionable insights. Common use cases include analyzing customer behavior in retail environments, monitoring safety protocols in workplaces, and ensuring social distancing in public areas. Spatial analysis leverages Azure's scalable architecture to handle large amounts of video data and can be integrated with IoT devices for enhanced automation.
- **Optical character recognition (OCR):** Azure's OCR capability transforms images of text and PDF documents into machine-readable formats. It supports multiple languages and a wide range of image formats. OCR is commonly used in scenarios like document scanning, form data extraction, and handwritten-note processing. The service can identify text in various fonts and styles and convert it into structured data that can be processed further or stored for future use.

As you can imagine, intelligent applications built on top of Azure AI Vision can help companies automate and improve their processes dramatically. The aforementioned services are divided into two main groups: vision, which includes image analysis, spatial analysis, and OCR; and Face API, under which facial recognition is categorized. Both go under the umbrella of the AI Vision services. It is also possible to create custom models via the AI Custom Vision service, but this is a topic too vast to cover in this ebook. If you want to create custom vision models, refer to the [official documentation](#).

Exclusions and limitations

The Azure AI Vision services are extremely powerful, and for this reason they are strictly regulated. For example, the Face service can detect biometric information and help identify people and their physical characteristics. As you can imagine, this can lead to enormous risks if the collected data is obtained, stored, and retrieved without the appropriate permissions or outside of regulations specified by national and international laws. Based on this, Microsoft decided to open the Face API only to customers and organizations that submit a registration form and that comply with specific legal requirements. All the necessary information can be found on the [Limited Access to Face API page](#). Consequently, this ebook will only describe how to set up an application and how to write the necessary code to work with the Face API, but no image will be analyzed, and no screenshot will be provided.



Note: For some Azure regions, Microsoft allows age estimation and gender detection with the limited access features of the Face API without registration.

Spatial analysis involves analyzing real-time videos via IP cameras and requires the presence of multiple people in the video to provide a result. Therefore, only a description of the service and the related .NET objects will be provided.

The sample application built throughout this ebook will utilize Azure AI Vision for image analysis, but not for facial recognition.

Azure AI Language

[Azure AI Language](#) specializes in processing and understanding human language, which makes it an essential tool for businesses seeking to extract actionable insights from large amounts of textual data. As companies accumulate vast amounts of unstructured text data from social media interactions, emails, surveys, and documents, the challenge becomes turning this data into useful information. Azure AI Language allows businesses to automate and scale the analysis of this data by leveraging advanced NLP models built on Microsoft's extensive research in artificial intelligence. Azure AI Language offers several functionalities that cater to different aspects of language processing:

- **Sentiment analysis:** This feature analyzes the sentiment of text data, classifying it as positive, neutral, or negative. This can be helpful in various scenarios, such as gauging customer satisfaction from feedback or analyzing public sentiment about a product.
- **Key-phrase extraction:** This feature extracts the most important phrases from a document, highlighting the key concepts or ideas within the text. This can be particularly useful for summarizing large documents or extracting main topics from customer reviews.
- **Named entity recognition (NER):** NER identifies and classifies entities within a text, such as people, organizations, locations, and dates. This is useful for data categorization and enhancing search functionalities within applications.
- **Language detection:** The service can automatically detect the language of the text input, which is helpful when dealing with multilingual data sources.
- **Text summarization:** This advanced feature distills long documents into a concise summary that captures the essence of the text.
- **Text translation:** Azure AI Language can translate text from one language to another, leveraging the same powerful models used in Microsoft Translator.

The service is integrated into the Azure ecosystem, which allows seamless connection with other Azure resources like Blob Storage, Azure AI Search (formerly Azure Cognitive Search), and Power BI, enhancing its capabilities in data-driven applications. Azure AI Language is designed to handle data privacy and security by adhering to stringent compliance standards, including GDPR and ISO/IEC certifications.



Note: Text summarization might not be available in all Azure regions. Make sure your region supports this feature when you create the AI Language service instance in the Azure Portal.

Azure AI Speech

[Azure AI Speech](#) leverages advanced machine learning models to convert speech into text, synthesize natural-sounding speech from text, and translate speech into various languages in real time. This service is designed to integrate seamlessly into various applications, providing developers with the tools they need to build voice-enabled applications, create conversational AI experiences, and enhance accessibility with speech-to-text and text-to-speech functionalities. The service supports a wide range of use cases, including voice-activated assistants, transcription services, real-time language translation, and the generation of audio content from textual data.

AI Speech offers high accuracy and is continually updated to support new languages, dialects, and scenarios, making it a versatile tool for global applications. The service is built on Microsoft's proprietary deep neural networks, ensuring that it delivers fast, reliable, and high-quality results. As with the other services, developers can use REST APIs, SDKs, and client libraries to interact with it, enabling easy integration into existing workflows and applications. Furthermore, Azure AI Speech supports customizable speech models, allowing businesses to fine-tune the service according to their specific needs, such as adapting to industry-specific terminology or enhancing recognition accuracy for certain accents. Azure AI Speech provides three primary functionalities: speech-to-text, text-to-speech, and speech translation, as described in the following sections.

Speech-to-text

This feature allows applications to convert speech into text. It supports the real-time transcription and batch processing of audio files, making it suitable for various applications, including meeting transcriptions, voice command processing, and accessibility services. The speech-to-text service is highly accurate, benefiting from continuous improvements in Microsoft's AI models. It supports multiple languages and can be customized using custom models to improve accuracy in specific contexts, such as industry jargon or regional accents.

Text-to-speech

This capability synthesizes natural-sounding speech from text, making it ideal for generating spoken content dynamically. The text-to-speech service can be used in various scenarios, including creating voice responses in chatbots, generating audiobooks, and providing vocal guidance in applications. It supports numerous languages and voices, including neural voices that provide more natural intonations and expressions. Custom voice fonts can also be created to match a brand's unique voice, ensuring consistency across various channels.

Speech translation

Azure AI Speech also offers the real-time translation of speech into another language, enabling cross-lingual communication in applications. This service can be integrated into multilingual chat applications, global conferencing tools, and customer support systems to facilitate communication between speakers of different languages. It supports a wide range of language pairs and delivers translations with high accuracy and low latency, making it suitable for real-time communication scenarios.

Additional features

In addition to these core features, Azure AI Speech offers features such as profanity filtering, punctuation addition, and the ability to identify multiple speakers in a conversation (speaker diarization). The service also integrates with other Azure services, such as Azure AI Language, for enhanced functionality like sentiment analysis or key-phrases extraction from transcribed text. These additional features will not be covered in this ebook, but you can find more information about them in the [documentation](#).

Azure OpenAI Service

Azure OpenAI Service is a cloud-based platform that provides REST API access to OpenAI's advanced language models, including the GPT family of models and DALL-E. It enables developers to integrate generative AI capabilities into their applications for tasks such as content generation, summarization, semantic search, and natural-language-to-code translation. Azure OpenAI Service is designed to be adaptable to various use cases, including customer support automation, personalized marketing, and workflow optimization. Key features include:

- **Model deployment:** Users can deploy models like GPT-4 and GPT-3.5 through Azure's portal or SDKs and tailor them to specific business needs.
- **Fine-tuning:** Models can be fine-tuned for specialized tasks, ensuring high relevance and accuracy.
- **Security and compliance:** Azure OpenAI Service integrates enterprise-grade security measures, including Azure Active Directory and encryption, ensuring data protection.
- **Scalability:** The service supports flexible deployment options, allowing businesses to scale resources based on workload demands.
- **Responsible AI:** Microsoft incorporates responsible AI principles, including content filtering and guidance to mitigate risks associated with generative AI.

Azure OpenAI Service is particularly suited for organizations looking to leverage AI for intelligent automation, data-driven insights, and enhanced user experiences.

Conversational apps with ChatGPT services

ChatGPT, developed by OpenAI, is a conversational AI service powered by generative pretrained transformer (GPT) models. It is designed to simulate human conversations and assist users in tasks ranging from answering questions to composing text, code, and brainstorming ideas. ChatGPT is available in various tiers, including free and subscription-based plans, offering access to advanced models like GPT-4 and DALL-E for image generation. Key features include:

- **NLP:** ChatGPT excels in generating human-like text and understanding human-written input, making it ideal for customer service, education, and creative writing.

- Customization: Users can refine responses by specifying the desired length, format, style, and level of detail.
- Multimodal capabilities: Advanced models support text, image, and audio inputs, enabling diverse applications such as digital art creation and real-time transcription.
- Integration: ChatGPT can be embedded into applications, websites, and workflows to enhance interactivity and productivity.
- Accessibility: The service is available on multiple platforms, including web, mobile apps, and APIs, ensuring ease of use.

ChatGPT is widely used for personal assistance, content creation, and problem-solving, making it a versatile tool for individuals and businesses alike. The combination of these AI technologies empowers users with innovative solutions, whether it is building intelligent applications with Azure OpenAI or engaging in dynamic conversations with ChatGPT.

Azure AI Search

[Azure AI Search](#) is a cloud-based search service that integrates AI capabilities to offer advanced search features. It provides an enriched search experience by combining traditional full-text search with advanced NLP, cognitive skills, and machine learning models. Azure AI Search can be used to develop custom search solutions that offer users accurate, fast, and meaningful search results across various data types, such as structured data, unstructured text, and media files. Azure AI Search is particularly useful for organizations that need to search large datasets efficiently. It can be employed in scenarios such as enterprise search, e-commerce, catalog search, document discovery, and knowledge management.

Azure AI Search is built on modern search technology, powered by Elasticsearch, and enhanced with Microsoft's proprietary AI services to make sense of complex datasets. It is a fully managed service, which means it abstracts the complexities of setting up, maintaining, and scaling search infrastructure. The core functionality of Azure AI Search revolves around indexing, querying, and refining data to make it searchable in a way that aligns with user intent.

Azure AI Search includes several subservices and components that work together to build a sophisticated search solution that can handle complex search scenarios with ease:

- Indexers: Indexers automate the process of ingesting data from various sources into the search index. Indexers can connect to multiple Azure data sources, like SQL Database, Cosmos DB, and Blob Storage, or to a custom data source. You can schedule indexers to update the search index periodically as the underlying data changes. During the indexing process, AI enrichment can be applied via cognitive skills.
- Cognitive skills: Cognitive skills allow you to enrich your data using various AI services. These skills range from text analysis, image recognition, and NER to more advanced tasks such as document translation. These skills are applied to documents or text as they are indexed, transforming raw content into searchable, insightful information.

- **Synonyms and analyzers:** Synonym maps allow you to define lists of equivalent terms for search queries. For example, "car" and "automobile" can be treated as synonyms. Analyzers determine how data is tokenized and processed during indexing and querying. They support different languages and customizations, helping improve search relevance.
- **Search units:** Search units are the scalable compute resources of Azure AI Search. Based on your workload and performance requirements, you can scale search units up or down. Search units handle query processing, indexing, and storage operations. They ensure that the service can accommodate large volumes of data and provide rapid query responses.

Azure AI Search also supports enterprise-grade security with Azure Active Directory integration, role-based access control, and encryption of data at rest and in transit.

Azure AI Document Intelligence

[Azure AI Document Intelligence](#) is designed to extract text, key-value pairs, tables, and other structured information intelligently from documents like invoices, receipts, contracts, or any form where data is entered. Azure AI Document Intelligence leverages machine learning models to identify and structure data, minimizing the need for manual intervention. It offers capabilities to customize and train models specific to your business documents, allowing for improved accuracy and adaptability. The service is especially useful in scenarios involving high volumes of documents, where manual data entry can be both time-consuming and error-prone. Azure AI Document Intelligence provides prebuilt and custom models to extract information from structured, semi-structured, and unstructured documents. The service supports multiple languages and formats, including PDFs, scanned images, and photographs. More specifically, it offers:

- **Prebuilt models:** Prebuilt models are designed to recognize and extract information from common document types, such as receipts, invoices, and business cards. These models are trained by Microsoft and can be used immediately without the need for further training.
- **Custom models:** Custom models allow users to train AI models tailored to their specific document layouts. Using labeled data, users can train models to recognize key-value pairs and other structured information in documents that do not conform to standard templates.
- **General document model:** This subservice extracts content and layout information from any document without the need for training or labeling. It can handle a wide variety of documents and extract text, tables, and other data.
- **Layout model:** This is a base model used to extract document layout information, including text, tables, selection marks, and the overall structure. It is often used as a preprocessing step before processing with other models.

Azure AI Document Intelligence exposes the aforementioned models through a number of services, described in the next section.

Azure AI Document Intelligence services

In terms of AI services that you can use in your applications, Azure AI Document Intelligence provides the following:

- **Form Recognizer:** Form Recognizer is the core component of Azure AI Document Intelligence. It allows the extraction of information from forms, invoices, and other documents through prebuilt or custom models. It can recognize fields, checkboxes, signatures, and tables from scanned or digital documents.
- **Invoice Recognizer:** This is a specialized prebuilt model that extracts information from invoices, including fields like invoice number, date, total amount, and due date. It helps automate accounts payable processing by accurately extracting and categorizing invoice data.
- **Receipt Recognizer:** The receipt recognizer model extracts information from sales receipts, such as the transaction date, total amount, items purchased, and merchant details. This model is useful in automating expense reporting and tracking.
- **Business Card Recognizer:** This prebuilt model extracts contact details from business cards, including name, phone number, email address, and company name. It is particularly useful for customer relationship management systems to automate the entry of new contacts.
- **ID Document Recognizer:** This model is used to extract key fields from identification documents, such as driver's licenses, passports, and ID cards. The fields it extracts typically include name, date of birth, and document number.

Each of these subservices is built to handle specific document types and extract information in a way that can be easily integrated into other business processes and workflows.

Programming with Azure AI services

Azure AI services are exposed through RESTful APIs, so it is possible to send web requests and receive responses over the HTTPS protocol. However, there are many convenient client libraries on the market, developed to make it easier to work with Azure resources from various development platforms and programming languages. Developers working with Microsoft technologies can leverage the Azure SDK, a collection of .NET libraries available via NuGet for interacting with all the Azure cloud services via .NET and managed code. In this ebook, you will work with C#, .NET, and the Azure SDK to interact with the Microsoft Azure AI services. The appropriate libraries from the Azure SDK will be mentioned as required. In the next chapter, you will instead set up your development environment to have all the necessary common .NET tools.



Note: In this ebook, Microsoft Visual Studio 2022 is the integrated development environment of choice. It is also possible to use Visual Studio Code and the official .NET MAUI extension to develop mobile applications, but these are not covered here.

The AI-powered travel companion app

To demonstrate the real-world application of these AI services, this ebook will guide you through the development of an AI-powered travel companion app. This app will enhance the travel experience by:

- Translating text using Text Analytics and image analysis via Computer Vision.
- Providing voice-based commands and assistance through Speech Services.
- Suggesting nearby attractions and personalized recommendations using Azure AI Search.
- Implementing a dynamic assistant via a ChatGPT-based chatbot.
- Allowing users to scan and extract important information from travel documents with Document Intelligence.

By the end of this ebook, you will have built a fully functional mobile application that seamlessly integrates AI-driven features, helping travelers navigate foreign environments more efficiently.



Note: *.NET MAUI allows you to target multiple operating systems from a single, shared codebase, including Android, iOS, Windows 10 and higher, macOS, and Tizen. However, since the target of this ebook is mobile applications, the main focus will be Android and iOS.*

Chapter summary

Azure AI services provide a robust suite of AI-driven tools that enable developers to integrate sophisticated intelligence into their applications, including mobile apps built with .NET MAUI. By leveraging these services, businesses can enhance their capabilities in global communication, intelligent search, automated data processing, natural language understanding, computer vision, and content moderation. Before integrating these services into your mobile apps, you need to set up your development environment. This is discussed in the next chapter.

Chapter 2 Setting Up the Development Environment

To build the sample AI-powered travel companion app, you first need to set up the development environment and configure an Azure subscription. This chapter will guide you through installing the necessary tools and creating an Azure account, while later chapters will cover setting up individual services.

Registering for an Azure subscription

To set up a development environment to build mobile apps with .NET MAUI that leverage Microsoft Azure AI services, the first requirement is a Microsoft Azure subscription. If you do not have one, you can register for a [30-day trial](#). You will need to register using a Microsoft account. Complete the registration process by providing your details, verifying your identity with a credit card (you will not be charged unless you upgrade your subscription), and accepting the terms. Once you register, you will receive \$200 in free credit, which can be used for various Azure services, including Azure AI services, and is perfect for walking through the examples described in this ebook.

Locating Azure AI services

Once you have registered for an Azure subscription, you need to log in to the [Azure Portal](#), which is the place where you manage all the available Azure services, not just AI services. The main page of the Azure Portal is a dashboard that provides shortcuts to the most popular Azure services. You will find a shortcut called AI services. If you click on this shortcut, you will access a page that contains the full list of available Azure AI services, as shown in Figure 1.

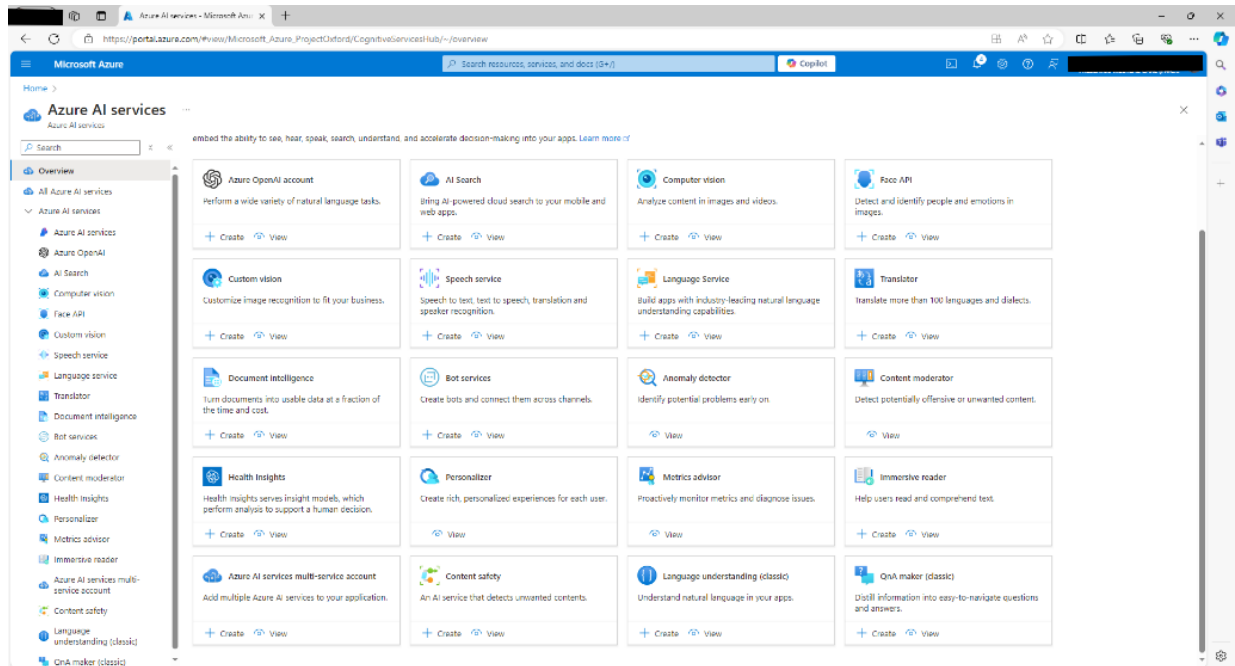


Figure 1: Full list of Azure AI services

In the next chapters, you will be asked to create new service instances for the AI service targeted within each chapter. Keep Figure 1 as a reference to find the specific service quickly and remember that you can access this page by clicking AI services in the Azure Portal home page.

Creating a resource group

As the name implies, an Azure resource group is a container for cloud services. It provisions all the resources that are common to the services it contains. A resource group is also needed to complete the code examples discussed in this ebook. To create a resource group, once you have logged in to the [Azure Portal](#), type **Resource Group** in the search bar and click the **Resource Groups** item that appears. This will open the Resource groups page shown in Figure 2.

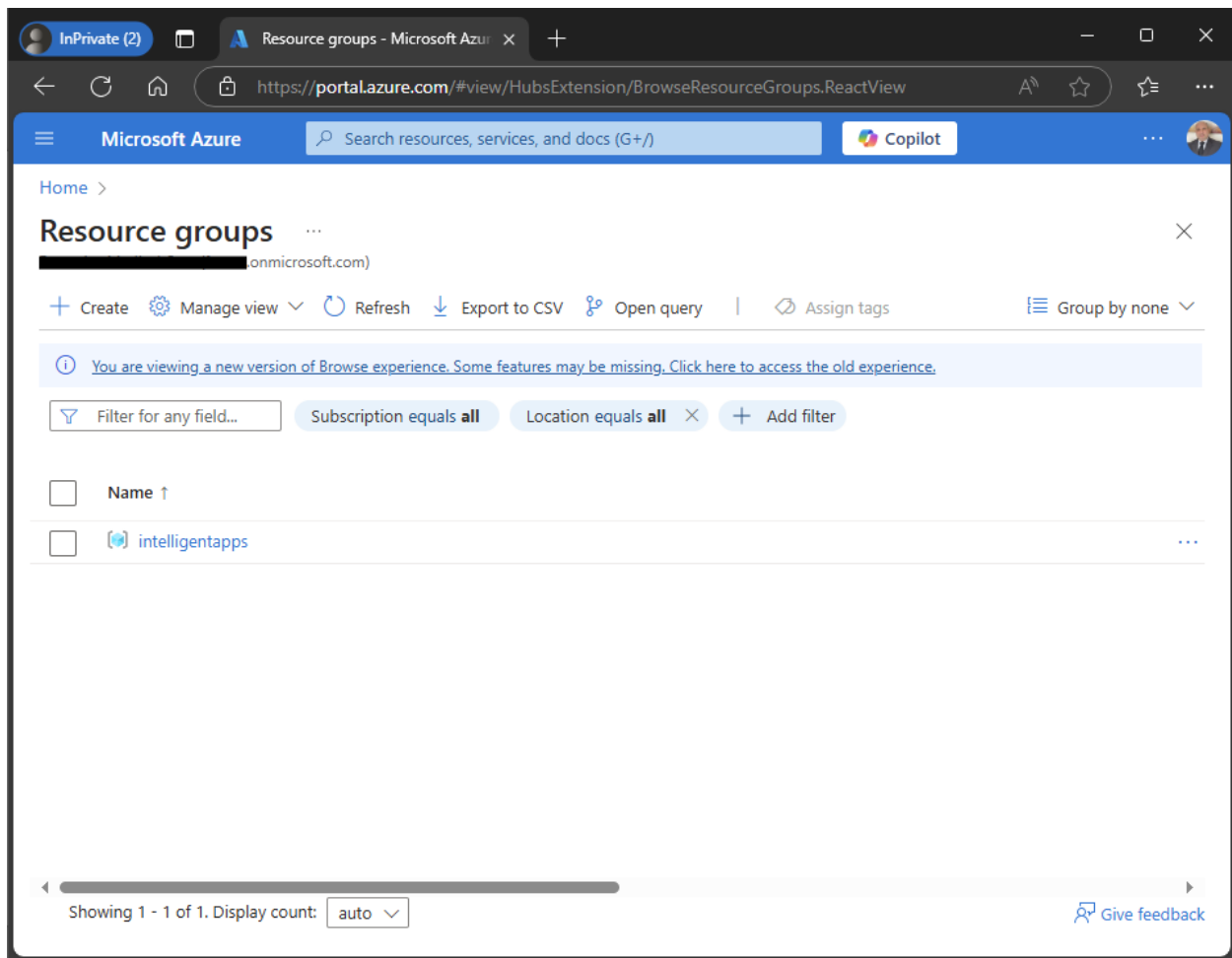


Figure 2: Locating resource groups

Click **Create**. In the Create a resource group page, select your Azure subscription, and then enter an all-lowercase name for the resource group, such as intelligentapps.



Note: Every time you specify a name for an Azure resource, it must be lowercase.

Figure 3 shows the completed resource creation form.

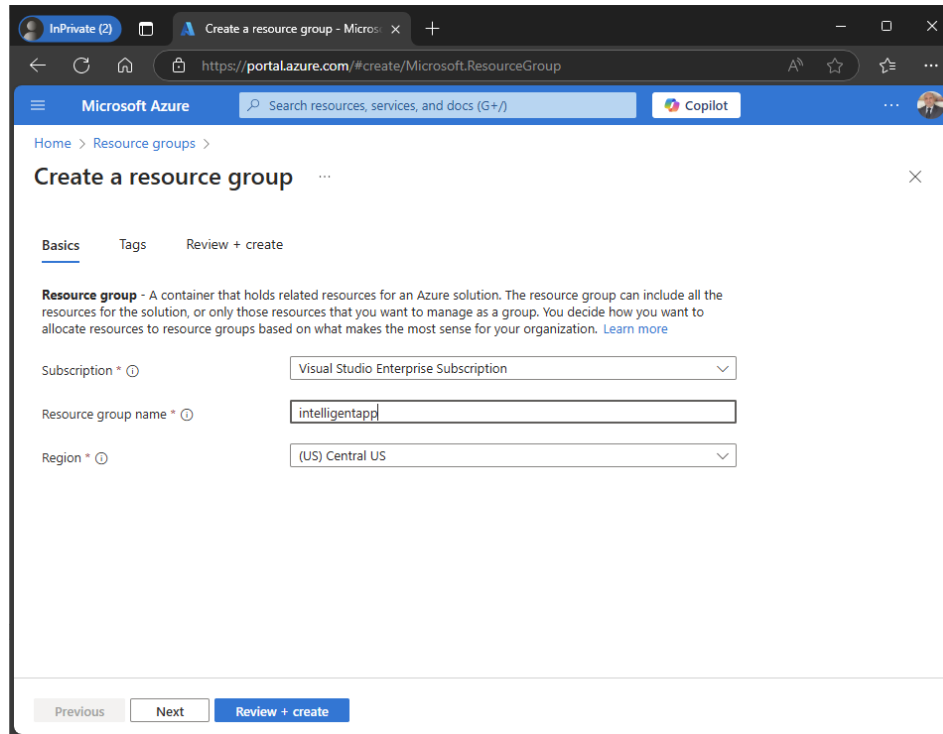


Figure 3: Creating a resource group

In the **Region** dropdown, select the region closest to your location. When ready, click **Review + create**, then **Create**, and wait for the resource group to be deployed.



Note: Remember to delete the Azure resources that you no longer use to avoid extra costs or credit consumption.

Installing .NET workloads

Assuming that you are working with Microsoft Visual Studio 2022, you need to ensure that .NET 9 and the .NET MAUI libraries are installed on your machine. To do so, launch the **Visual Studio Installer** and click **Modify**. Make sure the **.NET Multi-platform App UI development** workload is selected (see Figure 4).

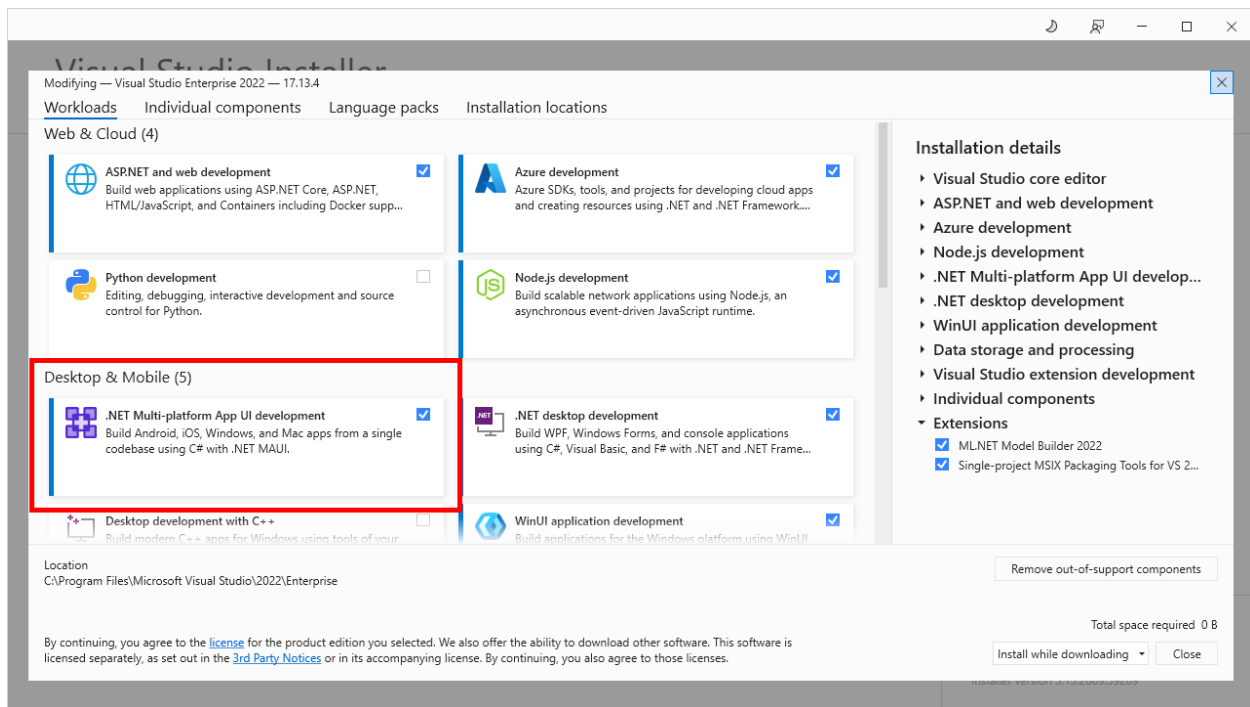


Figure 4: Selecting the .NET MAUI workload

Select the **Individual components** tab at the top and ensure that **.NET 9.0 Runtime** is selected, as shown in Figure 5.

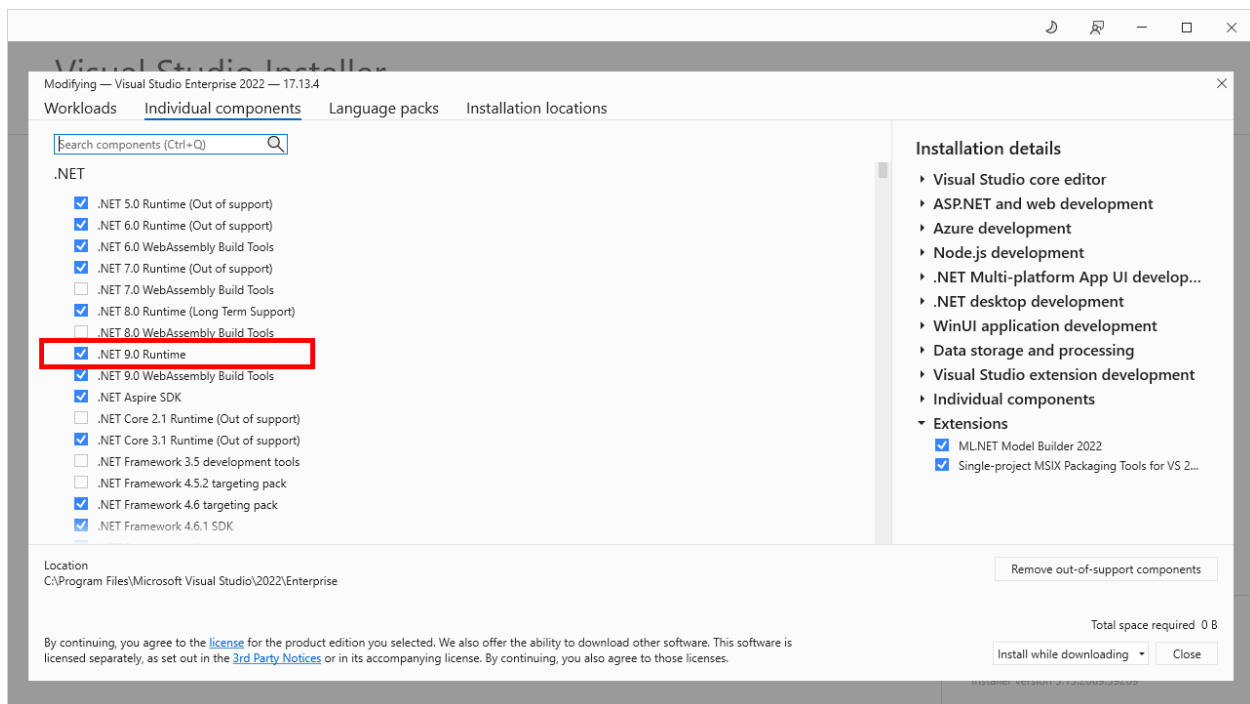


Figure 5: Selecting the .NET 9 runtime

This ensures that you have the latest and most updated versions of the development technology. When ready, click **Close** and wait for any necessary updates to be installed.



Note: If you plan to target iOS and macOS, you need a Mac computer and you need to follow the steps described in the [official documentation](#).

Creating a .NET MAUI solution

The last step in setting up the development environment is creating a .NET MAUI solution that you will extend in the next chapters with AI capabilities. In Visual Studio 2022, select **File > New Project**. In the new project dialog, locate the **.NET MAUI App** template (Figure 6), select it, and then click **Next**.

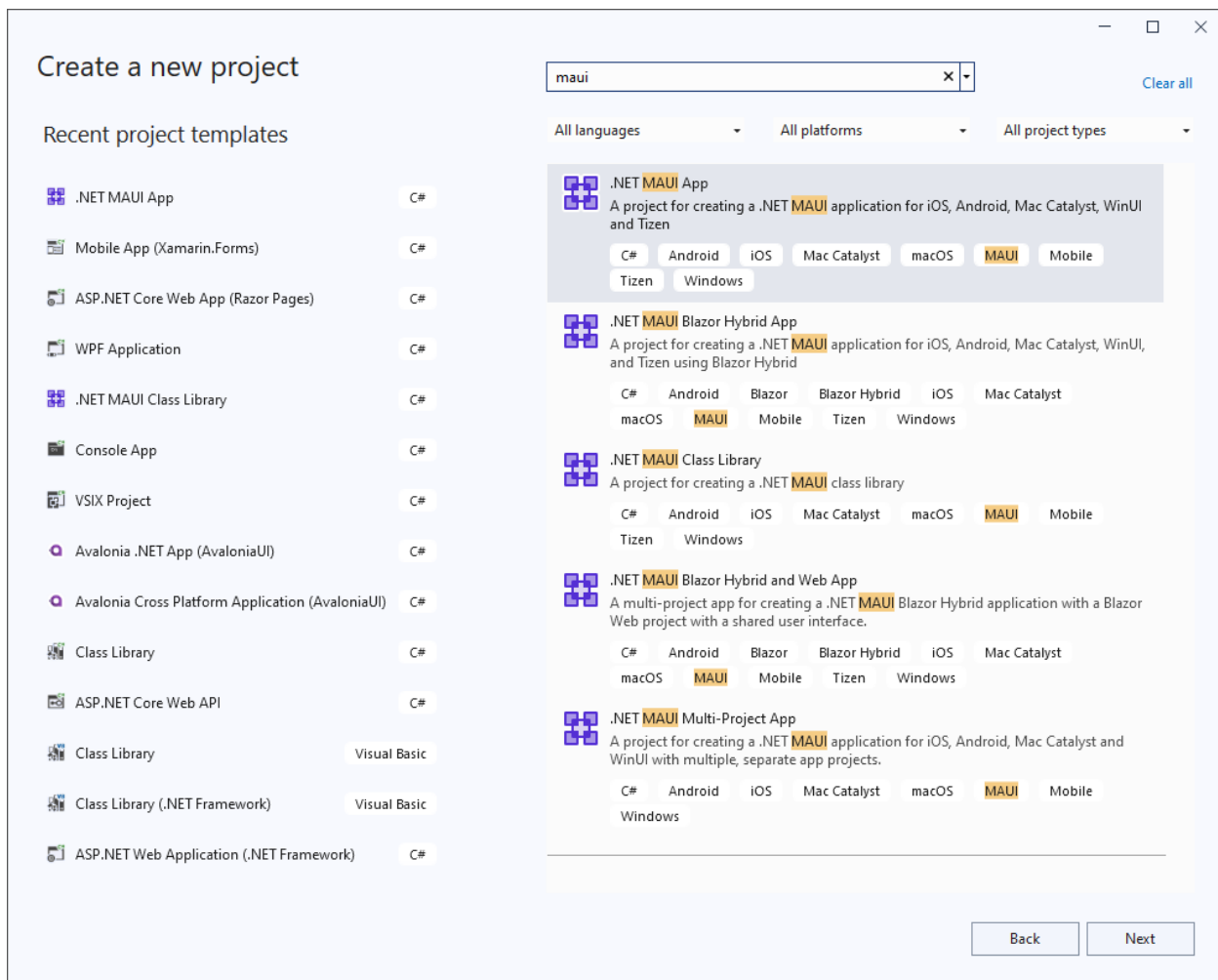
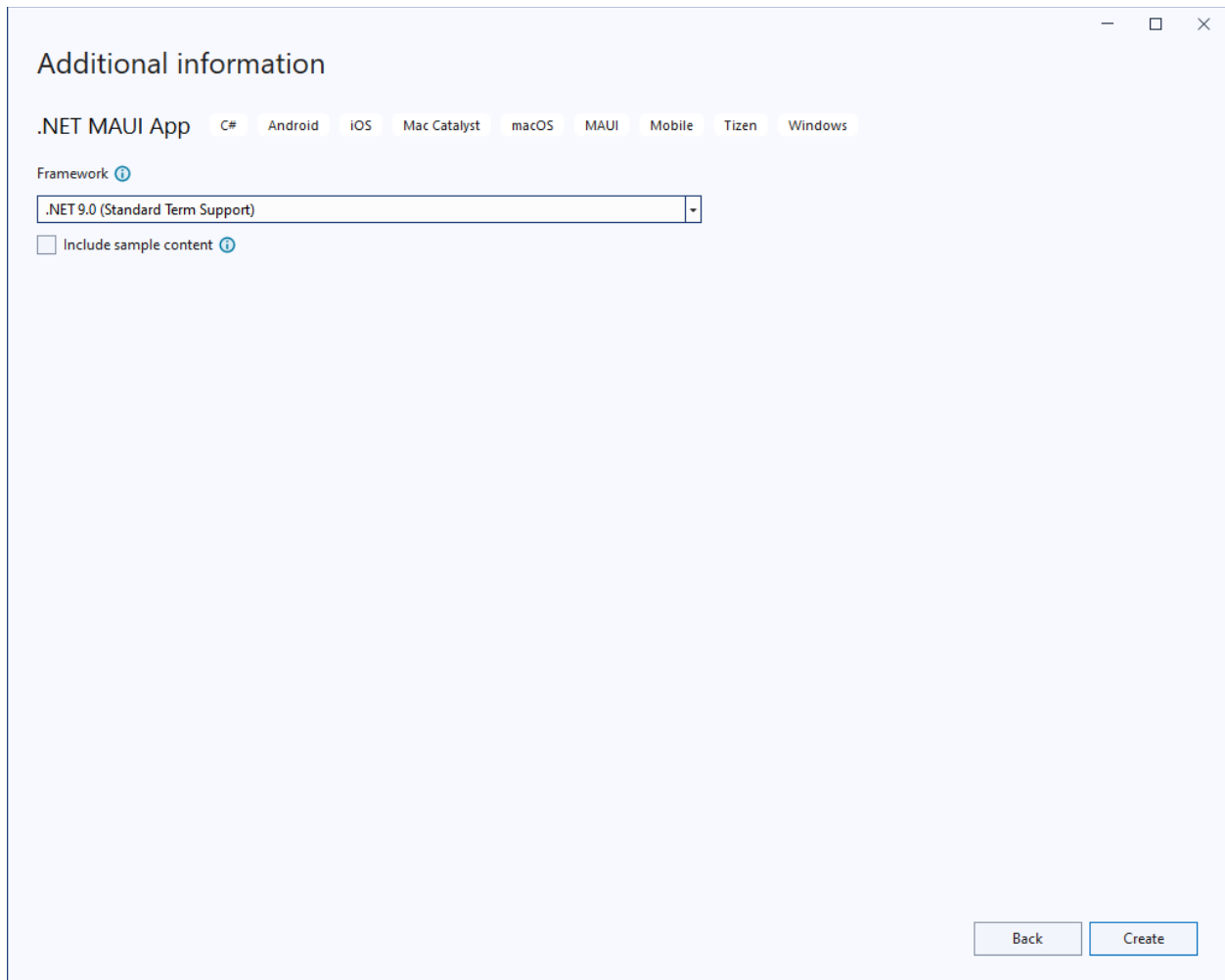


Figure 6: Creating a .NET MAUI App project

In the next screen, specify a project name (use **TravelCompanion** for consistency with the examples in this ebook) and the location, and then click **Next**. In the third and final screen, select **.NET 9 (Standard Term Support)** as the target framework and click **Create**. You can still select .NET 8 as the target framework if you prefer working on a long-term support release.



The screenshot shows a window titled "Additional information" with a standard Windows title bar (minimize, maximize, close buttons). Inside the window, there is a horizontal tab bar with the following tabs: ".NET MAUI App", "C#", "Android", "iOS", "Mac Catalyst", "macOS", "MAUI", "Mobile", "Tizen", and "Windows". The ".NET MAUI App" tab is selected. Below the tabs, the text "Framework ⓘ" is displayed. Underneath is a dropdown menu currently showing ".NET 9.0 (Standard Term Support)". Below the dropdown is a checkbox labeled "Include sample content ⓘ", which is currently unchecked. At the bottom right of the window, there are two buttons: "Back" and "Create".

Figure 7: Selecting the target framework

After a few seconds, the new solution will be available. Figure 8 shows how it appears in the Solution Explorer.

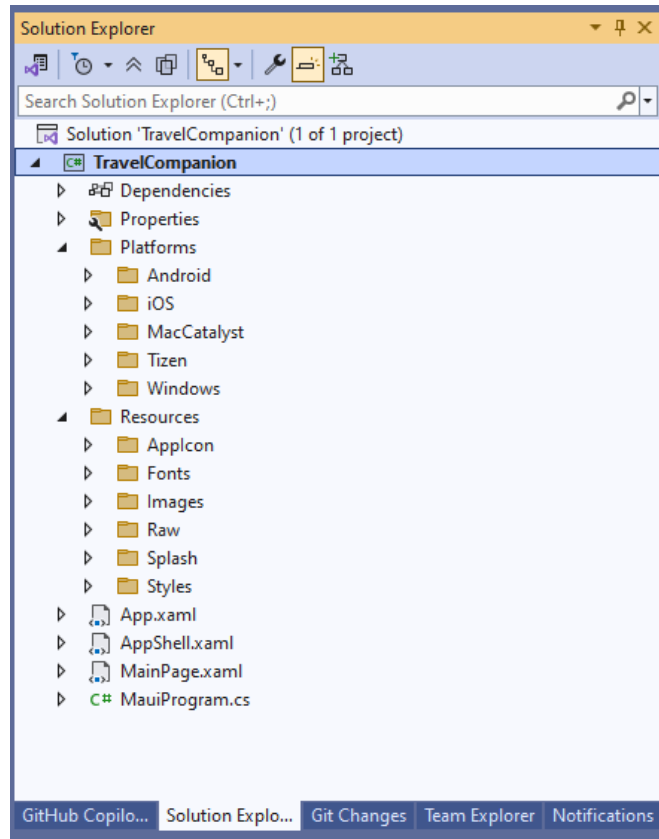


Figure 8: The sample solution in Visual Studio 2022



Tip: The companion solution for this ebook contains icons for the navigation bar that you can use if you prefer to write the sample app from scratch.

Chapter summary

In this chapter, you have seen how to configure the development environment to work with .NET MAUI in Visual Studio 2022 and the Microsoft Azure AI services. On the Azure side, you have seen how to access the Azure Portal, where to locate the Azure AI services, and how to create resource groups. On the desktop side, you have seen how to set up the .NET workloads for Visual Studio 2022, and you have created a new .NET MAUI solution that you will augment in the next chapters. Now you have everything you need to start building intelligent applications.

Chapter 3 Implementing Image Analysis

The Computer Vision API allows you to analyze images, identify objects in them, and extract text from them using OCR. In this chapter, you will set up the Azure AI Vision service for a .NET MAUI app, implement image analysis capabilities, and create a user interface for selecting and analyzing images to perform OCR and generate descriptions about them.

Setting up Azure AI Vision

Before diving into the code, you need to set up the Azure AI Vision service in the Azure Portal:

1. Go to the [Azure Portal](#) and sign in with your Azure account credentials.
2. Click **Azure AI services**. Figure 9 shows where you can find both the Computer Vision and Face API services. Remember that the Face API will not be used in this ebook.
3. Click **Create** in the Computer vision card.

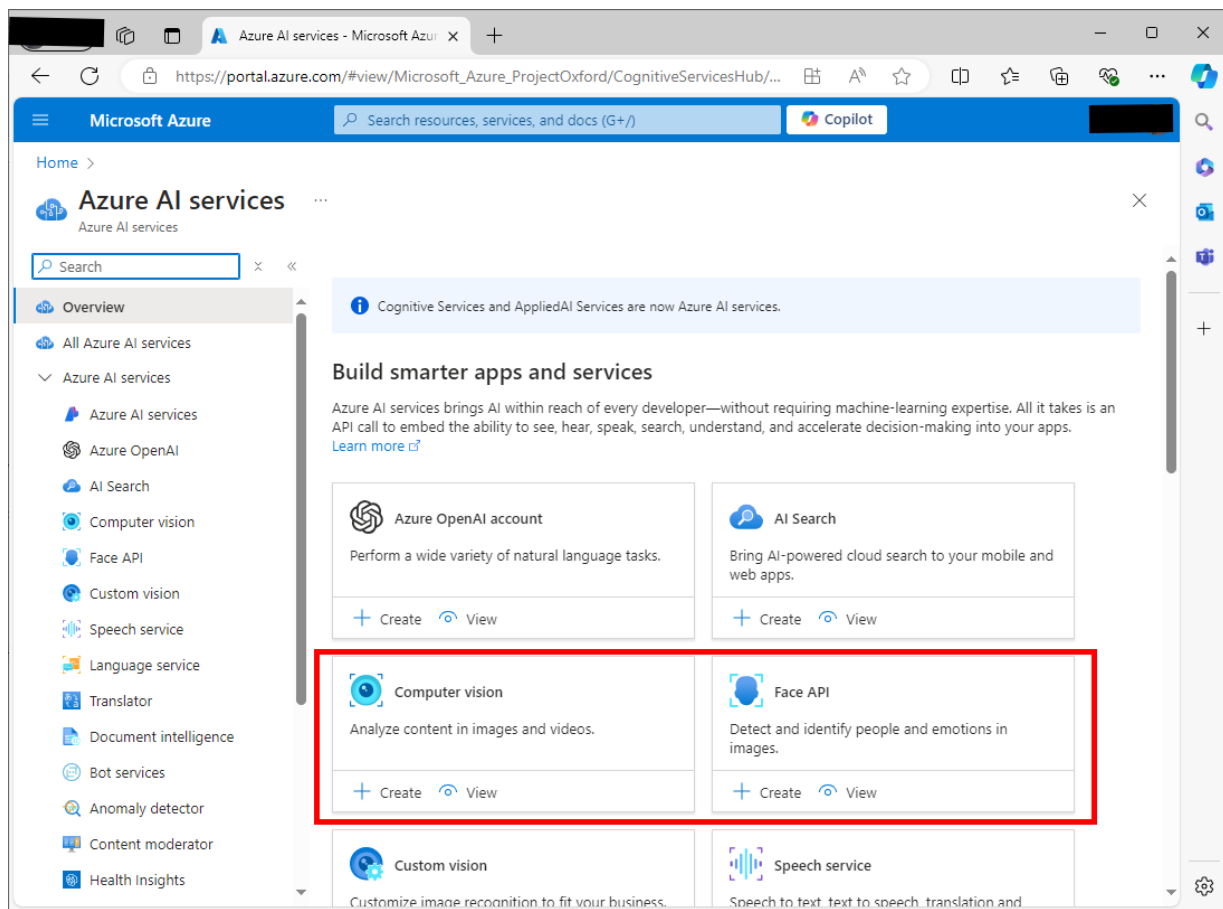


Figure 9: Locating the Computer Vision services

4. In the Create Computer Vision page, choose your Azure subscription in the **Subscription** dropdown and select the resource group you created previously in the **Resource group** dropdown.
5. For **Region**, choose the region closest to your location and enter a unique name for your Computer Vision instance in the **Name** field. For this example, I use the name vision-succinctly (see Figure 10).
6. In the **Pricing tier** dropdown, select **Free F0**.
7. Carefully read the responsible usage of AI notice and then select the acknowledgement checkbox.
8. Click **Review + Create** and finally, click **Create**.

The screenshot shows the 'Create Computer Vision' page in the Microsoft Azure portal. The page has a blue header with the Microsoft Azure logo, a search bar, and a Copilot button. Below the header, there's a breadcrumb trail: 'Home > Azure AI services >'. The main title is 'Create Computer Vision'. There are tabs for 'Basics', 'Network', 'Identity', 'Tags', and 'Review + create'. The 'Basics' tab is selected. Below the tabs, there's a description of Computer Vision: 'Boost content discoverability, accelerate text extraction, and create products that more people can use by embedding vision capabilities in your apps. Use visual data processing to label content (from objects to concepts), extract printed and handwritten text, recognize familiar subjects like brands and landmarks, and moderate content. No machine learning expertise is required.' There's a 'Learn more' link. Below that, there's a 'Project Details' section with two dropdowns: 'Subscription' (set to 'Visual Studio Enterprise Subscription') and 'Resource group' (set to 'intelligentapps'). There's a 'Create new' link below the 'Resource group' dropdown. Below that, there's an 'Instance Details' section with two dropdowns: 'Region' (set to 'West Europe') and 'Name' (set to 'vision-succinctly'). At the bottom, there are three buttons: 'Previous', 'Next', and 'Review + create'. The 'Review + create' button is highlighted. There's also a 'Give feedback' link at the bottom right.

Figure 10: Creating a new Computer Vision service

When the summary page appears, click **Go to resource**. The new service will appear in the list of services within the current resource group, so click on it. A webpage for the new service will appear. Here you will find a **Click here to manage keys** hyperlink, which brings you to the page shown in Figure 11 where you will be able to retrieve the API key and the service endpoint URL that you will need to access the service from C# code.

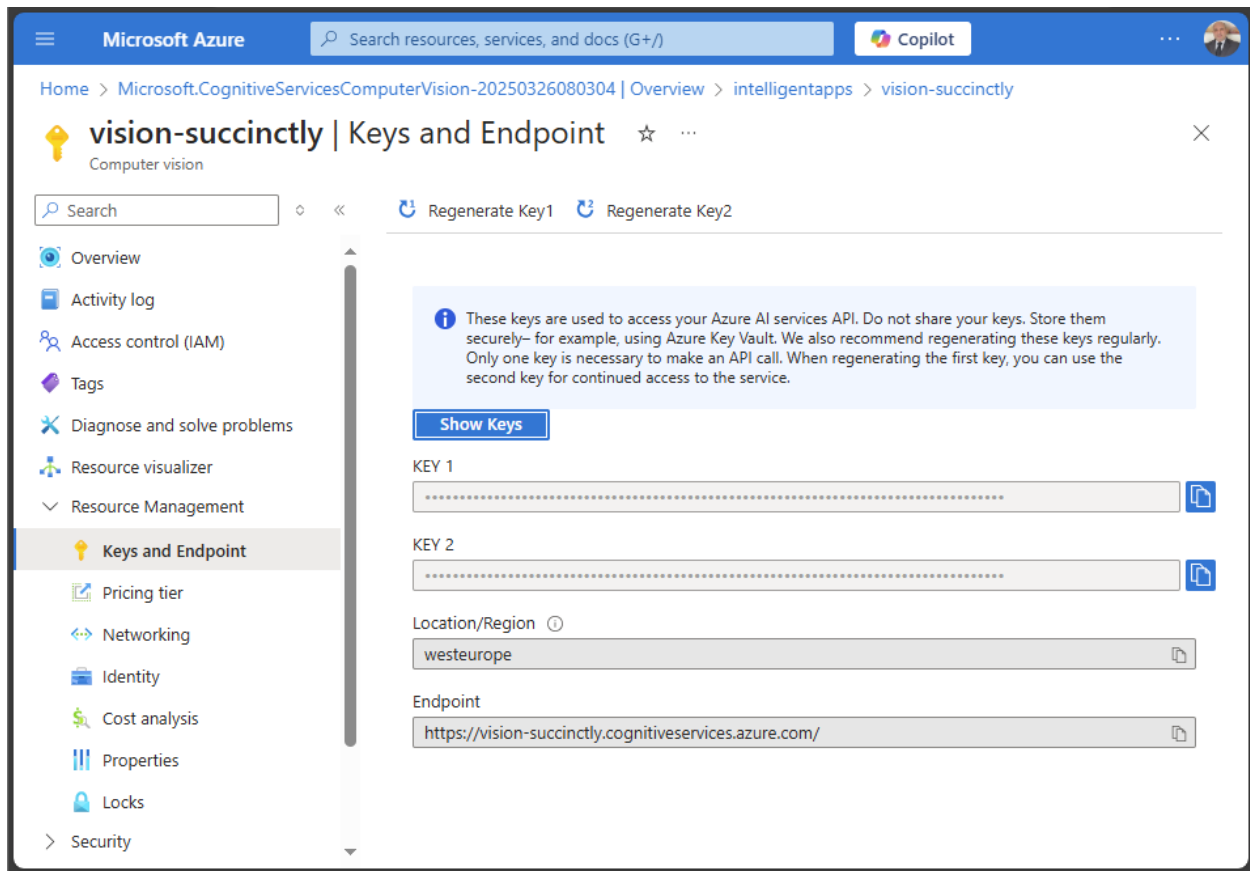


Figure 11: Retrieving API keys and service endpoint

By default, Azure generates two API keys, primary and secondary, but for development you only need one. They are hidden by default for security reasons. Click the **Copy** button next to the primary (KEY 1) API key text box and securely store the API key for later use in your application's code. For example, you could consider pasting the copied keys into a text editor, such as Notepad, and save the file locally or in the cloud.

 **Tip:** The preceding steps are common to each Azure AI service added to the sample app in this ebook, so keep them in mind.

Implementing image analysis in .NET MAUI

It is time to write code to implement image analysis in the sample application. This is composed of two steps: installing the necessary NuGet packages and writing the appropriate code.

Installing the AI Vision NuGet packages

As mentioned previously, Azure AI services can be queried via HTTP REST calls. However, Microsoft also offers client libraries for .NET that simplify the way you access services based on an object-oriented approach. With that in mind, open the **TravelCompanion** sample solution you created previously. In Solution Explorer, right-click the project name and select **Manage NuGet Packages**. When the NuGet user interface appears, search for the **Azure.AI.Vision.ImageAnalysis** library (see Figure 12) and install it.

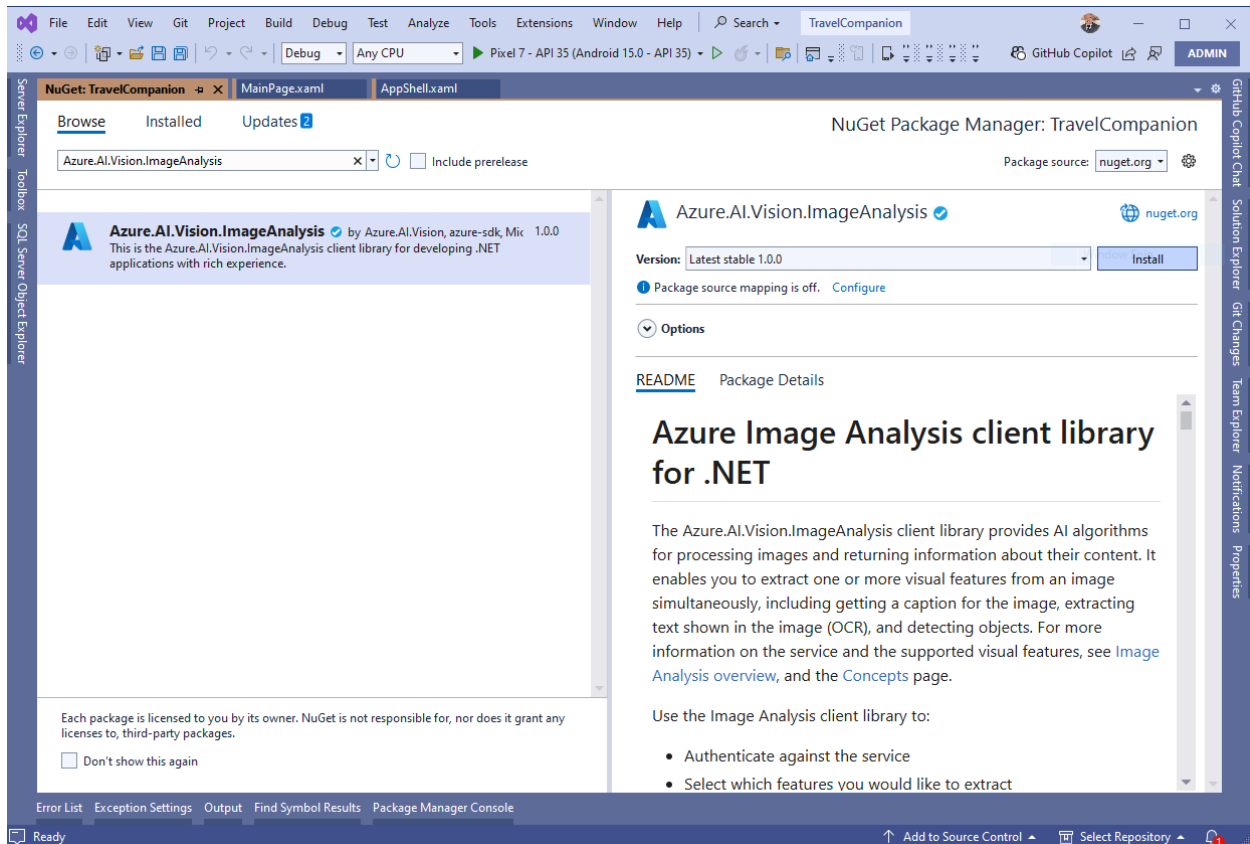


Figure 12: Installing the AI Vision client library

You will need to accept the license agreement, as usual. When the installation is complete, you can start writing code.

Specifying permissions

The sample application will need to access the photo library of the device it's deployed on so that it can select an existing picture. For this reason, you need to specify the appropriate access permissions in each platform's manifest file. For Android, double-click the **AndroidManifest.xml** file located in the **Platforms\Android** subfolder. When the manifest editor appears, select the **READ_MEDIA_IMAGES** permission as shown in Figure 13.

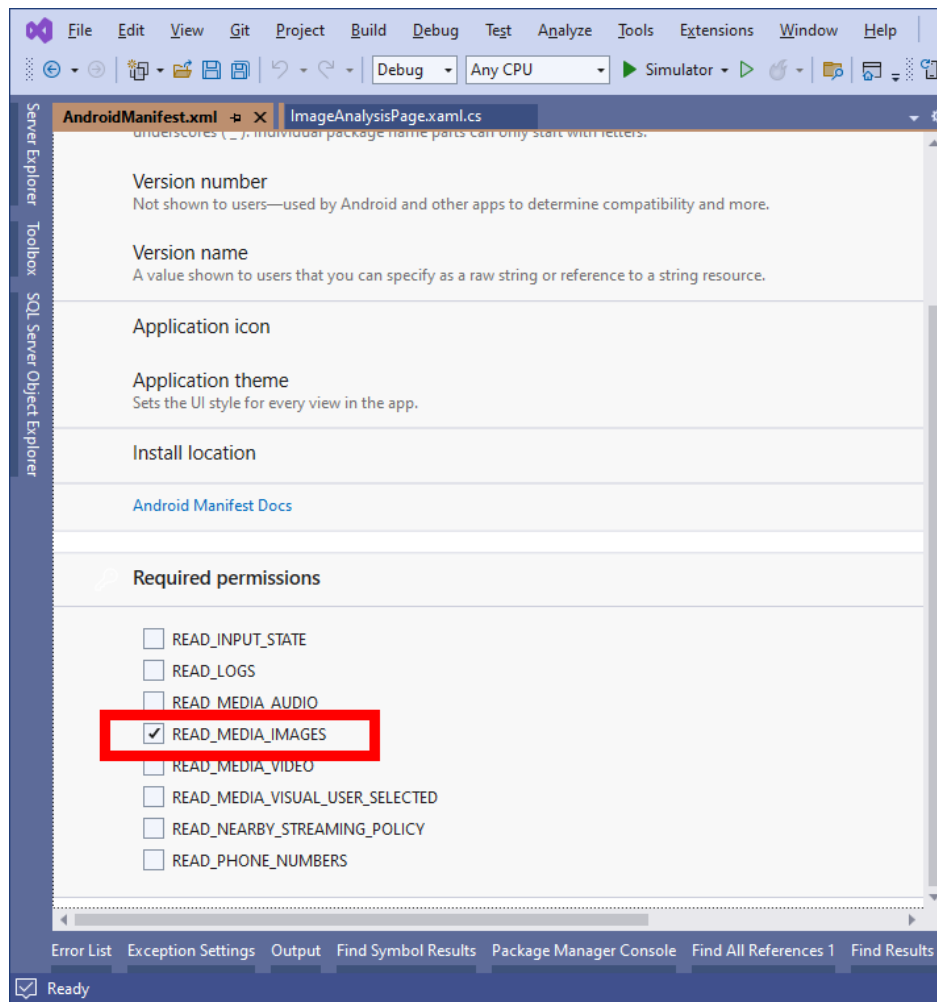


Figure 13: Assigning permissions in the Android manifest

If you plan to target older versions of Android, such as 6.0, you might want to add the **READ_EXTERNAL_STORAGE** permission.

To edit the iOS permissions, right-click the **Info.plist** file under **Platforms/iOS** and select **Open With**. In the dialog, select **XML (Text) Editor** and add the following code to the Info.plist:

```
<key>NSPhotoLibraryUsageDescription</key>
<string>App requires access to your photo library to analyze images.</string>
```

Info.plist stands for information property list. Replace the permission description with text that better fits your needs.



Tip: Visual Studio includes an Info.plist editor with a convenient user interface, but it does not allow specifying additional permissions. This is why you have to edit the Info.plist file manually.

Specifically for iOS, the application should already have built-in permissions to access the internet. If you experience problems connecting to Azure AI services, try adding the following lines to the Info.plist file:

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSExceptionDomains</key>
  <dict>
    <key>cognitiveservices.azure.com</key>
    <dict>
      <key>NSExceptionAllowsInsecureHTTPLoads</key>
      <false/>
      <key>NSIncludesSubdomains</key>
      <true/>
      <key>NSExceptionRequiresForwardSecrecy</key>
      <false/>
    </dict>
  </dict>
</dict>
```

This will ensure that the application will be able to reach the Azure AI services endpoints. Specifying permissions in the manifest is not enough; these need to be handled by the user interface, as you will see shortly in the code.

Defining the user interface

You will create a new page for each AI service, and pages will be opened through a navigation bar in the app shell. That said, you will now add a new page for the image analysis. Right-click the project name in Solution Explorer and then click **Add > New Item**. In the Add New Item window, in the left margin, click **.NET MAUI**. Then click the **.NET MAUI ContentPage (XAML)** item, as shown in Figure 14.

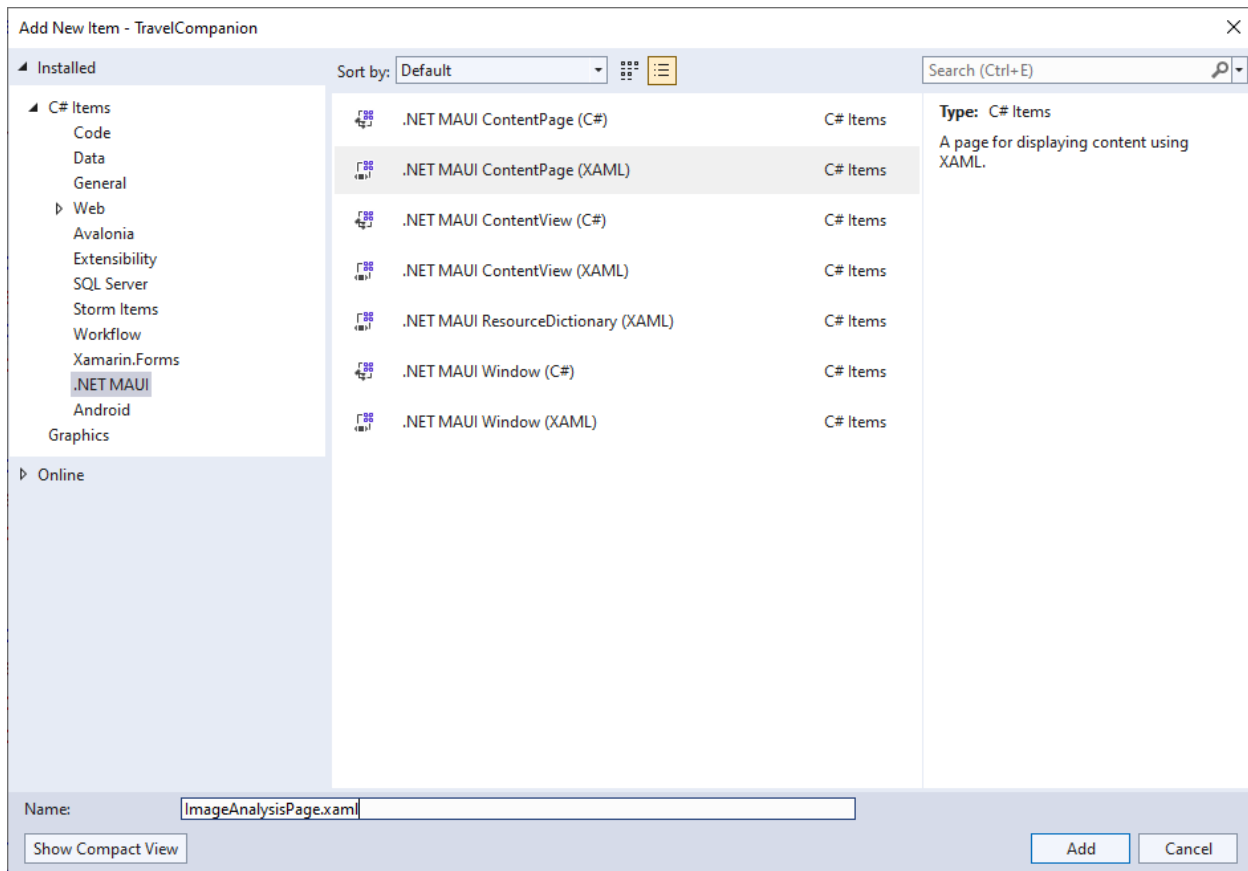


Figure 14: Adding a new ContentPage object

Assign **ImageAnalysisPage.xaml** as the page name and click **Add**. The purpose of the new page is to give users the option to pick an image from the device and analyze its content via AI Vision. There are two buttons: one for the image analysis and one for the OCR processing. The simple user interface for the new page is shown in Code Listing 1.

Code Listing 1

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="TravelCompanion.ImageAnalysisPage"
              Title="ImageAnalysisPage">
  <Grid Padding="20" VerticalOptions="Fill">
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition Height="300"/>
      <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Button Text="Pick an Image" Clicked="OnPickImageClicked"
            Margin="5" />
    <Image x:Name="SelectedImage"
```

```

        Aspect="AspectFit" Grid.Row="1" />
        <HorizontalStackLayout Grid.Row="2" HorizontalOptions="Fill">
            <Button Text="Analyze Image" Margin="5"
                Clicked="OnAnalyzeImageClicked" />
            <Button Text="OCR Recognition" Margin="5"
                Clicked="OnOcrAnalyzeImageClicked" />
        </HorizontalStackLayout>
    </Grid>
</ContentPage>

```

The explanation is very easy: a **Button** allows users to pick an image, the **Image** displays the selected picture, the second **Button** triggers image analysis, and the **Label** shows the analysis results. Finally, the third **Button** starts OCR processing for the selected image, and the result is shown inside the same **Label**.

Exception handling

Generally speaking, all the Azure AI services throw a **RequestFailedException** when the invocation to the service fails. This object also exposes the **ErrorCode** property, of type nullable **string**, which contains the details of the failure. In all the examples in this ebook, you will see how the code implements **try...catch** blocks to handle this exception, plus a generic **Exception** object as a best practice.

Performing image analysis in C#

The purpose of the code is to make the application analyze an image and generate a description and a list of tags and detected objects. For instance, uploading an image of a car might result in the description "A red car on the road," tags such as "car" and "road," and object detection for the car itself. Code Listing 2 demonstrates how to accomplish this, with code that you need to write into the **ImageAnalysisPage.xaml.cs** file (explanations will be provided after the code).

Code Listing 2

```

using Azure;
using Azure.AI.Vision.ImageAnalysis;
using System.Text;
using System.Threading.Tasks;

namespace TravelCompanion;

public partial class ImageAnalysisPage : ContentPage
{
    private const string SubscriptionKey =
        "your-ai-vision-api-key";
    private const string Endpoint =

```

```

        "your-ai-vision-endpoint";

public ImageAnalysisPage()
{
    InitializeComponent();
}

private async Task<bool> RequestPhotoPermissionsAsync()
{
    var mediaStatus = await Permissions.
        CheckStatusAsync<Permissions.Photos>();
    if (mediaStatus != PermissionStatus.Granted)
    {
        mediaStatus = await Permissions.
            RequestAsync<Permissions.Photos>();
    }

    return mediaStatus == PermissionStatus.Granted;
}

private async void OnPickImageClicked(object sender,
    EventArgs e)
{
    bool permissionCheck = await RequestPhotoPermissionsAsync();
    if (!permissionCheck)
    {
        await DisplayAlert("Error",
            "You do not have permissions to access the photo gallery",
            "OK");
        return;
    }

    var result = await FilePicker.PickAsync(
        new PickOptions
        {
            FileTypes = FilePickerFileType.Images
        });

    if (result != null && !string.
        IsNullOrEmpty(result.FullPath))
    {
        SelectedImage.Source = ImageSource.
            FromFile(result.FullPath);
        ((Button)sender).IsEnabled = true;
    }
}

```

```

private async void OnAnalyzeImageClicked(object sender,
    EventArgs e)
{
    try
    {
        if (SelectedImage.Source == null)
            return;

        var imagePath =
            ((FileImageSource)SelectedImage.Source).File;
        var imageBytes = File.ReadAllBytes(imagePath);

        var credential =
            new AzureKeyCredential(SubscriptionKey);
        var client = new ImageAnalysisClient(
            new Uri(Endpoint), credential);

        using var imageStream = new MemoryStream(imageBytes);
        var binaryData = BinaryData.FromStream(imageStream);
        var result = await client.AnalyzeAsync(binaryData,
            VisualFeatures.Caption | VisualFeatures.Tags |
            VisualFeatures.Objects,
            new ImageAnalysisOptions { Language = "en" });

        string descriptionResult = $"Description: " +
            $"{result.Value.Caption.Text}\n";
        descriptionResult += "Tags: " + string.Join(", ",
            result.Value.Tags.Values.Select(tag => tag.Name)) + "\n";
        descriptionResult += "Objects:\n";

        AnalysisResult.Text = descriptionResult ??
            "No description available.";
    }
    catch (RequestFailedException ex)
    {
        await DisplayAlert("Error", ex.ErrorCode, "OK");
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", ex.Message, "OK");
    }
}
}

```


The code first asks for user permissions when a user tries to upload a picture. This is accomplished by invoking the **CheckStatusAsync** and **RequestAsync** static methods from the **Permissions** class. In this case, the code checks for the **Permissions.Media** permission required to access the device's media library. In the next chapters, you will learn about other permission types. Moving to code that is relevant to Azure AI Vision, the **ImageAnalysisClient** is the class designed to interact with the service. It offers methods such as **AnalyzeAsync**, which accepts a **BinaryData** image, a set of **VisualFeatures**, and optional **ImageAnalysisOptions**. This method sends the image to the service and retrieves the analysis results. The **ImageAnalysisClient** also has properties like **Endpoint**, which specifies the Azure service endpoint. Access to the service is granted via an instance of the **AzureKeyCredential** class, whose constructor receives the API key.

The **VisualFeatures** enumeration defines various features available for image analysis. Among its values are **Caption**, which generates a human-readable caption for the image, and **Tags**, which identifies and lists objects found within the image. Additionally, the **Text** property extracts textual content from an image using OCR.

The **ImageAnalysisOptions** class provides configuration options for analyzing an image. It includes properties such as **Language**, which specifies the language for textual analysis (e.g., "en" for English), and **GenderNeutralCaption**, which, when set to **true**, ensures captions avoid gender-specific terms.

Finally, the **AnalysisResult** class holds the output of the image analysis operation. Its properties include **Value**, which acts as the container for the analysis results; **Caption**, which contains the generated textual description of the image; and **Tags**, which provides a list of identified objects and concepts present in the image. Additional information can be found in the Image Analysis SDK [documentation](#), where you can find examples for .NET and for other languages.

 **Tip:** *The implementation has been simplified to display only text from the image. However, the **AnalyzeAsync** method can also detect the physical position and confidence level of every word. An example of this is available on the [image analysis NuGet package repository page](#).*

Performing OCR

OCR is the process of extracting text from images and printed documents. The Azure AI Vision service makes this process very easy, and it can be very useful. In the context of the current sample app, imagine how this could help a person with low vision at a restaurant: they take a picture of the menu, get the text from it, and, using text-to-speech (see the next chapter), make the app speak the text aloud. To implement this functionality, add the following C# code, given that you already have a button for this:

```
private async void OnOcrAnalyzeImageClicked(object sender,
EventArgs e)
{
    try
    {
```

```

        if (SelectedImage.Source == null)
            return;

        var imagePath =
            ((FileImageSource)SelectedImage.Source).File;
        var imageBytes = File.ReadAllBytes(imagePath);

        var credential =
            new AzureKeyCredential(SubscriptionKey);
        var client = new ImageAnalysisClient(
            new Uri(Endpoint), credential);

        using var imageStream = new MemoryStream(imageBytes);
        var binaryData = BinaryData.FromStream(imageStream);
        ImageAnalysisResult result = await client.AnalyzeAsync(
            binaryData,
            VisualFeatures.Read);

        var resultStringBuilder = new StringBuilder();

        foreach (DetectedTextBlock block in result.Read.Blocks)
        {
            foreach (DetectedTextLine line in block.Lines)
            {
                // Only include the text of the line
                resultStringBuilder.AppendLine(line.Text);

                // Optionally, you can append each word here if needed
                // foreach (DetectedTextWord word in line.Words)
                // {
                //     resultStringBuilder.
                //     AppendLine($"Word: '{word.Text}'");
                // }
            }
        }
        string ocrResult = resultStringBuilder.ToString();

        AnalysisResult.Text = ocrResult ??
            "No description available.";
    }
    catch (RequestFailedException ex)
    {
        await DisplayAlert("Error", ex.ErrorCode, "OK");
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", ex.Message, "OK");
    }
}

```

You still invoke **AnalyzeAsync**, but this time you apply the **Read** value from the **VisualFeatures** enumeration. The return type is still **ImageAnalysisResult**, which contains the following relevant objects, exposed by the **Read** property:

- **DetectedTextBlock**: Represents a block of text in the image (a paragraph or section).
- **DetectedTextLine**: Represents an individual line of text within a block.
- **DetectedTextWord**: Represents an individual word within a line.

These objects are returned in the OCR result and are structured hierarchically to represent how the text appears in the image. The code loops through the blocks of text, then the lines within each block, and extracts the text from each line. The extracted text is then added to a **StringBuilder**, which is used to construct the output string. The OCR engine in Azure AI Vision supports a variety of [languages](#), and a full reference of its powerful features is available in the official [documentation](#).

Adding the page to the shell

In .NET MAUI, the **Shell** object provides a common infrastructure for navigation between pages, including a flyout menu. This is a convenient and low-cost approach when you have multiple pages and you want to implement navigation quickly. Before running the application, you need to add the newly created page to the **AppShell.xaml** file. Replace the original code with the following:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
    x:Class="TravelCompanion.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:TravelCompanion"
    Shell.FlyoutBehavior="Disabled"
    Shell.NavBarIsVisible="False"
    Title="TravelCompanion">

    <TabBar x:Name="RootBar">
        <ShellContent Title="Image" Icon="camera_solid"
            ContentTemplate="{DataTemplate
                local:ImageAnalysisPage}" />
    </TabBar>
</Shell>
```

The **TabBar** object allows you to implement a navigation bar at the bottom of the page, and each **ShellContent** object represents a shortcut to the specified page. Notice how the **Title** and **Icon** properties are assigned with the text and icon that visually represent the shortcut. The target page is represented by the **ContentTemplate** property, which uses a **DataTemplate** binding expression that ensures the page instance is instantiated and rendered only when necessary. These steps will be repeated in most of the next chapters, every time you add a new page.

Running the application

In the Visual Studio toolbar, select the target device, such as an Android or iOS simulator or a physical device. When ready, press **F5** to start debugging. When the application is running, click **Pick an image** to select a picture from your device. Make sure you have a picture with some details relevant to the AI-powered analysis capabilities of the Computer Vision service. When ready, click **Analyze image** and wait for the results. Figure 15 shows an example based on both Android and iOS.

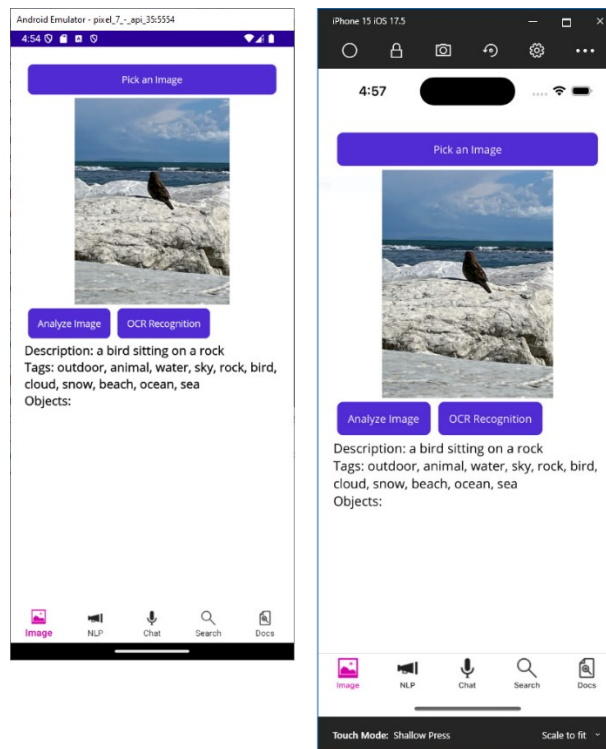


Figure 15: The result of AI-powered image analysis

As you can see, the application is able to display an accurate description and a list of tags that can help categorize the content—in this case, a bird sitting on a rock—with all the related tags. You can now choose a different image that contains text and launch the OCR processing. Figure 16 shows an example based on a picture of a restaurant menu in Italian, which means that the OCR engine can work across a variety of languages.

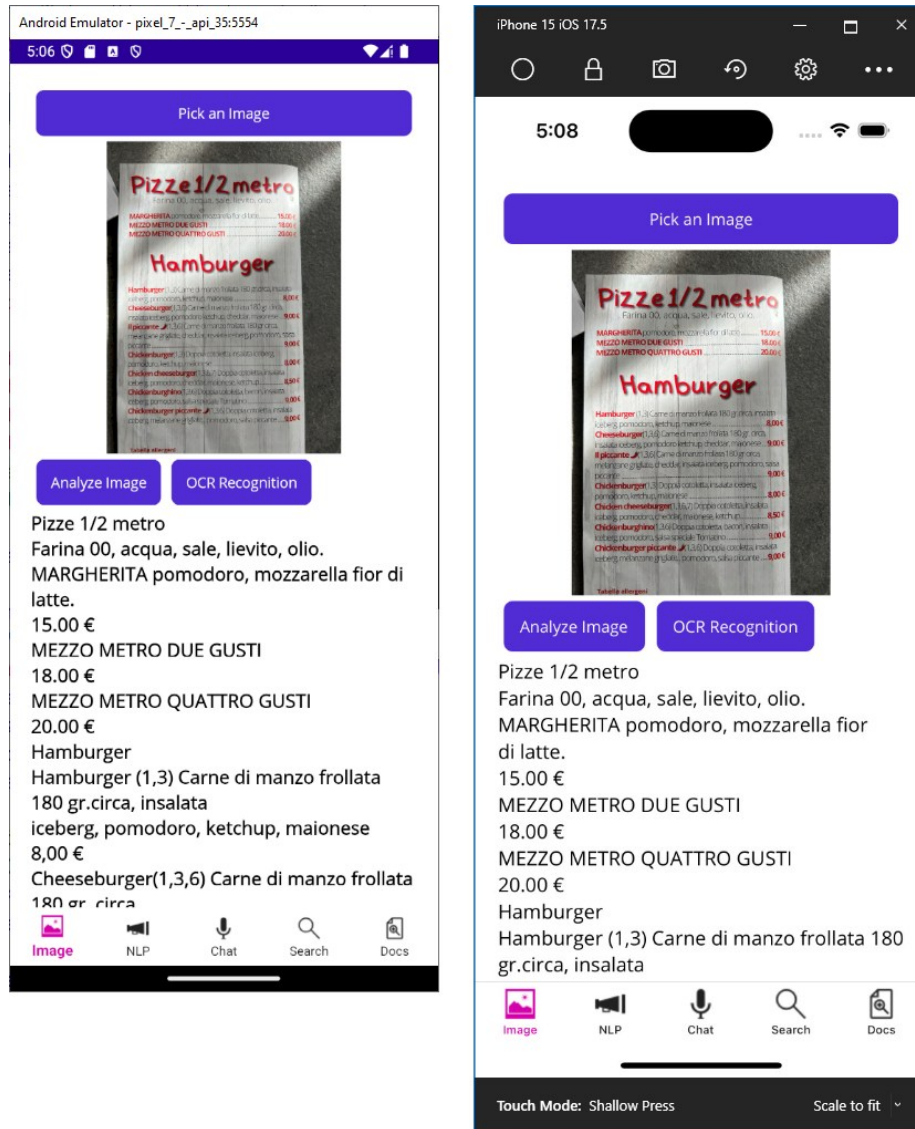


Figure 16: OCR processing of a restaurant menu

Notice how OCR detects individual lines in all the content. As you can imagine, the possible scenarios to which AI-powered image analysis can be applied are infinite. You can dramatically enhance the user's mobile experience by combining Azure AI Vision with other Azure AI services, as you will learn in the next chapter, where you will use the Speech services to make the app speak aloud the results displayed so far.

Chapter summary

In this chapter, you have seen how to integrate Azure AI Vision into the travel companion sample mobile app. You performed many steps that will be common to the next chapters, such as setting up the Azure service, retrieving API keys, installing the necessary NuGet package, and creating a functional user interface. The AI Vision client library provides the **ImageAnalysisClient** class with the **AnalyzeAsync** method that quickly runs AI-powered analysis over images, returning a detailed description and tags to categorize the content. The same class and method support OCR processing of text contained in images and printed documents. In the next chapter, you will implement speech recognition and text-to-speech capabilities.

Chapter 4 Speech Recognition and Text-to-Speech

In this chapter, you will integrate Azure Speech Services into the AI-powered travel companion app. This includes implementing speech recognition to execute voice commands and text-to-speech to convert text into spoken output. These capabilities improve accessibility and hands-free interaction within the app.

Setting up Speech Services

Similar to what you did in the previous chapter for Computer Vision, you need to set up the Azure AI Speech service in the Azure Portal. Follow these steps:

1. Go to the [Azure Portal](#) and sign in with your Azure account credentials.
2. Click **Azure AI Services**.
3. Click **Create** on the **Speech service** card.
4. Choose your Azure subscription in the **Subscription** dropdown and specify the resource group you created previously in the **Resource group** dropdown (see Figure 17).
5. Choose the **Region** closest to your location and give the service a unique **Name**.
6. In the **Pricing tier** dropdown, select **Free F0**.
7. Click **Review + Create**, and then click **Create**.

The screenshot shows the 'Create Speech Services' page in the Microsoft Azure portal. The page is titled 'Create Speech Services' and includes a brief description of the service. Below the description, there are two main sections: 'Project Details' and 'Instance Details'.

Project Details:

- Subscription:** Visual Studio Enterprise Subscription
- Resource group:** intelligentapps (with a 'Create new' link below it)

Instance Details:

- Region:** East US
- Name:** speechsuccinctly
- Pricing tier:** Free F0

At the bottom of the page, there are three buttons: 'Previous', 'Next', and 'Review + create'. There is also a 'View full pricing details' link and a 'Give feedback' button.

Figure 17: Creating and configuring the Speech Services

When the summary page appears, click **Go to resource**. The new service will appear in the list of services within the current resource group, so click on it. A webpage for the new service appears. Exactly like you did in Chapter 3 for AI Vision, click the **Click here to manage keys** hyperlink, which brings you to the page where you will be able to retrieve the API key, the service endpoint URL, and the service region. The result you get is similar to [Figure 11](#). You will need this information to access the service from C# code, so keep it ready while you open the solution in Visual Studio.

Specifying permissions

The application needs permission to access the microphone to provide speech recognition features. To accomplish this, in the **Android manifest**, add the **RECORD_AUDIO** permission. In the **Info.plist** iOS file, add the following lines:

```
<key>NSMicrophoneUsageDescription</key>
<string>Accessing the microphone is required for voice commands</string>
```

Follow the [instructions in Chapter 3](#) for adding permissions to the manifest files.

Implementing speech recognition

The first step to implement speech recognition is to install the **Microsoft.CognitiveServices.Speech** NuGet package. Follow [the steps in the previous chapter](#) to add this package to the project. Figure 18 shows how the package appears in the NuGet user interface.

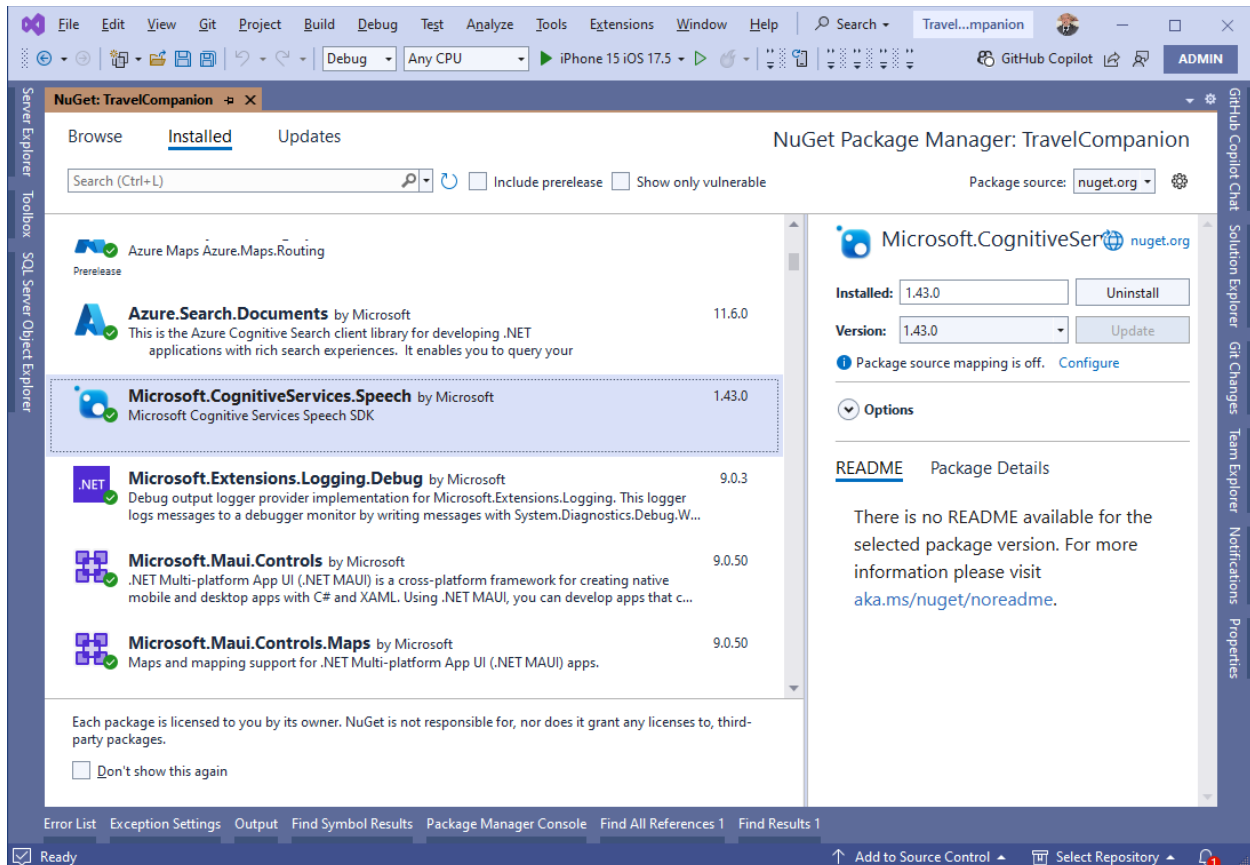


Figure 18: Installing the NuGet package for Speech Services

This package is also necessary to implement text-to-speech. The goal of the current example is to extend the image analysis feature from the previous chapter to recognize a voice command that starts the image analysis operation, instead of requiring users to click a button.

Defining the user interface

Instead of creating a new page, you will extend the existing image analysis page with two buttons: one that recognizes speech to initiate a voice command, and one that will be used to make the application speak the AI-generated description of the image. With these in mind, modify the XAML of the page as shown in Code Listing 3.

Code Listing 3

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="TravelCompanion.ImageAnalysisPage"
    Title="ImageAnalysisPage">
    <Grid Padding="20" VerticalOptions="Fill">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="300"/>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="200"/>
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <Button Text="Pick an Image" Clicked="OnPickImageClicked"
            Margin="5" />
        <Image x:Name="SelectedImage"
            Aspect="AspectFit" Grid.Row="1" />
        <Button Text="Analyze Image" Margin="5"
            Clicked="OnAnalyzeImageClicked" Grid.Row="2" />
        <Button Text="Use voice command" Margin="5"
            Clicked="OnVoiceCommandClicked" Grid.Row="3" />
        <Label x:Name="AnalysisResult" FontAttributes="Bold"
            FontSize="Medium"
            Text="Analysis results will appear here"
            Grid.Row="4" />
        <Button Text="Read Description" Clicked="OnReadDescriptionClicked"
            Grid.Row="5" Margin="5" />
    </Grid>
</ContentPage>
```

The changes are simple: the **Grid** has been extended with two rows and two new buttons have been added. The event handler for the button related to text-to-speech capabilities will be implemented in the appropriate section later in this chapter.

Performing speech recognition in C#

You will now extend the code in the **ImageAnalysisPage.xaml.cs** file as follows. First, add the following fields that must be assigned with the Speech Services API key and region, respectively:

```
private const string SpeechSubscriptionKey = "your-api-key";
private const string SpeechRegion = "service-region";
```

The next step is to check whether the app has permission to access the microphone. This is accomplished with the **RequestMicrophonePermissionsAsync** method, defined as follows:

```

private async Task<bool> RequestMicrophonePermissionsAsync()
{
    var micStatus = await Permissions.
        CheckStatusAsync<Permissions.Microphone>();
    if (micStatus != PermissionStatus.Granted)
    {
        micStatus = await Permissions.
            RequestAsync<Permissions.Microphone>();
    }

    return micStatus == PermissionStatus.Granted;
}

```

The method returns **true** if the value of **PermissionStatus** is **Granted**, which means that the user has granted the permission. The last piece for speech recognition is the following code, which handles the button click:

```

private async void OnVoiceCommandClicked(object sender, EventArgs e)
{
    bool permissionCheck = await RequestMicrophonePermissionsAsync();
    if (!permissionCheck)
    {
        await DisplayAlert("Error",
            "You do not have permissions to access the microphone",
            "OK");
        return;
    }

    var speechConfig = SpeechConfig.
        FromSubscription(SpeechSubscriptionKey, SpeechRegion);
    using var recognizer = new SpeechRecognizer(speechConfig);

    var result = await recognizer.RecognizeOnceAsync();

    if (result.Text.Contains("Analyze image",
        StringComparison.OrdinalIgnoreCase))
    {
        OnAnalyzeImageClicked(sender, e);
    }
}

```

The first part of the code checks for microphone permissions. The **SpeechConfig** class enables access to the Speech Services, invoking the **FromSubscription** method, which receives the API key and service region as credentials. The **SpeechRecognizer** class and its **RecognizeOnceAsync** method are used to interpret voice commands, specifically the phrase "analyze image" to activate the **OnAnalyzeImageClicked** method. In this example, the code searches for a specific phrase, but the **Text** property of the **SpeechRecognitionResult** return type contains the full recognized text. This is useful if you need to transcribe what the user says or make more complex decisions based on the input.

The following is a list of additional types and members from the Azure SDK for AI Speech:

- The **AudioConfig** class manages audio inputs and outputs, and an instance of it can be passed as the second argument of the **SpeechRecognizer** constructor. The **FromDefaultMicrophoneInput** method is used to capture audio from the default microphone of the system (which is also the default if you do not explicitly use this class). **AudioConfig** can also be initialized from other sources, such as **FromWavFileInput** for reading audio from a WAV file or **FromStreamInput** for capturing audio from a stream.
- The **SpeechRecognizer** class also supports continuous recognition via the **StartContinuousRecognitionAsync** method, which allows the application to keep listening and transcribing speech until explicitly stopped by **StopContinuousRecognitionAsync**. This can be useful for transcribing long conversations or speeches.
- The **SpeechRecognitionResult** class represents the outcome of a speech recognition operation. The **Reason** property is of type **ResultReason** and indicates whether the recognition was successful (**RecognizedSpeech**) or encountered issues, like **NoMatch** or **Canceled**. The recognized text is typically accessed through the **Text** property. Table 1 summarizes possible values for the **Reason** property.
- The **VoiceInfo** class represents individual voices available for speech synthesis. Its **ShortName** property identifies each voice by a concise identifier (such as **en-US-AriaNeural**), which is used to set the **SpeechSynthesisVoiceName** in the **SpeechConfig** instance.

Table 1: Values from the *ResultReason* enumeration

Value	Description
NoMatch	Indicates speech could not be recognized.
Canceled	Indicates that the recognition was canceled.
RecognizingSpeech	Indicates the speech result contains hypothesis text.
RecognizedSpeech	Indicates the speech result contains final text that has been recognized.
RecognizingIntent	Indicates the intent result contains hypothesis text and intent.
RecognizedIntent	Indicates the intent result contains final text and intent.
TranslatingSpeech	Indicates the translation result contains hypothesis text and its translations.

Value	Description
TranslatedSpeech	Indicates the translation result contains final text and corresponding translations.
SynthesizingAudio	Indicates the synthesized audio result contains a non-zero amount of audio data.
SynthesizingAudioCompleted	Indicates the synthesized audio is now complete for this phrase.
RecognizingKeyword	Indicates the speech result contains (unverified) keyword text.
RecognizedKeyword	Indicates that keyword recognition completed recognizing the given keyword.
SynthesizingAudioStarted	Indicates the speech synthesis is now started.
TranslatingParticipantSpeech	Indicates the transcription result contains hypothesis text and its translations for other participants in the conversation.
TranslatedParticipantSpeech	Indicates the transcription result contains final text and corresponding translations for other participants in the conversation.
TranslatedInstantMessage	Indicates the transcription result contains the instant message and corresponding translations.
TranslatedParticipantInstantMessage	Indicates the transcription result contains the instant message for other participants in the conversation and corresponding translations.
EnrollingVoiceProfile	Indicates the voice profile is being enrolled and more audio is needed to complete a voice profile.
EnrolledVoiceProfile	Indicates the voice profile has been enrolled.
RecognizedSpeakers	Indicates successful identification of some speakers.
RecognizedSpeaker	Indicates successful verification of a speaker.
ResetVoiceProfile	Indicates a voice profile has been reset.

Value	Description
DeletedVoiceProfile	Indicates a voice profile has been deleted.
VoicesListRetrieved	Indicates the voices list has been retrieved successfully.



Note: The official documentation contains the full list of [synthesized voices](#) and [supported languages](#).

Before running the example, you will complete the code with the implementation of text-to-speech features.

Implementing text-to-speech

Text-to-speech converts written text into spoken audio, leveraging the device's audio output. The Azure AI Speech service includes text-to-speech capabilities, which can be invoked via the .NET objects defined in the Microsoft.CognitiveServices.Speech NuGet package that you installed previously. Given that most of the code setup has been done in the previous section about speech recognition, the only code that you need to add is the following event handler:

```
private async void OnReadDescriptionClicked(object sender, EventArgs e)
{
    try
    {
        if (string.IsNullOrEmpty(AnalysisResult.Text))
            return;

        var speechConfig = SpeechConfig.
            FromSubscription(SpeechSubscriptionKey, SpeechRegion);
        using var synthesizer = new SpeechSynthesizer(speechConfig);

        await synthesizer.SpeakTextAsync(AnalysisResult.Text);
    }
    catch (RequestFailedException ex)
    {
        await DisplayAlert("Error", ex.ErrorCode, "OK");
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", ex.Message, "OK");
    }
}
```

The following is a list of the relevant types and members in the code:

- The **SpeechConfig** class has the same purpose as the previous example. In this case, it is worth mentioning the **SpeechSynthesisVoiceName** property, which allows you to specify the voice to use for synthesis. For instance, you might set this to **en-US-JennyNeural** for a specific American English voice.
- The **SpeechSynthesizer** class handles the conversion of text into spoken audio. It requires a **SpeechConfig** object for initialization. The **SpeakTextAsync** method takes a string of text and synthesizes it into speech, which is then played through the configured audio output device. For more control, the **SpeakSsmlAsync** method can be used to synthesize speech from Speech Synthesis Markup Language (SSML) input, enabling finer control over speech characteristics. Additionally, **StartSpeakingAsync** begins synthesis and returns immediately, allowing the app to perform other tasks while synthesis continues in the background.
- The **SpeechSynthesisResult** class contains the result of the text-to-speech operation. The **Reason** property indicates whether the synthesis was successful (**SynthesizingAudioCompleted**) or encountered problems. Possible values are listed in Table 1. The **AudioData** property contains the synthesized audio data in raw bytes, which can be saved or processed further.

At this point, you can finally run the application and observe the result. More information about what is available in the Speech SDK can be found in the [official documentation](#).

Running the application

Press **F5** to run the application on the device (physical or emulator) of choice. Select an image from your gallery and select **Use voice command**. Say “Analyze image,” and after a few seconds you will see the description of the image in the label. You have achieved the same result as Chapter 3, but with a voice command. Figure 19 shows an example of the image analysis result, along with the user interface edits.

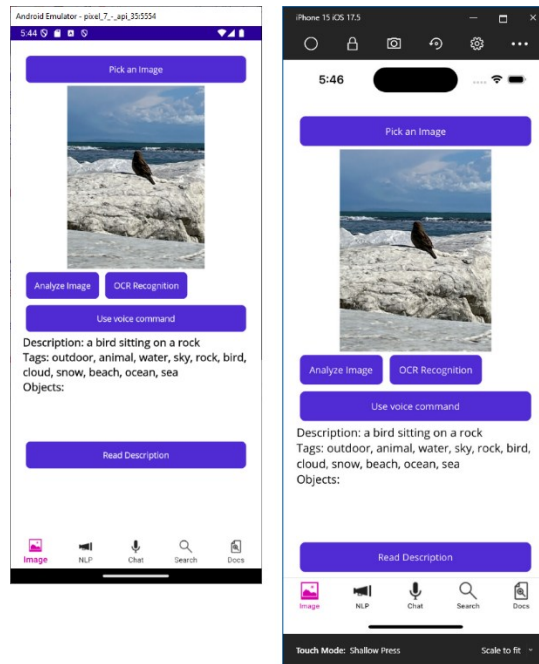


Figure 19: The image analysis result triggered by a voice command

If you now select **Read Description**, your device will read the description text aloud (make sure the device is not in silent mode or muted). With very little effort, you have implemented voice commands with speech recognition and text-to-speech, dramatically enhancing the user experience.

Chapter summary

In this chapter, you have seen how to implement speech recognition and text-to-speech offered by the Azure AI Speech service to improve the user experience in a .NET MAUI project. You have seen how to request microphone permissions and how to implement voice commands with the **SpeechRecognizer** class. Finally, you have seen how to leverage the **SpeechSynthesizer** class to convert text into spoken audio. This enhances the user experience by allowing hands-free interaction. In the next chapter, you will explore natural language processing services.

Chapter 5 Natural Language Processing and Chatbots

Natural language processing (NLP) focuses on the interaction between computers and humans through natural language. NLP analyzes text or speech data, enabling applications to support language translation, sentiment analysis, chatbots, speech recognition, and text summarization. In this chapter, you will learn how to integrate the Azure AI NLP services into a .NET MAUI project to analyze text sentiment, translate text, and implement a basic conversational UI based on a ChatGPT model.

Setting up the Azure AI services

In this chapter, you will use three AI services: Azure AI Language (formerly Text Analytics) to analyze text sentiment; Azure Translator to implement translation services; and Azure OpenAI to build a chatbot based on one of the available ChatGPT models. Each of these services must be set up in the [Azure Portal](#).

Setting up the Azure AI Language service

To set up the Azure AI Language service and implement text sentiment analysis, follow these steps, similar to what you have done previously for other services:

1. Locate the **Language Service** on the **Azure AI services** page (see Figure 20).
2. Click **Create** on the service's card.

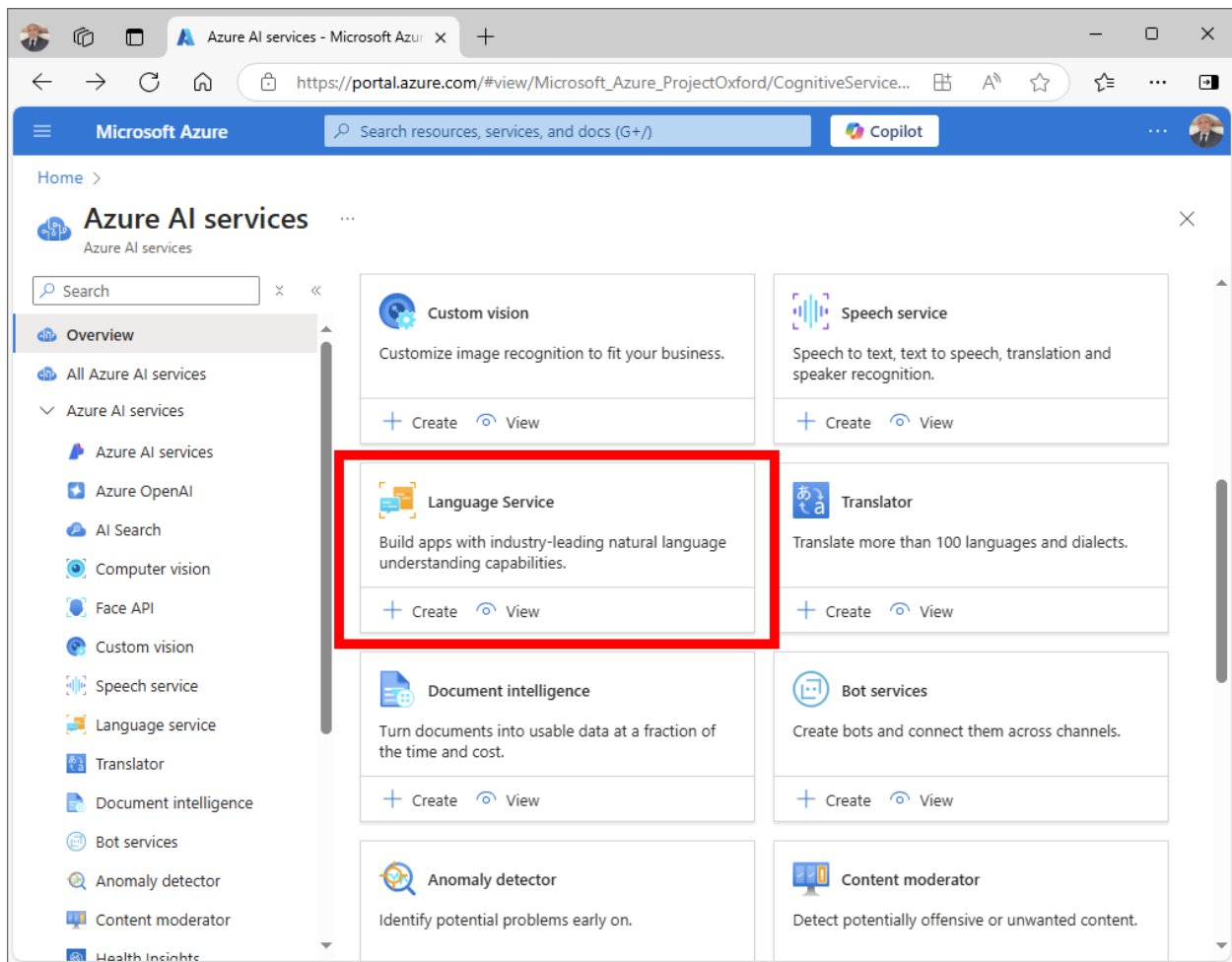


Figure 20: The Language Service on the Azure AI services page

3. Choose a **Subscription** and **Resource Group**.
4. Select the closest **Region** and the free **Pricing Tier**.
5. Assign **languagesuccinctly** as the service name.
6. Review and accept the **Responsible AI Notice**.
7. Click **Review + Create** and then **Create**.

During the service setup, you will have options to configure custom features in addition to the preconfigured ones, as shown in Figure 21.

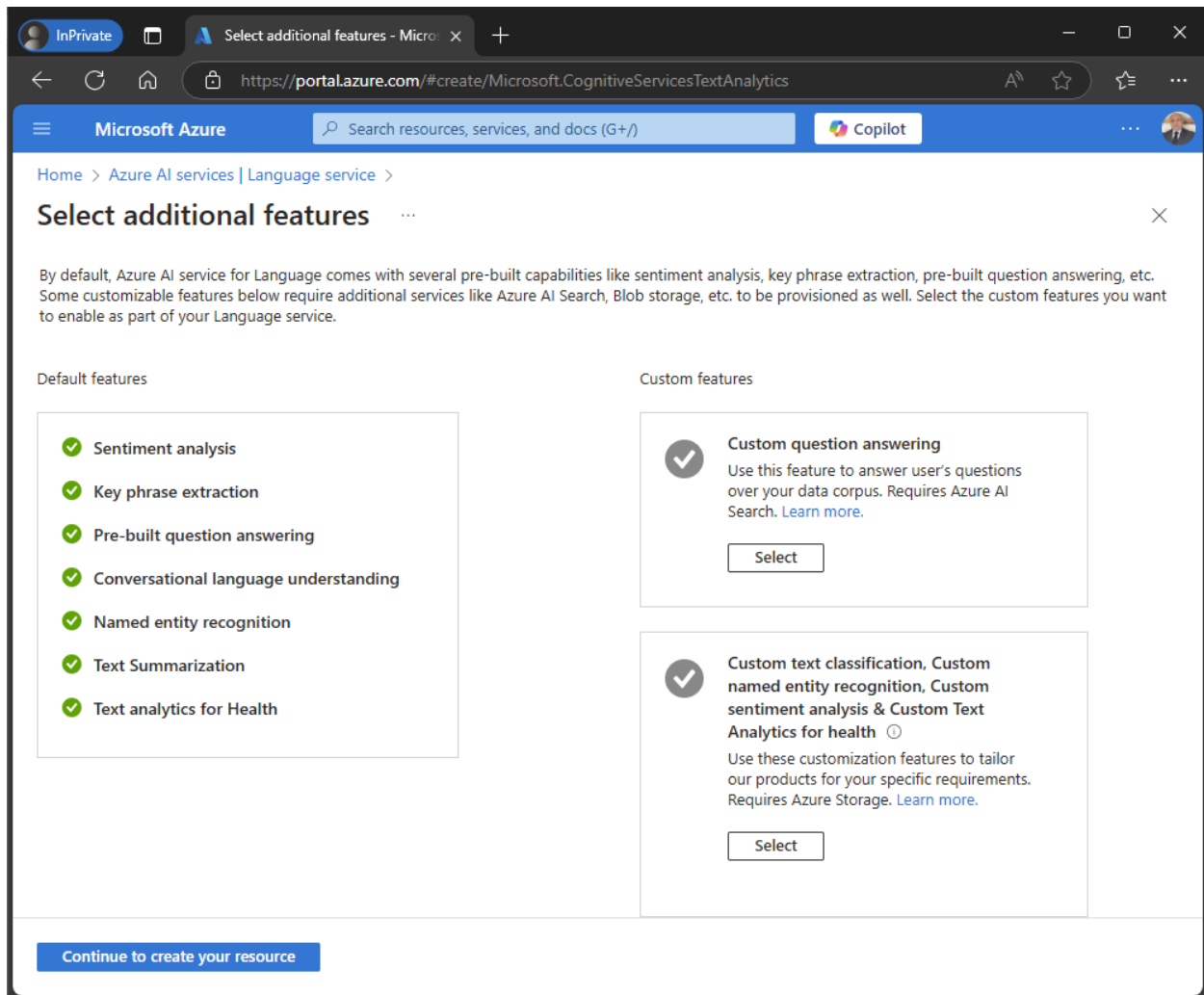


Figure 21: Options to add custom features to the AI Language service

These are not covered in this ebook and can be skipped, so click **Continue to create your resource**. When the service is ready, follow the lessons learned in the previous chapters to retrieve and store the API key and the endpoint URL for the service.

Setting up the Azure Translator service

Setting up the Azure Translator service is very easy, and involves the steps that you have followed so far in the Azure Portal:

1. Locate the **Translator** service in the **Azure AI services** page. If you look at Figure 20, it is located to the right of the Language Service.
2. Click **Create** on the service's card.
3. Choose a **Subscription** and **Resource Group**.

4. Select the closest **Region** and the free **Pricing Tier**.
5. Assign **translatorsuccinctly** as the service name.
6. Click **Review + Create** and then **Confirm**.

When the service is ready, go to the new resource page and retrieve the API key and endpoint URL, exactly as you did previously. The resource page also provides a playground with a nice user interface that you can use to see how the translator works.

Setting up the Azure OpenAI service



Note: Azure OpenAI services are offered with a pay-as-you-go model. This means that every transaction is charged, depending on your plan, and that no free plan is available. Make sure to set a spending limit in your Azure subscription to avoid extra costs, especially at development time, and don't forget to completely delete resources that you no longer use.

Creating the Azure OpenAI service to provide basic chatbot capabilities via one of the available ChatGPT models involves different steps. You need to open the [Azure AI Foundry website](#) and log in with the same Microsoft account that you use for the Azure subscription. Azure AI Foundry is a platform that allows you to build and deploy generative AI services, simplifying the entire process by leveraging existing, popular models instead of customizing your own. The [list of existing models](#) is very long and includes AI models from other vendors, such as the popular DeepSeek and Nvidia NIM. Once you log in, you will see a welcome page with a button labeled **Create project** that you have to click. In the **Create a project** page (see Figure 22), you will create a hub, which is a collaboration environment for your team. The portal automatically generates a name for the hub, which you can change if you want. Leave the proposed one and take note of it because it will be used shortly in the code. You will then need to specify the Azure subscription, the resource group, and the service location. An Azure OpenAI service will also be generated with the same name as the hub. When ready, click **Create**.

Create a project

2 Create a hub

Create a hub for your projects
A hub is the collaboration environment for your team to share your project work, model endpoints, compute, connections, and security settings. [Learn more](#)

Do you need to customize security or the [dependent resources](#) of your hub? [Go to Azure Portal](#)

Hub name *
alessandrodelsole-0664_ai

Subscription * [Create new subscription](#)
Visual Studio Enterprise Subscription

Resource group * [Create new resource group](#)
intelligentapps

Location * [Help me choose](#)
West Europe

Connect Azure AI Services or Azure OpenAI Service * [Create new AI Services](#)
(new) ai-alessandrodelsole-0664_ai

Connect Azure AI Search [Create new AI Search](#)
Skip connecting

[Back](#) [Next](#) [Create](#) [Cancel](#)

Figure 22: Creating a new AI Foundry project

After a few seconds, the new service will be provisioned, and the portal will generate an identifier that is visible at the top of the page (see Figure 23). In this summary page, you will also see the API key and the service endpoint URL. Take note of both.

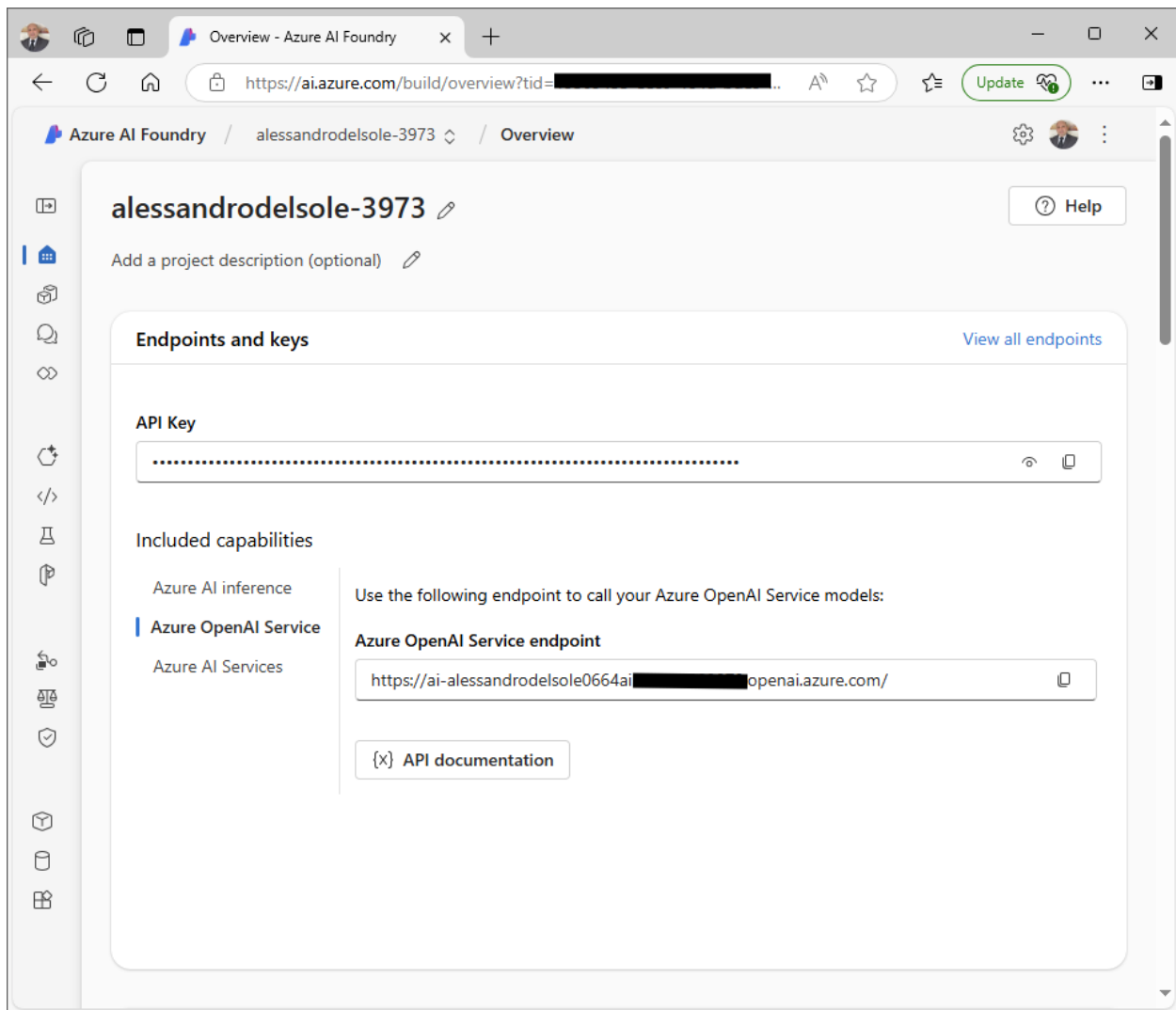


Figure 23: The details required to consume the service

The next step is to deploy an AI model to the service that satisfies the requirement to implement a chatbot that processes natural language and returns responses in the same way. In order to make the navigation simpler, open the [Management center](#) page. Select the newly created hub, and in the details page, click the **Models + endpoints** shortcut in the sidebar (see Figure 24). This will open the **Model deployments** page, where you have to click **Deploy model**. You will be prompted with two options: Deploy base model or Deploy fine-tuned model.

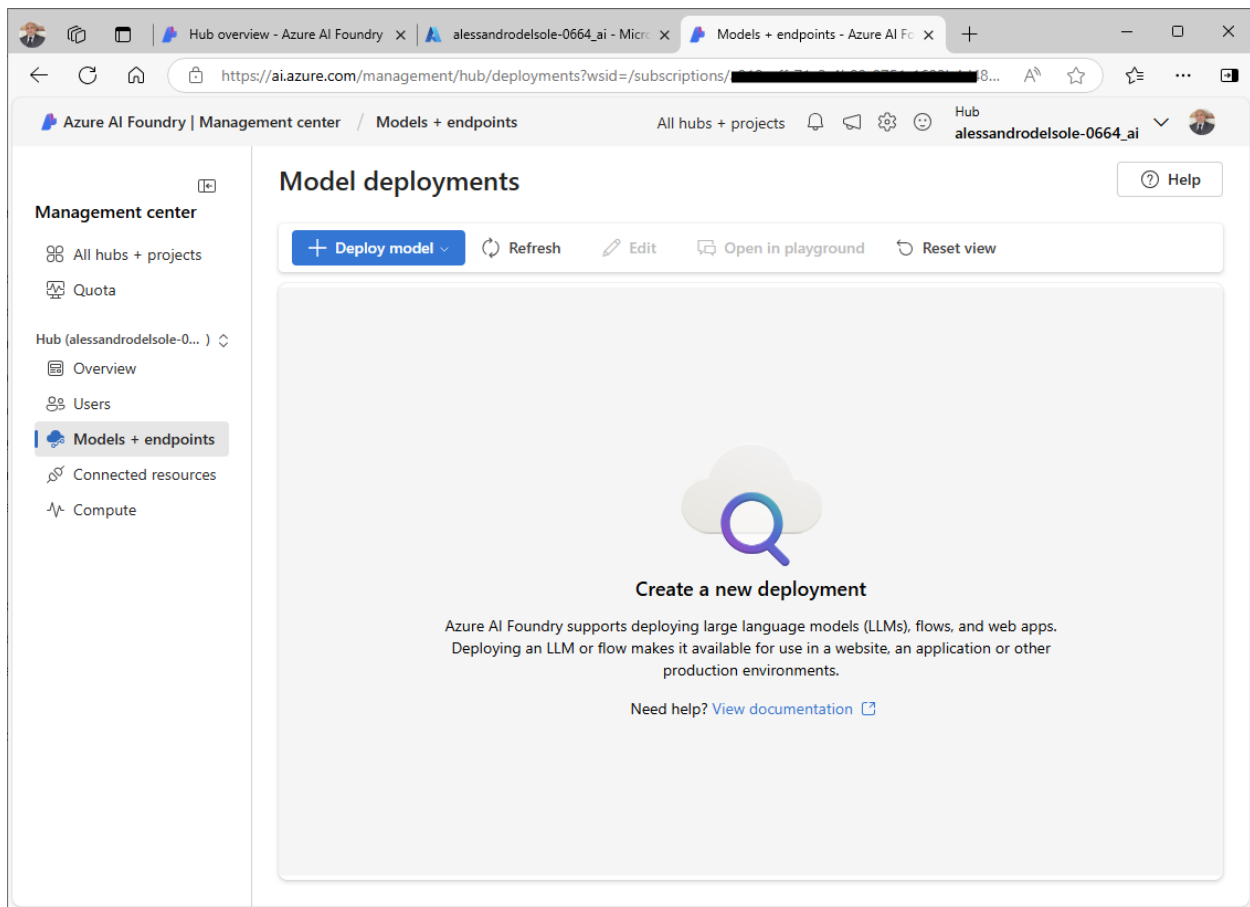


Figure 24: Creating a model deployment

Select **Deploy base model**. As you can see in Figure 25, you will be able to select from a list of OpenAI models that you can use to implement a natural language chatbot. You can read the description of each model by clicking its name. The availability depends on the service plan that you chose. For example, gpt-4 models are not available under the Standard plan. However, you can select the gpt-35-turbo model, as shown in Figure 25; that is enough to implement a basic ChatGPT conversational user experience in the sample app.

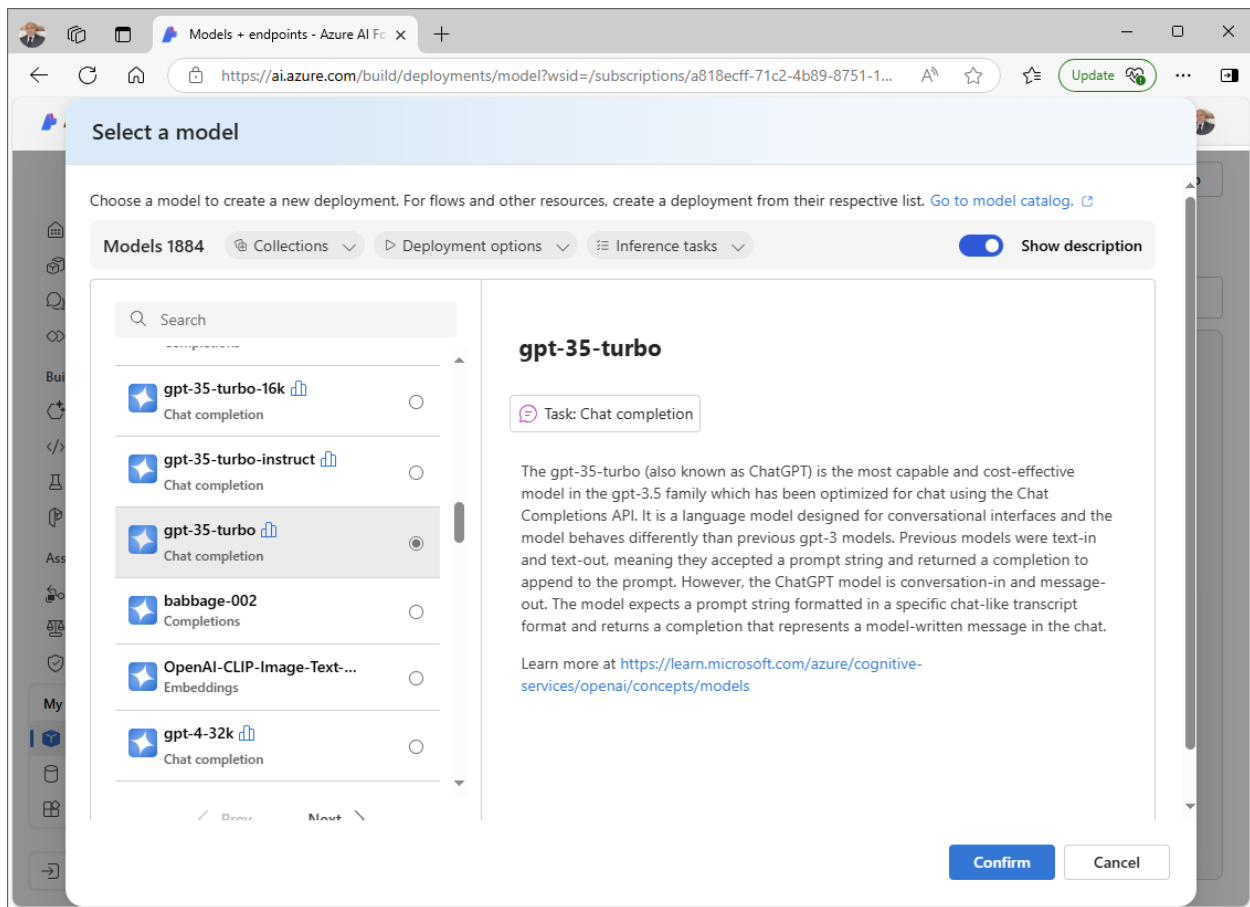


Figure 25: Selecting a pretrained AI model



Note: Available models change over time. New models can be released, and older models can be retired. This could have happened between the time of writing this ebook and its publication. For this reason, it is not guaranteed that the model used in this chapter will be available at the time of your reading. If this is the case, select a different model that satisfies your needs and that fits into your service quota (an error message appears if it does not fit).

Click **Confirm** and wait for the completion page to appear. At this point, you are ready to install the necessary NuGet packages in the project.

Installing NuGet packages

You will need the following NuGet packages to proceed with the upcoming examples:

- **Azure.AI.TextAnalytics:** This provides the .NET client library for implementing sentiment analysis.

- **Azure.AI.Translation.Text:** This provides the .NET client library that you use to deliver AI-powered real-time translations.
- **Azure.AI.OpenAI:** This provides the .NET client library that makes it easier to send messages and receive responses from the selected OpenAI model.

The simplest way to install these in Visual Studio 2022 is to use the NuGet Package Manager user interface, so that you can install them quickly by just searching for their names.

Analyzing text sentiment

Sentiment analysis is the process of using NLP techniques to analyze textual data and determine the emotional tone or sentiment expressed in it, for example, positive, negative, or neutral. Possible applications include gauging customer opinions, social media attitudes, and general market research. In a mobile application like the sample travel companion app, it could be useful to understand the user's perception of a place they visited, like a restaurant. For the sake of simplicity, this example allows the user to enter some text and then the app analyzes its sentiment. Make sure you add a new page called **NaturalLanguagePage** to the project and then add the XAML shown in Code Listing 4.

Code Listing 4

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="TravelCompanion.NaturalLanguagePage"
              Title="NaturalLanguagePage">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition />
            <RowDefinition />
        </Grid.RowDefinitions>
        <Grid RowDefinitions="Auto, Auto, *" Padding="10">
            <Editor x:Name="SentimentInputText" FontSize="18"
                  Placeholder="Enter text for sentiment analysis..."
                  HeightRequest="100" Grid.Row="0"/>
            <Button Text="Analyze Sentiment"
                   Clicked="OnAnalyzeSentimentClicked" Grid.Row="1"/>
            <Label x:Name="ResultLabel" TextColor="Black" FontSize="Medium"
                  Grid.Row="2"/>
        </Grid>
    </ContentPage>
```

The code is very simple, and the relevant points are the use of an **Editor** as the input view and a **Button** that triggers the analysis. With an approach similar to what you have done for the previous services, in the C# code for the page, you will need to declare variables that represent the API key, the endpoint, and the client. Code Listing 5 shows the full implementation for the page.

Code Listing 5

```
using Azure;
using Azure.AI.TextAnalytics;

namespace TravelCompanion;

public partial class NaturalLanguagePage : ContentPage
{
    private const string AILanguageApiKey =
        "your-ai-language-api-key";
    private const string AILanguageEndpoint =
        "your-ai-language-endpoint";
    private readonly TextAnalyticsClient _languageClient;

    private async void OnAnalyzeSentimentClicked(object sender,
        EventArgs e)
    {
        try
        {
            var text = SentimentInputText.Text;
            if (string.IsNullOrEmpty(text)) return;
            var result = _languageClient.AnalyzeSentiment(text);
            ResultLabel.Text =
                $"Sentiment: {result.Value.Sentiment}";
        }
        catch (RequestFailedException ex)
        {
            await DisplayAlert("Error", ex.ErrorCode, "OK");
        }
        catch (Exception ex)
        {
            await DisplayAlert("Error", ex.Message, "OK");
        }
    }

    public NaturalLanguagePage()
    {
        InitializeComponent();
        _languageClient =
            new TextAnalyticsClient(new Uri(AILanguageEndpoint),
                new AzureKeyCredential(AILanguageApiKey));
    }
}
```

The **TextAnalyticsClient** class is the client object that allows you to consume the service, and it is initialized by passing the service endpoint and the API key via the **AzureKeyCredential** class. The **AnalyzeSentimentAsync** method returns an object of type **DocumentSentiment** that represents the result of the sentiment analysis. It contains the **Sentiment** property, which indicates the overall sentiment of the text, and **ConfidenceScores** (not used here), which provides confidence levels for each sentiment category (positive, neutral, and negative).

Hints about more advanced text processing

As you learned previously, Text Analytics is part of the AI Language services, and it is much more than sentiment analysis. With AI Language, you can perform entity recognition, keyphrase extraction, text summarization, and language detection. Though these services are not described in this chapter, it is worth mentioning the .NET objects from the **TextAnalyticsClient** class that you can use to implement them:

- The **RecognizeEntitiesAsync** method detects categorized entities in text and returns the **CategorizedEntityCollection** object. This collection contains entities identified by categories, subcategories, and confidence scores for detailed analysis.
- The **DetectLanguageAsync** method determines the primary language of a document or set of documents and returns the **DetectedLanguage** object. This object includes properties like **Name** for the language name and **ConfidenceScore** for the detection accuracy.
- The **ExtractiveSummarizeAsync** method performs text summarization and can return results in various forms. It provides a collection of summarized results as an **ExtractiveSummarizeResultCollection** object, in which each result corresponds to a summarized document and can be accessed by index. For individual documents, it returns an **ExtractiveSummarizeResult** object, which contains a **Sentences** property (a collection of **SummarySentence** objects) and additional properties like **HasError** (for processing errors) and **Id** (to identify the document). Each sentence in the summary is represented by a **SummarySentence** object that includes properties such as **Text**, **RankScore** (relevance), **Offset**, and **Length**.

Further information and examples can be found in the official [documentation](#) and in the [Azure AI Services Succinctly](#) ebook.

Before running the example, the new page must be added to the app's shell so that it is included in the navigation stack. To do so, add the following XAML code to the AppShell.xaml file:

```
<ShellContent Title="NLP" Icon="comments_solid.png"
              ContentTemplate="{DataTemplate local:NaturalLanguagePage}" />
```

At this point you are ready to test your code.

Running the example

Press **F5** to start debugging the application on your device of choice and tap the new page's icon when the app is running. In the new page, enter the following sentence:

“We went for some snorkeling in the coral reef of the Red Sea. It was an incredible experience with a variety of beautiful fishes”

Then, click **Analyze Sentiment**. As you can see in Figure 26, the application detects a positive sentiment from the statement. You can definitely try writing different sentences—for example, some representing disappointment—and see how the result changes.

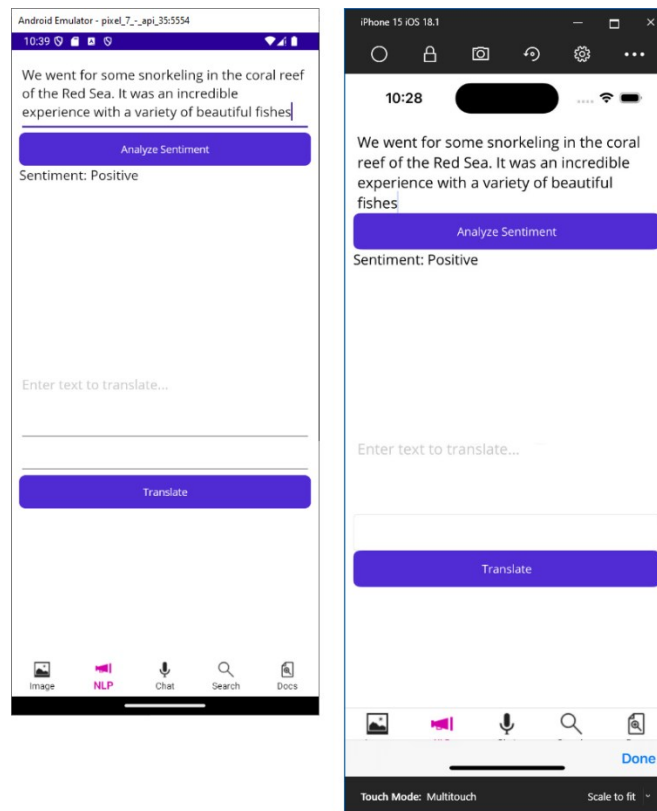


Figure 26: Detecting sentiment from text

You can run text sentiment analysis over full documents, or you could even implement speech recognition (as in Chapter 4) to run sentiment analysis over the string that is retrieved from spoken audio.

Executing AI-powered translations

The Azure Translator service offers a wide range of features for text and document translation in real-time or batch processes. It supports numerous languages and allows both general and specialized translation scenarios, such as translating conversational text, technical documents, or entire websites. For the current example, you will implement text translation. You can extend the code of the **NaturalLanguagePage** with the XAML shown in Code Listing 6.

Code Listing 6

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="TravelCompanion.NaturalLanguagePage"
    Title="NaturalLanguagePage">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition />
            <RowDefinition />
        </Grid.RowDefinitions>
        <Grid RowDefinitions="Auto, Auto, *" Padding="10">
            <Editor x:Name="SentimentInputText" Placeholder="Enter text for
sentiment analysis..." HeightRequest="100" Grid.Row="0"/>
            <Button Text="Analyze Sentiment" Clicked="OnAnalyzeSentimentCli
cked" Grid.Row="1"/>
            <Label x:Name="ResultLabel" TextColor="Black" FontSize="Medium"
Grid.Row="2"/>
        </Grid>

        <Grid RowDefinitions="Auto, Auto, Auto, *"
            Padding="10" Grid.Row="1">
            <Editor x:Name="TranslatorInputText" FontSize="18"
                Placeholder="Enter text to translate..."
                HeightRequest="100" Grid.Row="0"/>
            <Picker x:Name="LanguagePicker" Grid.Row="1">
                <Picker.Items>
                    <x:String>fr</x:String>
                    <x:String>es</x:String>
                    <x:String>de</x:String>
                </Picker.Items>
            </Picker>
            <Button Text="Translate"
                Clicked="OnTranslateClicked"
                Grid.Row="2"/>
            <Label x:Name="TranslatedTextLabel"
                TextColor="Black"
                FontSize="Medium" Grid.Row="3"/>
        </Grid>
    </Grid>
```

```
</ContentPage>
```

The code adds an **Editor** for text input, a **Picker** to select the target language, and a **Button** to trigger the translation. Notice that you do not need to specify the source language because it is automatically detected. Code Listing 7 shows how to extend the code-behind file with the translation logic.

Code Listing 7

```
using Azure;
using Azure.AI.TextAnalytics;
using Azure.AI.Translation.Text;

namespace TravelCompanion;

public partial class NaturalLanguagePage : ContentPage
{
    private const string AILanguageApiKey =
        "your-ai-language-api-key";
    private const string AILanguageEndpoint =
        "your-ai-language-endpoint";
    private readonly TextAnalyticsClient _languageClient;
    private const string TranslatorApiKey =
        "your-ai-translator-api-key";
    private const string TranslatorEndpoint =
        "your-ai-translator-endpoint";
    private readonly TextTranslationClient _translatorClient;

    private async void OnAnalyzeSentimentClicked(object sender,
        EventArgs e)
    {
        try
        {
            var text = SentimentInputText.Text;
            if (string.IsNullOrEmpty(text)) return;
            var result = _languageClient.AnalyzeSentiment(text);
            ResultLabel.Text =
                $"Sentiment: {result.Value.Sentiment}";
        }
        catch (RequestFailedException ex)
        {
            await DisplayAlert("Error", ex.ErrorCode, "OK");
        }
        catch (Exception ex)
        {
            await DisplayAlert("Error", ex.Message, "OK");
        }
    }
}
```

```

private async void OnTranslateClicked(object sender,
    EventArgs e)
{
    try
    {
        var text = TranslatorInputText.Text;
        var targetLanguage =
            LanguagePicker.SelectedItem as string;
        if (string.IsNullOrEmpty(text) ||
            string.IsNullOrEmpty(targetLanguage)) return;
        var response =
            await _translatorClient.
                TranslateAsync(targetLanguage, text);
        TranslatedTextLabel.Text =
            response.Value.FirstOrDefault()?.Translations.
                FirstOrDefault()?.Text ?? "Translation failed";
    }
    catch (RequestFailedException ex)
    {
        await DisplayAlert("Error", ex.ErrorCode, "OK");
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", ex.Message, "OK");
    }
}

public NaturalLanguagePage()
{
    InitializeComponent();
    _languageClient =
        new TextAnalyticsClient(new Uri(AILanguageEndpoint),
            new AzureKeyCredential(AILanguageApiKey));
    _translatorClient =
        new TextTranslationClient(
            new AzureKeyCredential(TranslatorApiKey),
            new Uri(TranslatorEndpoint));
}
}

```

The **TextTranslationClient** class allows you to interact with Azure Translator's Text Translation API. You still pass an instance of the **AzureKeyCredential** class to provide authentication via the API key. The **TranslateAsync** method translates text from one language to another; the first argument is the target language, and the second argument is the input text. The return type is an object of type **Response<IReadOnlyCollection<TranslatedTextItem>>**, and the actual collection is stored inside the **Value** property. Each item in the collection is a **TranslatedTextItem** object, which exposes the **Translations** property. This is an **IReadOnlyList<TranslationText>** object, and the **TranslationText** class actually contains the translated text and the detected language with the level of confidence.

For simple translations like the current example, it is sufficient to retrieve the string contained in the **Text** property of the first element in the collection. It is worth mentioning that you are not limited to one language. In fact, the **TranslateAsync** method offers an overload that takes an object of type **TextTranslationTranslateOptions** as an argument and allows you to specify multiple languages and multiple strings to be translated. Further information on developing with the Azure Translator can be found in the [official documentation](#).

Running the example

If you run the example now, you will discover the power of the Azure Translator service by writing any sentence. Figure 27 shows an example based on the user's need to translate a sentence as a visitor from another country.

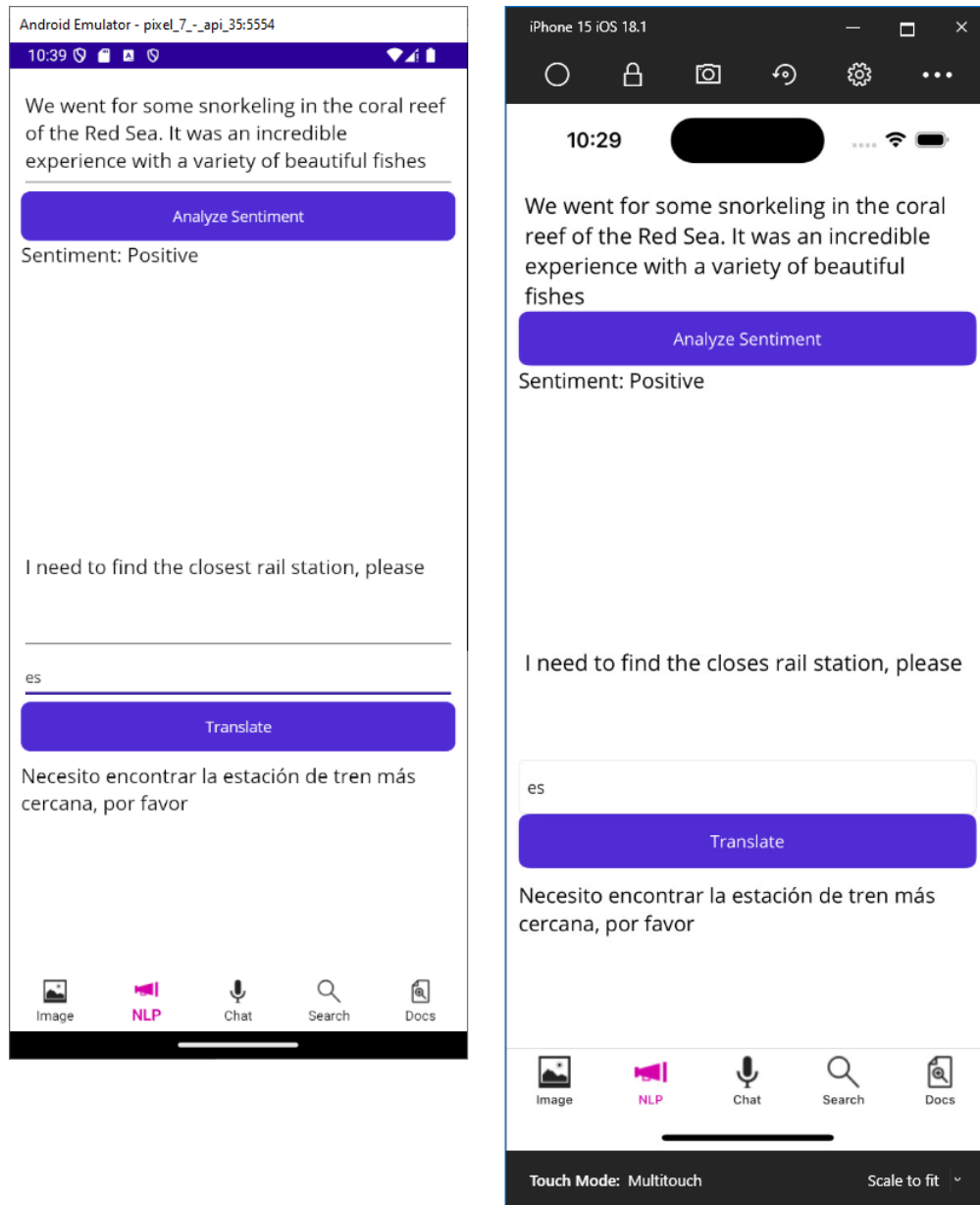


Figure 27: Translating text with the Azure Translator service

This is just one possible application of the translator, but it gives you an idea of its power.



Note: The Azure Translator service is capable of much more than this. It can translate entire documents while preserving the original format, and it can perform transliteration, the process of converting text from one script to another. Converting Arabic script to Latin script or Devanagari script to Latin script is an example of transliteration. This allows you to make content readable for people who understand the language but use a different script.

Implementing a basic ChatGPT user interface

Simple chatbots have existed for many years and have helped businesses solve a variety of customer issues, reducing the amount of human intervention. As AI advances, chatbots take advantage of more sophisticated NLP features. In this section, you will learn how to implement a basic chatbot that leverages one of the available ChatGPT models. Remember that the ChatGPT application and, more generally, OpenAI GPT models, are not simply chat engines; they are extremely sophisticated AI models that serve multiple purposes, and chat functionality is just one. With that in mind, add a new page called **ChatbotPage.xaml** to the project and define a simple user interface as shown in Code Listing 8.

Code Listing 8

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="TravelCompanion.ChatbotPage"
    Title="ChatbotPage">
    <Grid RowDefinitions="*, Auto" Padding="10">
        <CollectionView x:Name="ChatMessages"
            ItemsSource="{Binding Messages}">
            <CollectionView.ItemsLayout>
                <LinearItemsLayout Orientation="Vertical"
                    ItemSpacing="10"/>
            </CollectionView.ItemsLayout>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding}"
                        FontAttributes="Bold"
                        FontSize="Medium"/>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
        <Grid RowDefinitions="Auto,Auto" RowSpacing="5" Grid.Row="1">
            <Entry x:Name="MessageEntry" Placeholder="Type a message..."/>
            <Button Text="Send" Clicked="OnSendMessageClicked"
                Grid.Row="2"/>
        </Grid>
    </Grid>
</ContentPage>
```

This is really simple code. You are adding a **CollectionView** control to display the list of messages exchanged between the user and the AI assistant, and an **Entry** as the input view. A button is used to send the user's message to the service. Obviously, the magic happens in the C# code, shown in Code Listing 9.

Code Listing 9

```
using Azure;
```

```

using Azure.AI.OpenAI;
using OpenAI.Chat;
using System.Collections.ObjectModel;

namespace TravelCompanion;

public partial class ChatbotPage : ContentPage
{
    private const string ApiKey =
        "your-api-key";
    private readonly AzureOpenAIClient openAiClient;
    private ChatClient chatClient;
    public ObservableCollection<string> Messages { get; set; } = new();

    public ChatbotPage()
    {
        InitializeComponent();

        var endpoint =
            new Uri("your-service-endpoint");
        var credential = new AzureKeyCredential(ApiKey);

        openAiClient = new AzureOpenAIClient(
            endpoint,
            credential);

        ChatMessages.ItemsSource = Messages;
    }

    private async void OnSendMessageClicked(object sender, EventArgs e)
    {
        var userInput = MessageEntry.Text;
        if (string.IsNullOrEmpty(userInput)) return;

        Messages.Add($"You: {userInput}");
        MessageEntry.Text = string.Empty;
        await GetChatResponseAsync(userInput);
    }

    private async Task GetChatResponseAsync(string userInput)
    {
        try
        {
            chatClient = openAiClient.GetChatClient("gpt-35-turbo");

            var message = new UserChatMessage(userInput);

            var response = await chatClient.CompleteChatAsync(message);
            Messages.Add($"Assistant: {response.Value.Content[0].Text}");
        }
    }
}

```

```

    }
    catch (Exception ex)
    {
        Messages.Add($"Bot: An error occurred - {ex.Message}");
    }
}
}

```

The **AzureOpenAIClient** class is used to authenticate and interact with the Azure OpenAI API, initialized with the specified service endpoint and API key. The **Messages** property is an **ObservableCollection<string>** that stores chat messages exchanged between the user and the assistant. This collection is bound to the **CollectionView** for real-time display, ensuring that updates are instantly reflected in the user interface. The constructor sets up the OpenAI client while binding the chat message collection to the interface.

When the user enters a message and clicks the Send button, the user input is added to the **Messages** collection. The asynchronous **GetChatResponseAsync** method creates an instance of the **ChatClient** class for the GPT model, sends the user input as a **UserChatMessage**, and awaits a response through the **CompleteChatAsync** method. This method returns a **Response<ChatCompletions>** object, which encapsulates both the content of the assistant's generated response and metadata related to the request.

Understanding completion responses

The core of the response is the **ChatCompletions** object, which represents the overall result of the chat completion operation. Within this object, the **Choices** property contains a collection of **ChatChoice** objects, each representing a possible reply generated by the GPT model. If multiple responses are requested, all of them will be included in this collection. Additionally, the **ChatCompletions** object includes an **Id** property to uniquely identify the chat completion request, a **Model** property specifying which GPT model was used, and a **Usage** property that details token usage for the request and response. The **Usage** information is particularly helpful for monitoring API consumption and associated costs. Before running the example, you must add the new page to the app shell so that it is included in the navigation stack. To do so, add the following XAML code to the AppShell.xaml file:

```

<ShellContent Title="Chat" Icon="microphone_solid.png"
    ContentTemplate="{DataTemplate local:ChatbotPage}" />

```



Tip: You could extend this example further by implementing speech recognition and sending the spoken text to the OpenAI service. This is left to you as an exercise. If you implement this enhancement, remember to ask for microphone permissions.

Running the example

Press **F5** to start debugging the application, and when ready, open the new page. Enter a question in a conversational way and the application will show the response coming from the selected GPT model. Figure 28 shows an example based on the following question:

“Can you summarize in a short paragraph the top 10 places to visit in Seattle?”



Note: The result returned by the AI service varies depending on the model that you selected, on the request that you enter, and on the way the service itself elaborates your request.

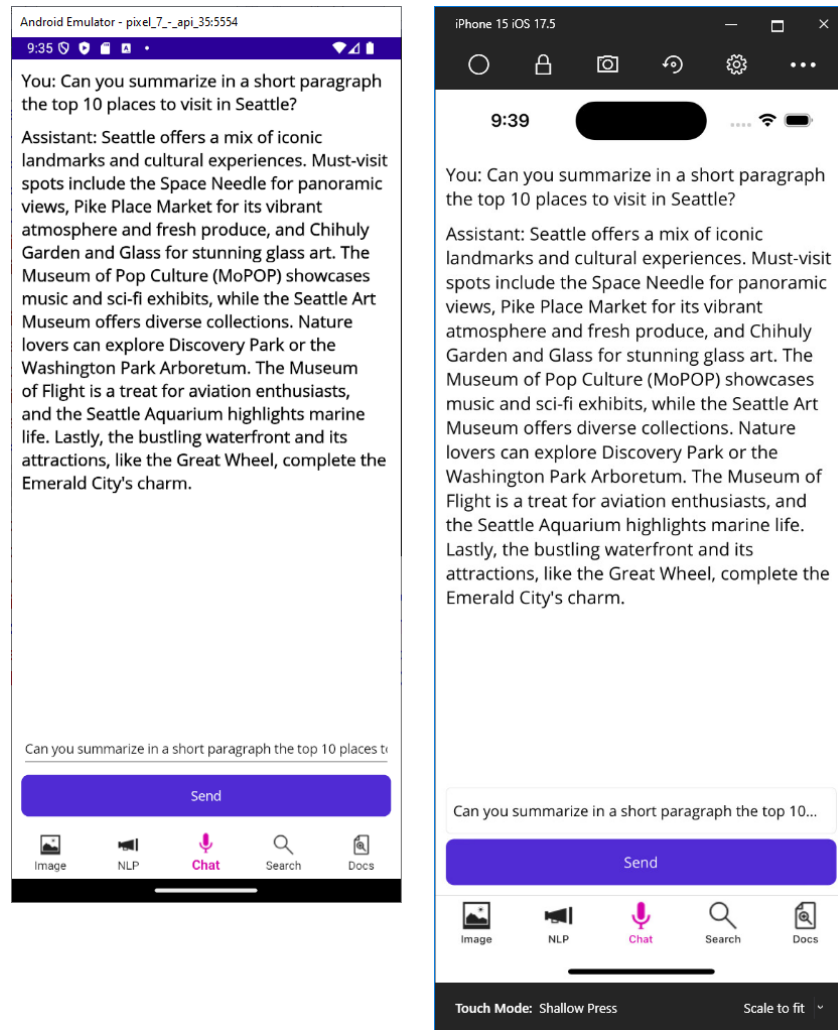


Figure 28: Implementing a conversational UI through a GPT model

Notice how the history of the conversation is added to the **CollectionView**. There are certainly many ways to improve this functionality, and most depend on the type of application that you are developing, but now you have a clear idea of how the Azure AI services can dramatically improve the user experience of a mobile app built with .NET MAUI, with limited effort.

Chapter summary

In this chapter, you explored three key NLP features: sentiment analysis, text translation, and chatbot functionality. You learned how to set up and configure Azure services, integrate them into a .NET MAUI app, and use the `Azure.AI.TextAnalytics`, `Azure.AI.Translation.Text`, and `Azure.AI.OpenAI` libraries. These features enhance the travel companion app by enabling language understanding and interaction capabilities. In the next chapter, you will learn how to implement AI-powered recommendations and personalization.

Chapter 6 Implementing AI-Driven Recommendations

In this chapter, you will learn how to integrate Azure AI Search to enhance the user experience to find relevant recommendations efficiently. You will incorporate Azure AI recommendations to personalize results based on user preferences. You will also learn how to implement a search schema and how to upload data to a search index.

Setting up the Search service

Similar to what you have done in previous chapters, you need to set up the Azure AI Search service in the Azure Portal. Follow these steps:

1. Go to the [Azure Portal](#) and sign in with your Azure account credentials.
2. Click **Azure AI Services**.
3. Click **Create** on the **AI Search** card (see Figure 29).
4. In the Create a search service page, choose your Azure subscription and specify the resource group you created previously.
5. Choose the **Region** closest to your location and enter a unique **Name**, such as searchsuccinctly.
6. In the **Pricing tier** dropdown, click the **Change Pricing Tier** hyperlink and then select the **Free** plan. This is necessary because the standard plan is assigned by default.
7. Click **Review + Create** and then **Create**.

When the summary page appears, take note of the service endpoint URL that will be used shortly and do not leave the page.

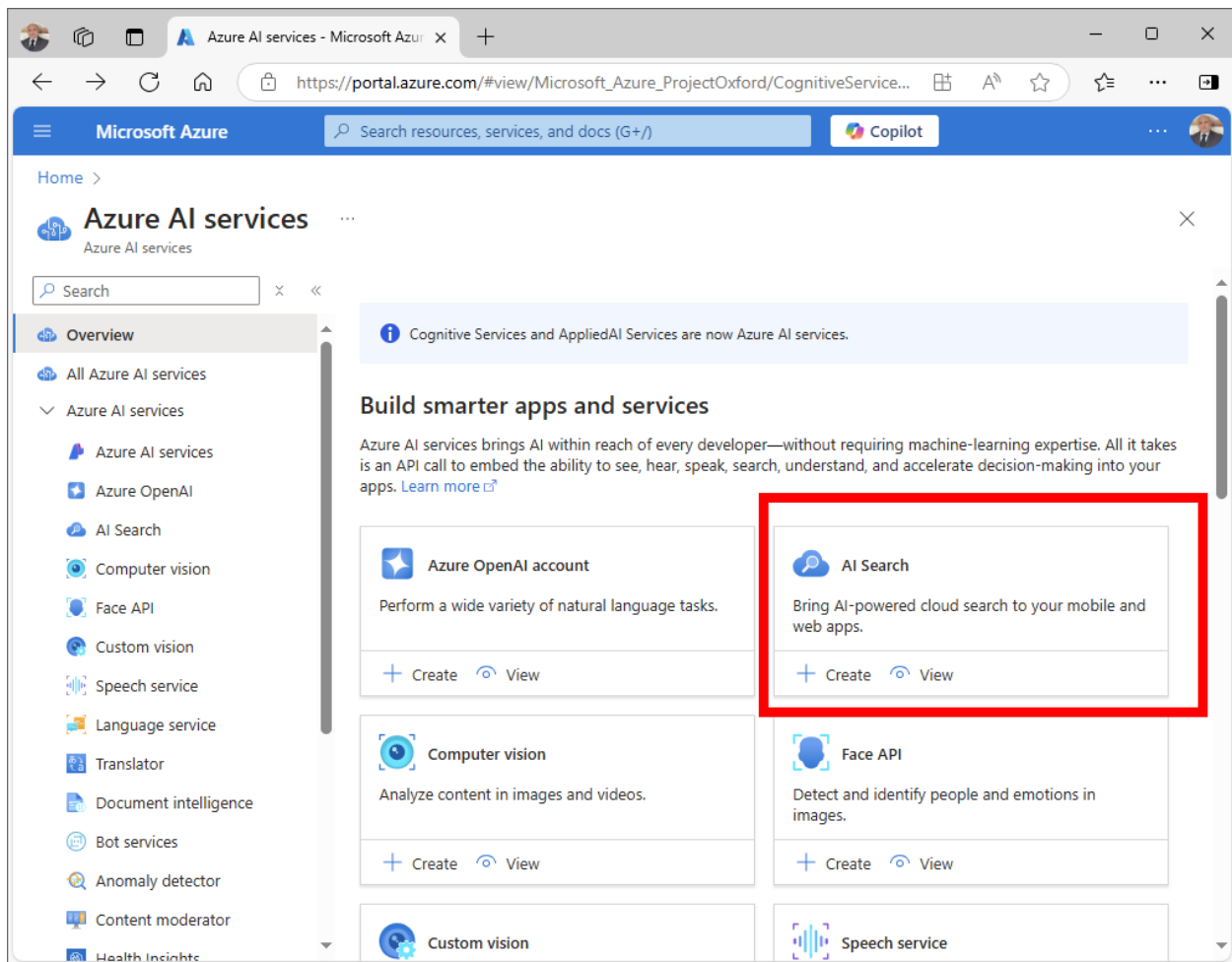


Figure 29: Locating the AI Search service

Defining a search schema

Before performing searches, you need to define a search schema that structures your data and defines how information will be indexed and searched. In the current example, the information will be travel destinations. Defining a search schema is done by creating indexes, and it is a crucial step in configuring Azure AI Search, as indexes determine how your data will be stored, searched, and retrieved. In Azure AI Search, an index is a collection of fields, each representing a piece of information within your searchable documents, such as titles, descriptions, or metadata. Each field has specific attributes, such as whether it is searchable, filterable, sortable, or facetable. In the toolbar at the top of the page, click **Add index**, and then **Add index** again. The **Create index** page appears and it looks like Figure 30.

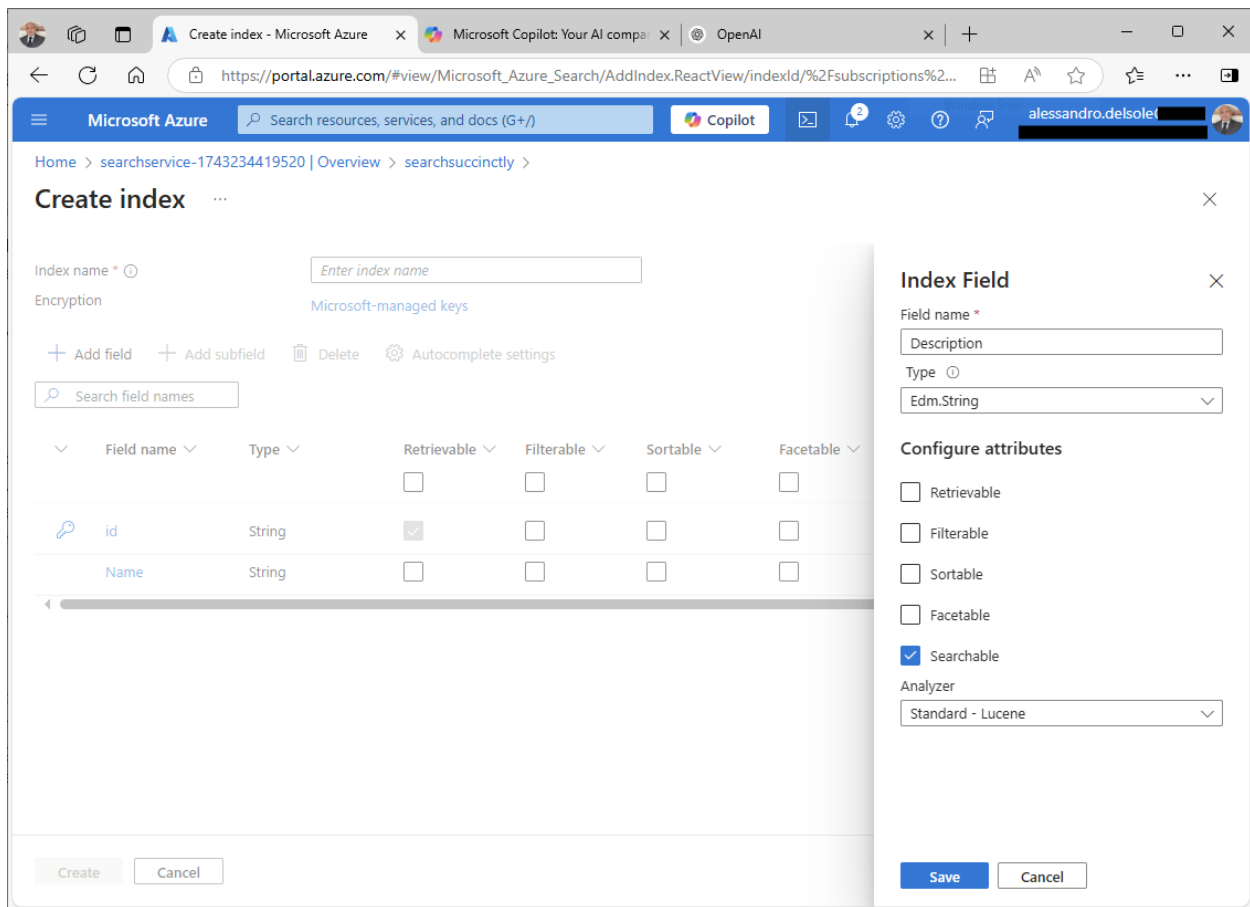


Figure 30: Creating a search index

Here, you have to define the index properties:

- **Index name:** A unique name for your index. This name will be used to reference the index in your queries, API calls, and C# code.
- **Fields:** The fields you define will represent the schema of your data within the index. Each field corresponds to a specific property or piece of information in your documents.

You add a new field by clicking **Add field**. The first piece of information you set for a field is the **Field name**, which represents the name of the field as it will appear in your index. Then you set the appropriate data type in the **Type** dropdown (see the right side of Figure 30), such as **Edm.String** for text, **Edm.Int32** for integers, **Edm.Double** for floating-point numbers, or **Edm.DateTimeOffset** for dates. For collections, you can select a collection type such as **Collection(Edm.String)**. You can also assign one or more of the following attributes:

- **Searchable:** Determines whether the field's content can be searched. Typically, you would enable this for text fields like **Name** or **Description**.
- **Filterable:** Allows filtering on the field's content. For example, you might want to filter travel destinations by rating.

- **Sortable:** Allows the results to be sorted based on this field.
- **Facetable:** Enables faceting, which is useful for generating search facets or categories based on the values in this field, such as showing counts or ranges.

Notice that one field must be designated as the key field, which serves as the unique identifier for documents in your index. By default, Azure generates a field called **id**. Every time you add and assign a field, click **Save**. Repeat the steps to add fields for the **Category** and **Price** properties, to be set with **Edm.String** and **Edm.Double** types, respectively. When finished, click **Save** on the index page. Indexes are very flexible and allow you to implement complex operations, but these are beyond the scope of this ebook. For further information, you can read the Azure [documentation about indexes](#). Supposing that you are implementing AI-driven search for recommended travel destinations, the sample schema consists of the following fields:

- **Name:** Of type **string**, this represents the destination name. Apply the **Searchable** attribute to this field so that users can search by destination name.
- **Description:** Of type **string**, this represents a brief description of the destination. Apply the **Searchable** and **Retrievable** attributes to this field. The **Retrievable** attribute ensures that the field is included in the returned search results.
- **Rating:** Of type **double**, this represents user ratings. Apply the **Filterable** and **Sortable** attributes to this field. These ensure that the user can filter and sort search results by ratings.

You will now retrieve the secret key required to connect your code to the search service.

Retrieving the admin key

Unlike other services where the API key is available in the service summary page, for the AI Search service, you need to expand the **Settings** node in the sidebar and then select **Keys**. In the AI Search service, the API keys are called admin keys, and you can copy them to the clipboard when the Keys page appears (see Figure 31).

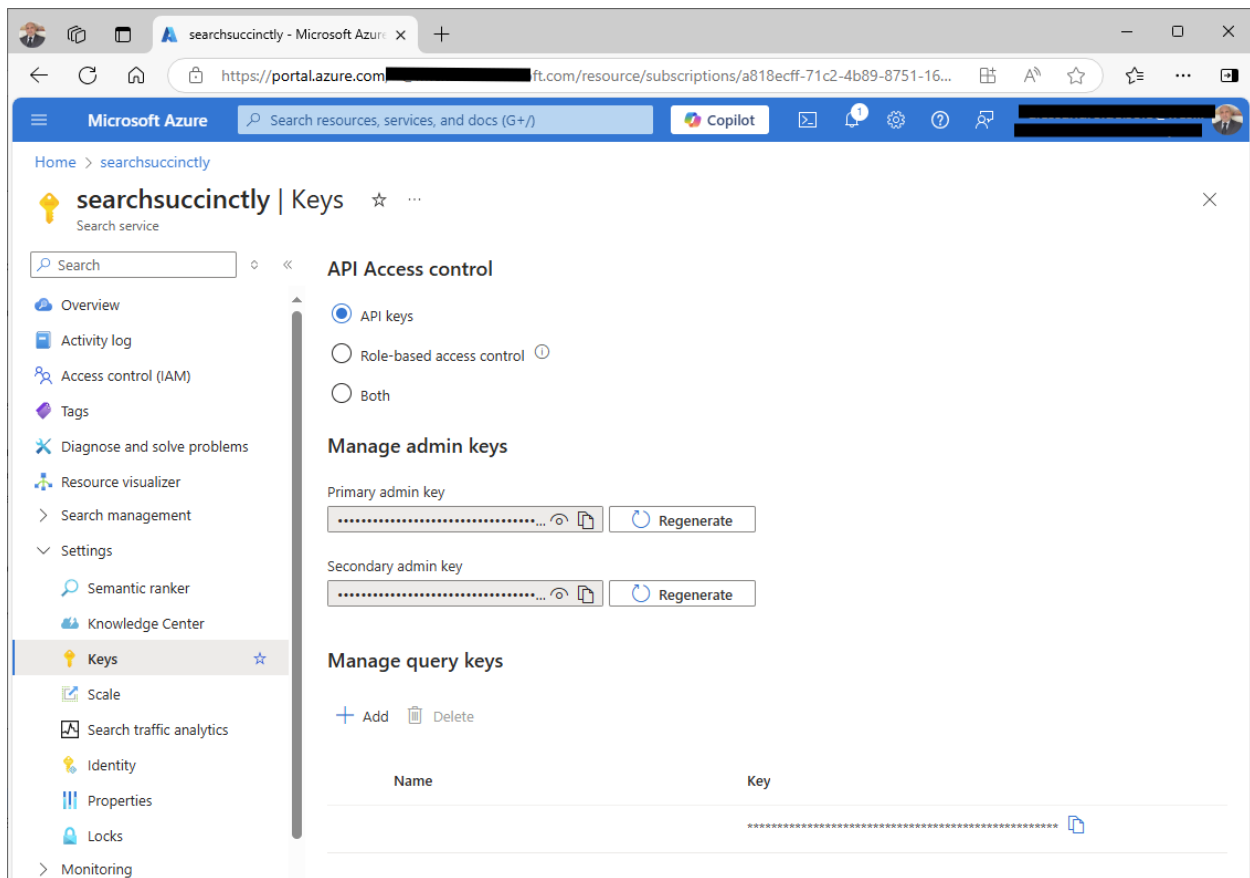


Figure 31: Retrieving the admin keys

Now you have everything you need, and you can start writing code.

Implementing AI Search

Go to Visual Studio 2022 and install the **Azure.Search.Documents** NuGet package in the solution. This is required to use the client .NET library for AI Search services. Figure 32 shows how the package appears in the NuGet user interface.

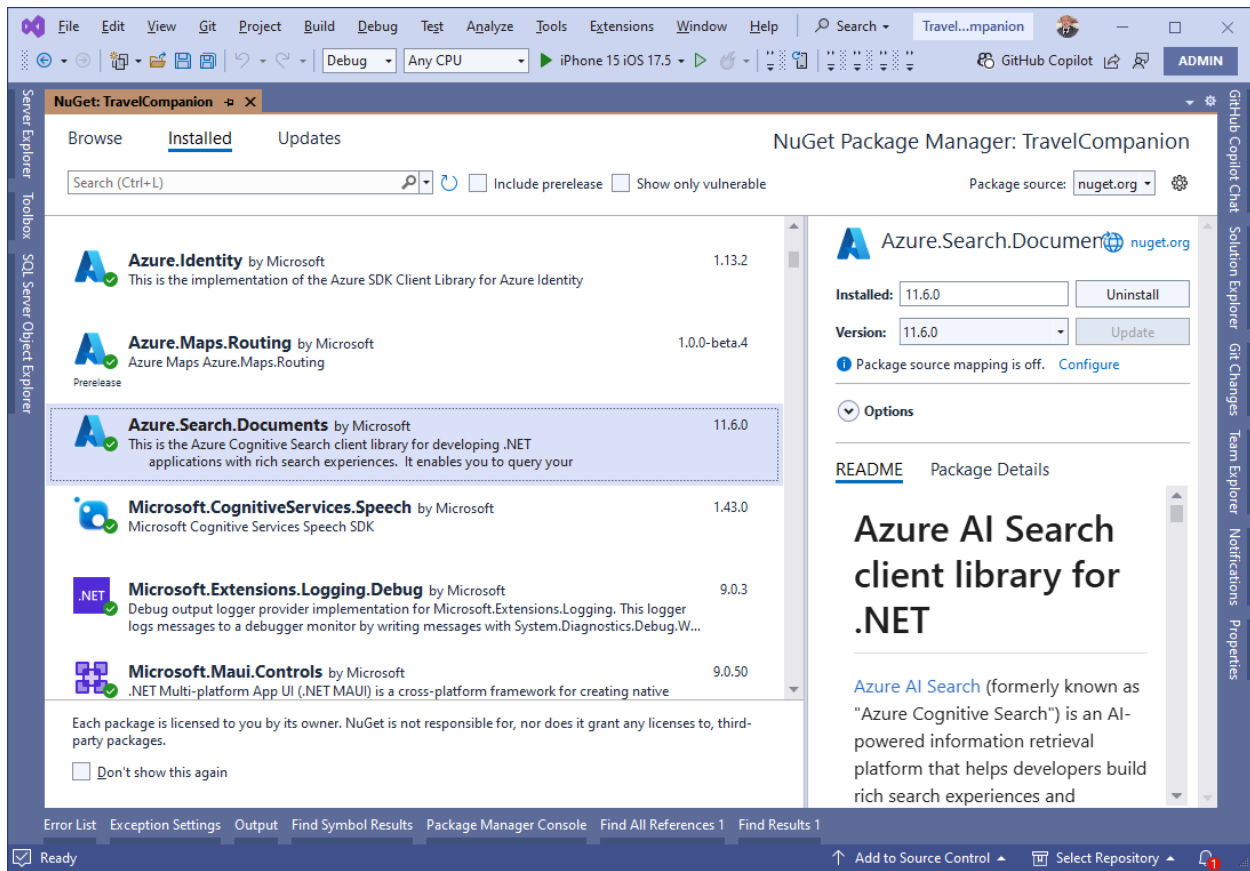


Figure 32: The AI Search NuGet package

When ready, add a new page named **SearchPage.xaml**, where you will define the user interface.

Defining the user interface

The user interface of the new page must include a search box, a button that launches the search operation, and a **CollectionView** that displays the search results in the form of a collection of travel destinations. In the SearchPage.xaml file, add the code shown in Code Listing 10.

Code Listing 10

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="TravelCompanion.SearchPage"
    Title="SearchPage">
    <Grid Padding="10" BindingContext="{Binding}">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
    </Grid>
</ContentPage>
```

```

        <RowDefinition Height="Auto"/>
        <RowDefinition />
    </Grid.RowDefinitions>
    <Entry x:Name="SearchEntry"
        Placeholder="Search destinations..."/>
    <Button Text="Search"
        Clicked="OnSearchClicked" Grid.Row="1"/>
    <CollectionView x:Name="ResultsList" Grid.Row="2"
        ItemsSource="{Binding}">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <VerticalStackLayout>
                    <Label Text="{Binding Name}"
                        FontSize="Medium"
                        FontAttributes="Bold"/>
                    <Label Text="{Binding Description}"
                        FontSize="Medium" />
                </VerticalStackLayout>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</Grid>
</ContentPage>

```

The **ItemTemplate** of the **CollectionView** represents how each search result will be presented, binding the two **Label** views to the **Name** and **Description** fields of the index that you defined previously, and that in C# are represented by a class called **TravelDestination**, defined as shown in Code Listing 11.

Code Listing 11

```

using Azure.Search.Documents.Indexes;
using System.ComponentModel;
using System.Runtime.CompilerServices;

public class TravelDestination : INotifyPropertyChanged
{
    private string _id;
    private string _name;
    private string _description;
    private double _rating;

    [SearchableField(IsKey = true)]
    public string Id
    {
        get => _id;
        set
        {

```

```

        if (_id != value)
        {
            _id = value;
            OnPropertyChanged();
        }
    }
}

[SearchableField]
public string Name
{
    get => _name;
    set
    {
        if (_name != value)
        {
            _name = value;
            OnPropertyChanged();
        }
    }
}

[SearchableField]
public string Description
{
    get => _description;
    set
    {
        if (_description != value)
        {
            _description = value;
            OnPropertyChanged();
        }
    }
}

[SimpleField(IsFilterable = true, IsSortable = true)]
public double Rating
{
    get => _rating;
    set
    {
        if (_rating != value)
        {
            _rating = value;
            OnPropertyChanged();
        }
    }
}

```

```

public event PropertyChangedEventHandler?
    PropertyChanged;

protected virtual void OnPropertyChanged(
    [CallerMemberName] string propertyName = null)
{
    PropertyChanged?.Invoke(this,
        new PropertyChangedEventArgs(propertyName));
}
}

```

The **TravelDestination** class represents an individual search result and is the .NET representation of the index created previously in the Azure Portal. It implements the **INotifyPropertyChanged** interface to raise a change notification so that the user interface can react to changes. Properties are decorated with attributes that represent the way you defined filters in the index. More specifically:

- **[SearchableField(IsKey = true)]** marks the **Id** field as the unique identifier.
- **[SearchableField]** allows the **Name** and **Description** fields to be searchable.
- **[SimpleField(IsFilterable = true, IsSortable = true)]** allows the **Rating** field to be filtered and sorted.

Now that you have a data model, you can implement the search logic.

Executing intelligent search in C#

The goal is to implement a search feature that allows users to look up destinations and retrieve recommendations based on their preferences. Code Listing 12 shows how to accomplish this, including how to upload sample data.

Code Listing 12

```

using Azure;
using Azure.Search.Documents;
using System.Collections.ObjectModel;

namespace TravelCompanion;

public partial class SearchPage : ContentPage
{
    private const string AdminKey = "your-admin-key";
    private const string Endpoint = "your-endpoint-url";
    private const string IndexName =
        "travel-destinations";
}

```

```

private readonly SearchClient searchClient;
public ObservableCollection<TravelDestination>
    SearchResults
{ get; set; } = new();

public SearchPage()
{
    InitializeComponent();
    var credential = new AzureKeyCredential(AdminKey);
    searchClient =
        new SearchClient(new Uri(Endpoint),
            IndexName, credential);
    this.BindingContext = SearchResults;
}

protected override async void OnAppearing()
{
    base.OnAppearing();

    bool destinationsUploaded =
        Preferences.Get("destinations", false);
    if (!destinationsUploaded)
        await UploadSampleDataAsync();
}

private async void OnSearchClicked(object sender, EventArgs e)
{
    var query = SearchEntry.Text;
    if (string.IsNullOrEmpty(query)) return;
    await PerformSearch(query);
}

private async Task PerformSearch(string query)
{
    try
    {
        var options = new SearchOptions
        { IncludeTotalCount = true };
        var response =
            await searchClient.
                SearchAsync<TravelDestination>(query, options);

        await foreach (var result in
            response.Value.GetResultsAsync())
        {
            SearchResults.Add(result.Document);
        }
    }
}

```



```

        catch (RequestFailedException ex)
        {
            await DisplayAlert("Error", ex.ErrorCode, "OK");
        }
        catch (Exception ex)
        {
            await DisplayAlert("Error", ex.Message, "OK");
        }
    }

    public async Task UploadSampleDataAsync()
    {
        try
        {
            {
                var destinations = new List<TravelDestination>
                {
                    new TravelDestination { Id = "1",
                        Name = "Paris",
                        Description = "The city of love.",
                        Rating = 4.8 },
                    new TravelDestination { Id = "2",
                        Name = "Rome",
                        Description = "The Eternal City.",
                        Rating = 4.7 }
                };

                await searchClient.UploadDocumentsAsync(destinations);
                Preferences.Set("destinations", true);
            }
            catch (RequestFailedException ex)
            {
                await DisplayAlert("Error", ex.ErrorCode, "OK");
            }
            catch (Exception ex)
            {
                await DisplayAlert("Error", ex.Message, "OK");
            }
        }
    }
}

```

The following is an explanation of the .NET objects and members that are relevant to AI Search in the previous code:

- The **SearchClient** class provides a way to interact with a specific search index in Azure AI Search. It allows performing search queries and managing documents within that index. The **SearchClient** object is instantiated with the service endpoint (**Uri**), index name (**string**), and **AzureKeyCredential** for authentication. In the code, the **UploadDocumentsAsync<T>** method is used to asynchronously upload a batch of test travel destinations (in this case, **TravelDestination** objects) to the index, otherwise you would have no results. This method returns a **Response<IndexDocumentsResult>** object, which contains information about the indexing operation. The **SearchAsync<T>** method is used to execute search queries asynchronously, returning a **SearchResults<T>** object that contains the search results. Additionally, methods like **MergeDocumentsAsync<T>** and **DeleteDocumentsAsync<T>** allow updating and removing documents from the index, respectively, and could be useful for scenarios where you need to modify or delete indexed data.
- The **SearchIndexClient** class enables you to manage the structure of your search indexes. Although it is not heavily used in the code, this client is essential for creating, updating, and deleting indexes. The client is instantiated similarly to **SearchClient**, with the service endpoint and **AzureKeyCredential**. The **CreateIndexAsync** method can be used to programmatically create a new search index, while **DeleteIndexAsync** can be used to delete an existing index. The **GetIndexAsync** method retrieves the schema of an index, which can be useful for inspecting or modifying the structure.
- The **SearchOptions** class configures the behavior of search queries performed with the **SearchClient**. In the code, **SearchOptions** is used to specify that the total count of results should be included in the search response (**IncludeTotalCount = true**). Other available properties include **Filter**, which can be used to apply filters to the search query, and **Facets**, which allows you to specify fields for faceted navigation. Additionally, **OrderBy** can be set to control the sort order of the search results.
- The **SearchResults<T>** class represents the generic collection of search results returned by the **SearchAsync<T>** method. It includes properties such as **TotalCount**, which gives the total number of documents that match the search query, and **Results**, which contains a collection of **SearchResult<T>** objects. The **GetResultsAsync** method can be used to retrieve the collection of individual search results. This class encapsulates not only the results themselves but also metadata about the query, which can be useful for building pagination or providing more detailed insights into search performance.

- The **SearchResult<T>** class represents an individual document retrieved from the search index. Each **SearchResult** object includes properties like **Score**, which represents the relevance score assigned by the search engine, and **Highlights**, which contains the highlighted portions of the document that match the query, if highlighting is enabled. The **Document** property gives access to the actual document (in this case, a **TravelDestination** object) stored in the index. The **SearchResult<T>** class also exposes methods like **GetDouble**, **GetString**, and other type-specific accessors to retrieve individual fields from the result when working with dynamic schemas.
- The **IndexDocumentsResult** object is returned by document indexing methods like **UploadDocumentsAsync<T>** and **MergeDocumentsAsync<T>**. This object contains information about the result of the indexing operation, such as the status and number of documents successfully processed. It helps in verifying whether the documents were indexed properly and if there were any errors.



Tip: Uploading test data is necessary in a sample application, but in the real world, you will need to populate the index with your own data. There are different ways to do this; for example, you can upload JSON data directly to the Azure Portal.

The search functionality is triggered when the user clicks the Search button. The application retrieves the user's query from the **Entry** and sends it to Azure. Matching results are added to the **SearchResults** collection. Further information on developing with Azure AI Search can be found in the [official documentation](#).

Running the application

Add the new page to the AppShell.xaml file as follows:

```
<ShellContent Title="Search" Icon="magnifying_glass_solid.png"
              ContentTemplate="{DataTemplate local:SearchPage}" />
```

After doing this, press **F5** to start debugging the app on your device of choice and open the **Search** page. Enter your query (e.g., Rome) and click **Search**. As shown in Figure 33, the application displays the search results.

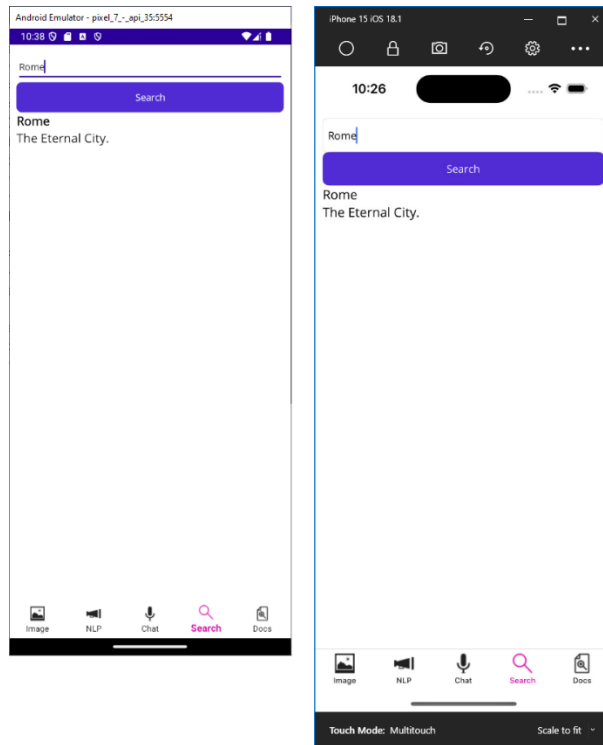


Figure 33: Retrieving items with intelligent search

This is obviously a simple example, but it should at least give you an idea of Azure AI Search's powerful features.

Implementing recommendations

Azure AI Search can also return AI-generated recommendations. The following method finds highly rated destinations based on the filters applied to the index fields:

```
public async Task<List<TravelDestination>>
GetRecommendedDestinationsAsync()
{
    var options = new SearchOptions
    {
        Filter = "Rating ge 4.7",
        OrderBy = { "Rating desc" }
    };

    var searchResults =
        await searchClient.SearchAsync<TravelDestination>("", options);
    var recommendedList =
        new List<TravelDestination>();

    await foreach (var result in
        searchResults.Value.GetResultsAsync())
    {
```

```
        recommendedList.Add(result.Document);  
    }  
  
    return recommendedList;  
}
```

You still invoke the **SearchAsync** method from the **SearchClient** class, but you configure the **SearchOptions** instance by setting the **Filter** and **OrderBy** properties.

Chapter summary

In this chapter, you integrated Azure AI Search to power search functionality in the travel companion app. You created a search index, implemented a search UI, and retrieved relevant destinations based on user queries. Additionally, you implemented ranking and filtering to personalize recommendations. The use of the **SearchClient** class and the **SearchAsync** method allowed the app to perform efficient searches. With this foundation, the app can now provide users with intelligent search and recommendation features. In the next chapter, you will focus on extracting data from documents using AI, enabling users to process text-based information effectively.

Chapter 7 Extracting Data from Documents with AI

In this chapter, you will learn how to use Azure AI Document Intelligence to extract structured data from invoices, receipts, and IDs in your .NET MAUI apps. This allows you to automate document processing, reducing manual effort and improving efficiency. More specifically, you will set up the Azure Form Recognizer service, then you will create a sample invoice for testing, and finally use AI to extract structured data from an invoice in your .NET MAUI app.

Setting up the Azure AI Document Intelligence service

You will now follow steps similar to those in the previous chapters to set up Azure AI Document Intelligence:

1. Open the [Azure Portal](#).
2. Click **AI Services** and locate the **Document intelligence** service (see Figure 34).
3. Click **Create** on the service's card.
4. Choose a **Subscription** and **Resource group**.
5. Select a **Region** and **Pricing tier** (the Free F0 plan is recommended).
6. Specify a name for the service, for example, documentsuccinctly.
7. Click **Review + Create** and then **Confirm**.
8. When the service has been created, click **Go to resource** and retrieve the endpoint and API key.

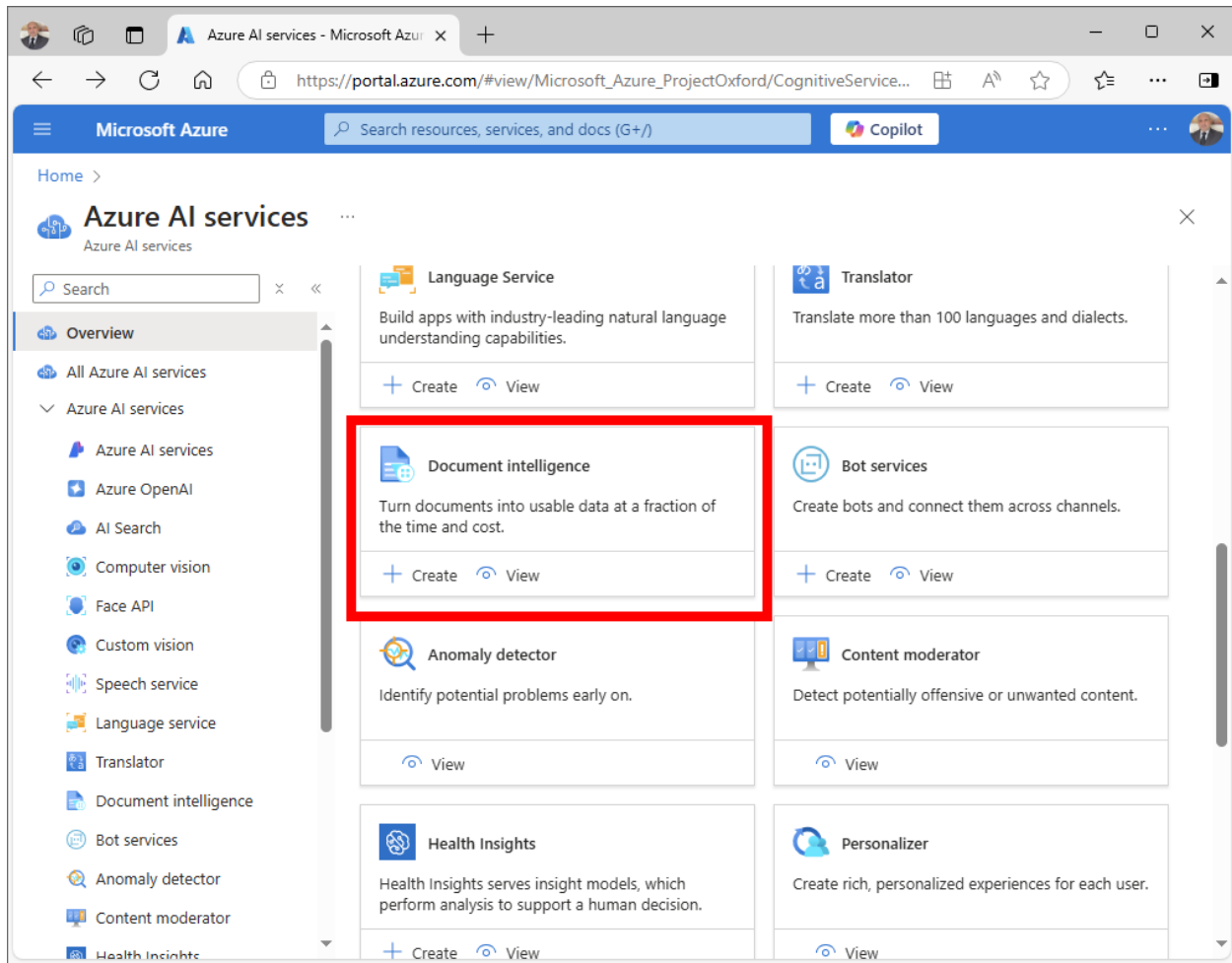


Figure 34: Locating the Document Intelligence service

Now that you have set up the service, it is time to create a sample document to analyze.

Creating a sample invoice

The goal of the example is to analyze an invoice stored inside a PDF document and extract the information as structured data that the mobile app can consume. As you will learn shortly, the Azure AI Document Intelligence service provides pretrained models that you can use to analyze the most common types of documents. You can also create custom models, but this is not discussed in this chapter. That said, use your word processor of choice and create the following document:

Invoice #: INV-2345
Invoice Date: March 31, 2025
Due Date: April 30, 2025
Bill To:
Alessandro Del Sole

123 Main St
Seattle, WA 98000
Item Description Amount

Hotel room for 6 nights \$800.00
Taxes \$100.00

Total Amount Due: \$900.00

Save the document as **sample-invoice.pdf** in the **Resources\Raw** folder of the sample app. Make sure that the **Build Action** property of the file is set as **MauiAsset**.

Installing NuGet packages

In the sample project, open the NuGet Package Manager, search for the **Azure.AI.FormRecognizer** package, and make sure you install the latest version. Figure 35 shows how the package appears in the NuGet user interface.

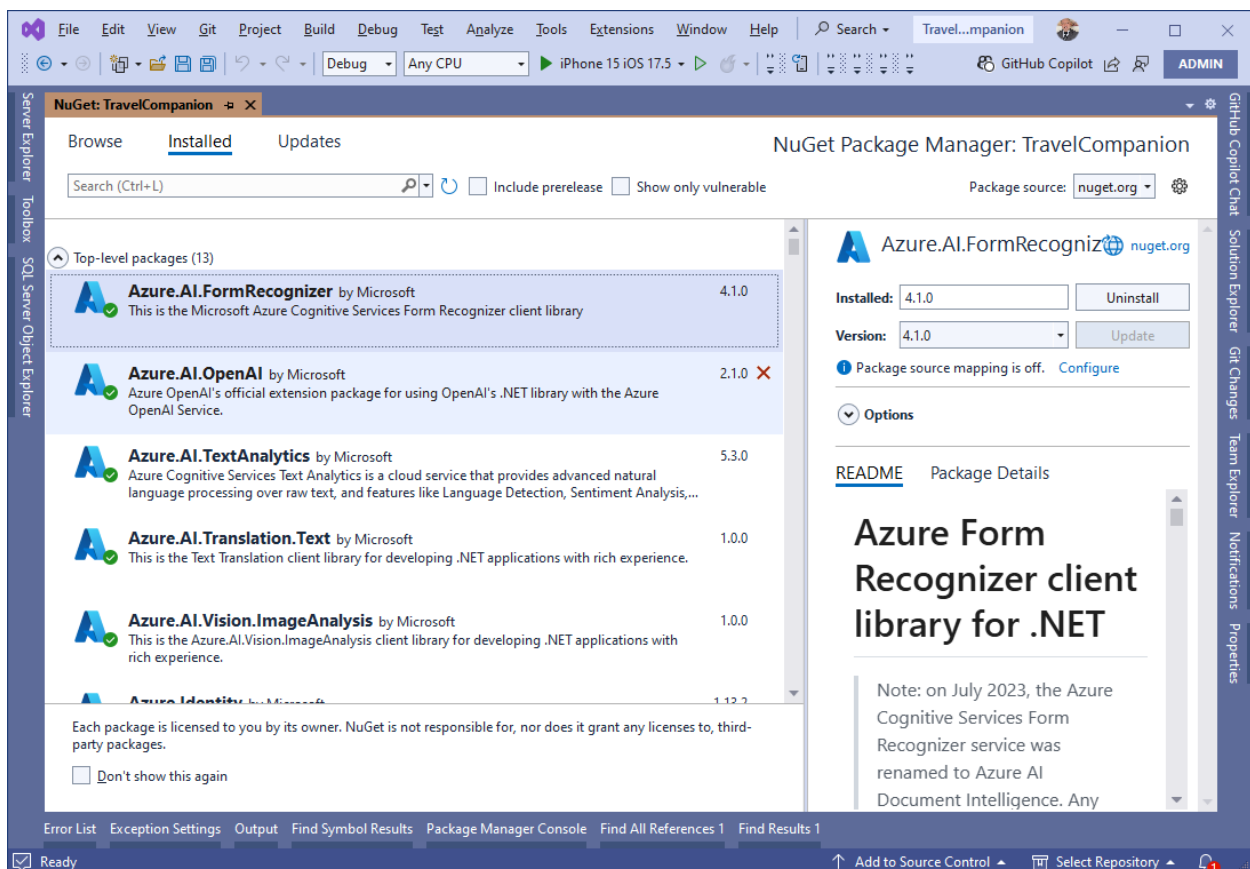


Figure 35: The Document Intelligence NuGet package

This package contains the Document Intelligence client library for .NET that you will use to extract document data.

Defining the user interface

Add a new page called `DocProcessorPage.xaml` to the project. When ready, add the XAML code shown in Code Listing 12, which defines a simple user interface made of a **Button** that triggers the analysis of the sample invoice and a **Label** that displays the result.

Code Listing 12

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="TravelCompanion.DocProcessorPage"
              Title="DocProcessorPage">
  <Grid BindingContext="{Binding}">
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition />
    </Grid.RowDefinitions>

    <Button Text="Analyze invoice" x:Name="AnalyzeButton"
            Clicked="AnalyzeButton_Clicked" Margin="10"
            HorizontalOptions="Center"/>

    <Label x:Name="TxtInvoiceData"
           FontSize="Medium" Grid.Row="1"
           Margin="10" LineBreakMode="WordWrap"/>
  </Grid>
</ContentPage>
```

The next step is implementing the analysis logic in C#.

Extracting document data in C#

The goal of the example is to analyze the content of the sample PDF invoice created previously, extract its information as structured data, and display the result in the app. For the sake of simplicity, each piece of data will be concatenated into a string, but you will easily understand how to consume data differently. Code Listing 13 demonstrates how to accomplish this.

Code Listing 13

```
using Azure;
using Azure.AI.FormRecognizer.DocumentAnalysis;
using System.Reflection;
```

```

namespace TravelCompanion;

public partial class DocProcessorPage : ContentPage
{
    private readonly DocumentAnalysisClient _client;

    public DocProcessorPage()
    {
        InitializeComponent();

        var endpoint =
            "you-endpoint-url";
        var credential =
            new AzureKeyCredential("your-api-key");
        _client = new DocumentAnalysisClient(
            new Uri(endpoint), credential);
    }

    private async Task ExtractInvoiceData(string filePath)
    {
        try
        {
            using var stream = new FileStream(filePath, FileMode.Open);
            AnalyzeDocumentOperation operation =
                await _client.AnalyzeDocumentAsync(
                    WaitUntil.Completed, "prebuilt-invoice", stream);

            AnalyzeResult result = operation.Value;

            string invoiceNumber =
                result.Documents[0].Fields["InvoiceId"].Content;
            string totalAmount =
                result.Documents[0].Fields["AmountDue"].Content;
            string dueDate =
                result.Documents[0].Fields["DueDate"].Content;

            TxtInvoiceData.Text =
                $"Invoice Number: {invoiceNumber}\nTotal Amount: " +
                $"{totalAmount}\nDue Date: {dueDate}";
        }
        catch (RequestFailedException ex)
        {
            await DisplayAlert("Error", ex.ErrorCode, "OK");
        }
        catch (Exception ex)
        {
            await DisplayAlert("Error", ex.Message, "OK");
        }
    }
}

```

```

    }

    public Stream GetInvoicePdfStream()
    {
        var assembly = Assembly.GetExecutingAssembly();
        var resourceName =
            "TravelCompanion.Resources.Raw.sample-
            invoice.pdf"; // Adjust namespace

        return assembly.GetManifestResourceStream(resourceName);
    }

    private async void AnalyzeButton_Clicked(object sender, EventArgs e)
    {
        string fileName =
            Path.Combine(FileSystem.CacheDirectory,
                "sample_invoice.pdf");

        if (!File.Exists(fileName))
        {
            using Stream resource =
                await FileSystem.
                    OpenAppPackageFileAsync("sample_invoice.pdf");
            using FileStream outputStream =
                File.Create(fileName);
            await resource.CopyToAsync(outputStream);
        }

        await ExtractInvoiceData(fileName);
    }
}

```

The following is an explanation of the .NET objects used in the code to interact with Azure AI Document Intelligence:

- The **DocumentAnalysisClient** class is the core client used to interact with Azure AI Document Intelligence. This class facilitates document analysis by enabling the submission of documents to prebuilt models or custom models. In the current example, the **AnalyzeDocumentAsync** method is invoked to analyze an invoice. Additionally, this client provides other methods like **StartAnalysisAsync** for handling larger documents in a more granular way. The constructor requires the service endpoint and an instance of **AzureKeyCredential** to authenticate API requests.

- The **AnalyzeDocumentOperation** class represents an asynchronous operation that processes documents using a specific model. In the sample code, it is used to handle the document analysis request, waiting for the operation to complete with **await**. **AnalyzeDocumentOperation** can be polled periodically for long-running document processing tasks, which is particularly useful when working with documents that contain multiple pages or complex layouts. While the code invoked **AnalyzeDocumentAsync** for a straightforward analysis, other methods in this class, like **GetDocumentResult**, allow retrieving the analysis results directly, which can be useful when integrating with workflows that need finer control over document processing.
- The **AnalyzeResult** object holds the result of a document analysis. It contains extracted content, such as key-value pairs, tables, and text. The **Documents** property is of type **IReadOnlyList<AnalyzedDocument>**, which represents the analyzed documents with fields and their corresponding values. Besides basic extraction, the **AnalyzeResult** class supports complex scenarios. For instance, if your document contains tables, you can access them via the **Tables** property, which provides structured table data. The **Pages** property also provides detailed information about the layout of each page, such as lines of text, selection marks, and bounding boxes, making it easier to implement custom rendering or export logic based on the extracted data.
- Each field extracted from the document is represented by the **DocumentField** class. This class provides access to the field's content, confidence score, and bounding box (if available). In our example, we use the **Content** property to extract the recognized values, such as invoice numbers and total amounts. Depending on the type of field, the **DocumentField** class has several properties that provide strongly typed access to data, such as **ValueType**, **ValueString**, **ValueDate**, and **ValueCurrency**. These specialized properties allow type-safe data extraction, reducing the need for manual parsing and enhancing the reliability of the extracted information.

Notice how the **Fields** property requires specifying a conventional identifier that the AI service can use to detect the various document parts quickly, such as **InvoiceId**, **AmountDue**, and **DueDate**. This is based on the pretrained invoice model (**prebuilt-invoice**). The full list of conventional identifiers is available in the [official documentation](#), where you will also find the identifier of the other prebuilt models.

Running the application

You can finally run the application by pressing **F5**. If you click **Analyze invoice**, the result of the document analysis will be displayed, as shown in Figure 36.

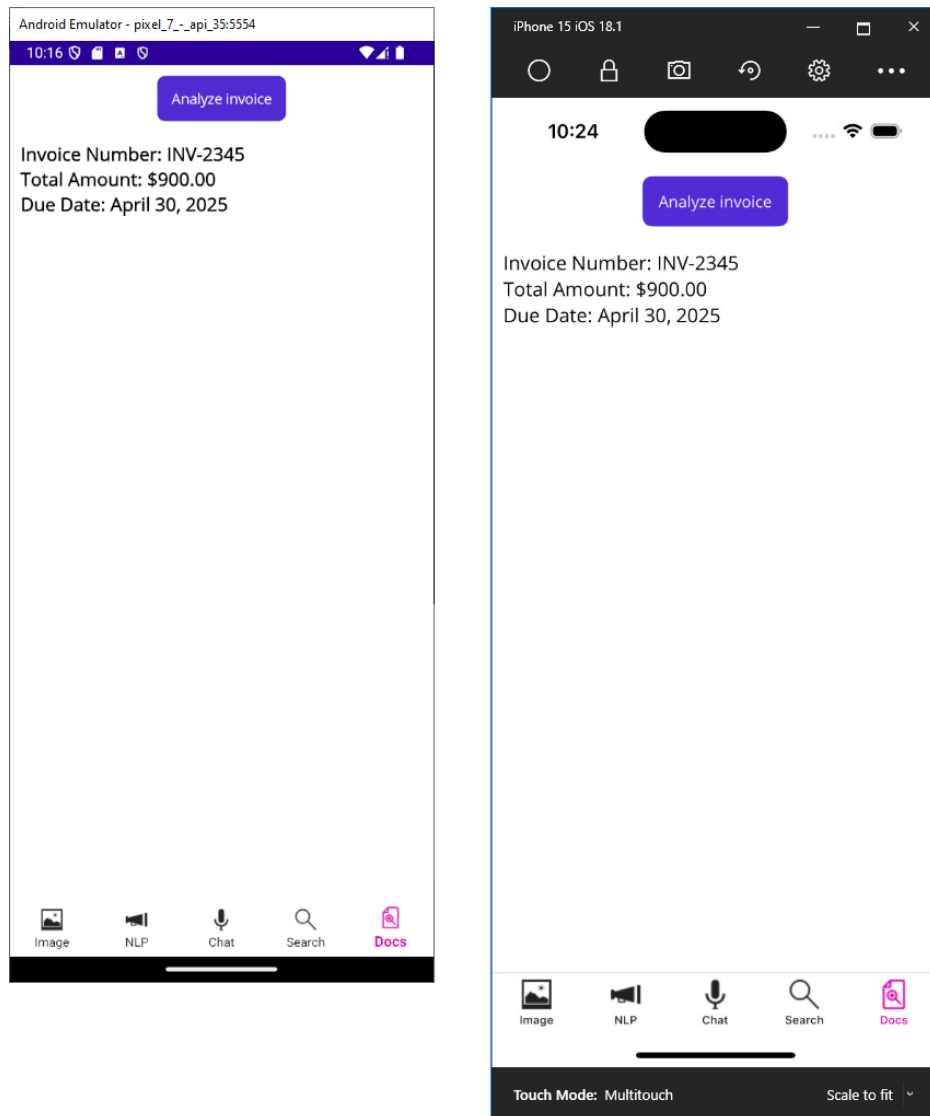


Figure 36: Analyzing documents and data extraction

The key concept is that the displayed information is not just text, it is data that can be stored inside a .NET object for further manipulation.

Chapter summary

In this chapter, you learned how to use Azure AI Document Intelligence to extract structured data from invoices and how to consume such data in a .NET MAUI project. You set up the Azure resource, created a sample invoice, and implemented a solution to analyze and display document data. This automation can streamline financial workflows, reducing manual data entry. In the next chapter, you will explore AI-optimized travel routes by integrating the Azure Maps Accounts service into your mobile apps.

Chapter 8 AI-Optimized Routes with Azure Maps Accounts

Azure Maps Accounts, or Azure Maps for short, is a comprehensive suite of geospatial services designed to empower developers and organizations to create intelligent, location-based applications. It allows the integration of maps, geocoding, routing, real-time traffic, weather data, and geospatial analytics into web and mobile solutions. In this chapter, you will discover how Azure Maps can improve the user experience by providing route optimization in mobile apps. You will implement real-time route data, such as traffic conditions, weather updates, and user preferences. These optimized routes are then displayed seamlessly on a map using .NET MAUI, providing a user-friendly interface.

Setting up the Azure Maps service

You will now follow steps similar to those in the previous chapters to set up the Azure Maps service:

1. Open the [Azure Portal](#).
2. In the search box, type **Azure Maps Accounts**, as shown in Figure 37. This is required the first time, because this service might not be visible in the AI services list.
3. Click the **Create resource** hyperlink.
4. Choose a **Subscription** and **Resource Group**.
5. Select a **Region** and **Pricing tier** (the Free F0 plan is recommended).
6. Specify a name for the service, for example, mapssuccinctly.
7. Click **Review + Create** and then **Confirm**.
8. When the service has been created, click **Go to resource** and retrieve the API key. No endpoint is necessary with this service.

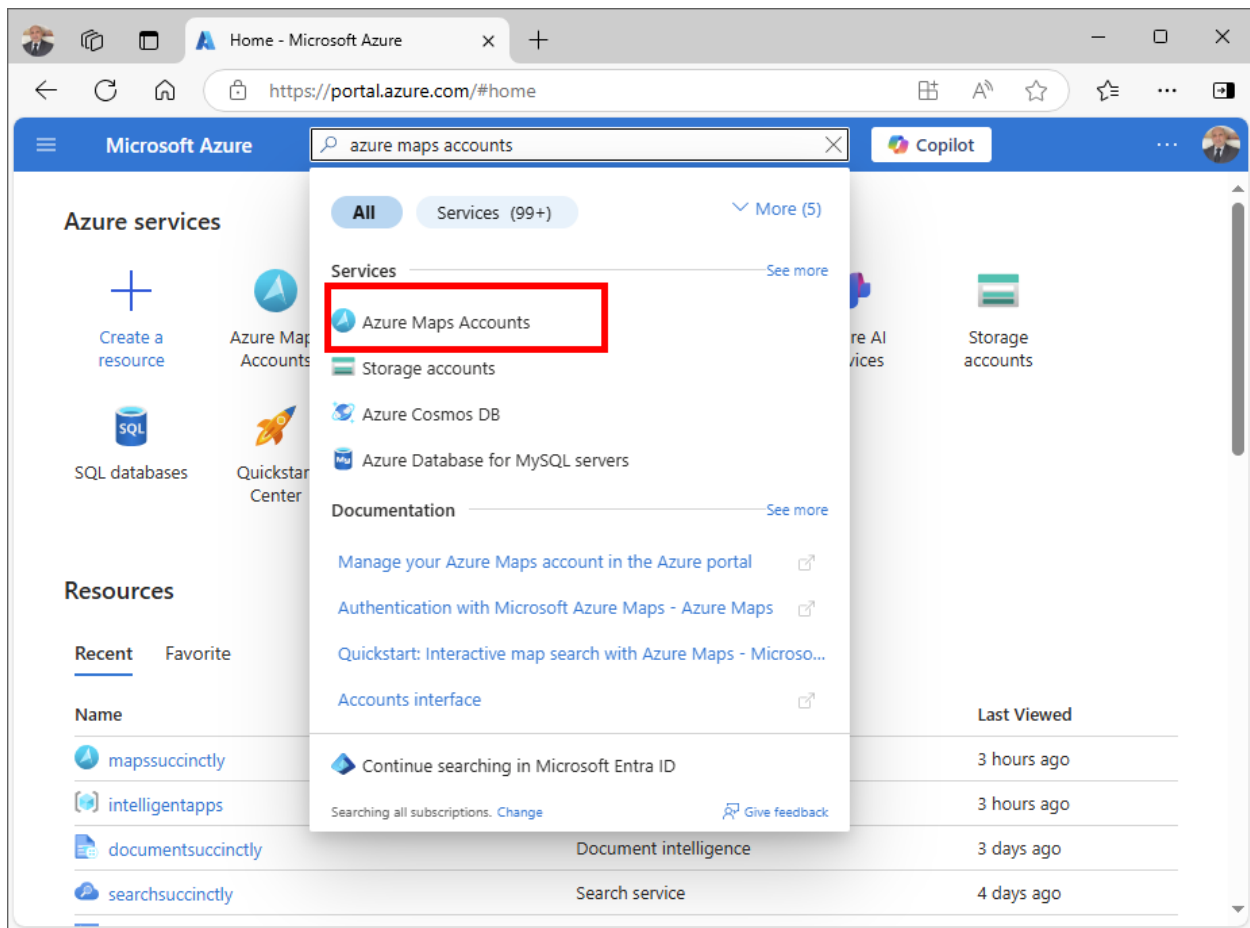


Figure 37: Locating the Azure Maps Accounts service

For iOS, this is enough. However, on Android, the .NET MAUI **Map** control relies on the native Google Maps component, so a few more steps are necessary.

Setting up the Google Maps SDK

Before you can use maps on Android, you need to register for an account on the [Google Cloud Developer Console](#). There is a 90-day free trial that includes \$300 in free credits and about 40,000 calls to the Maps service, which is absolutely enough for the purposes of the current example. So, you have all the time to decide and evaluate your needs. On the Developer Console home page, click **Try for free**. You will be asked to sign in with a Google account. Once signed in, you will be asked to:

1. Specify your country.
2. Select the service that you intend to use. Make sure you select **Google Maps**.
3. Verify your identity by providing a credit card (see Figure 38). Do not worry, you will not be charged, and a transaction of \$0 will be made only to avoid abuse. Charges start only when you decide to continue with a paid subscription.

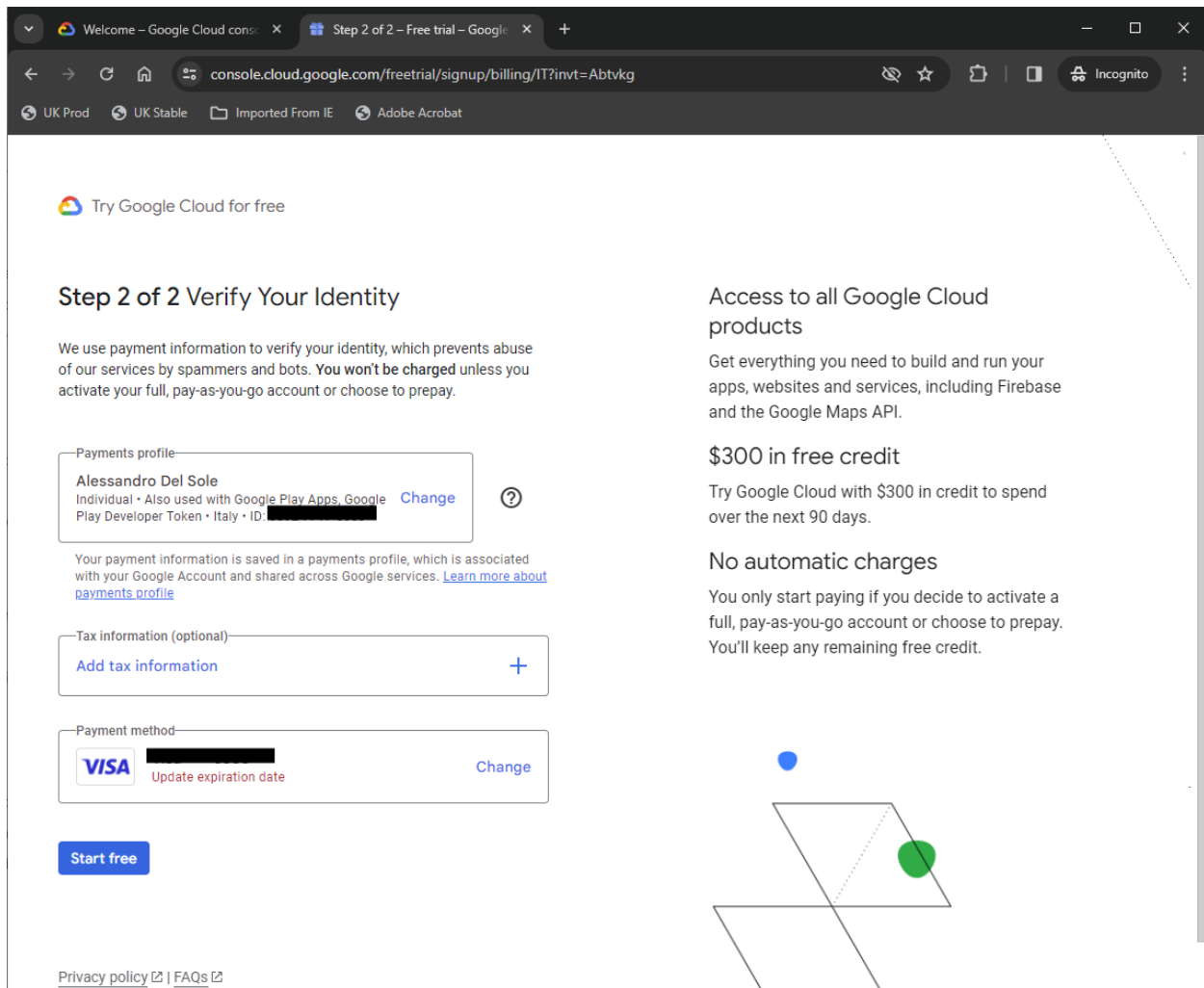


Figure 38: Verifying your identity in the Google Cloud Developer Console

When ready, click **Start free**. Since you previously specified Google Maps as the service that you want to use, the Google Cloud Developer Console will provide you with an API key on the next page, as shown in Figure 39.

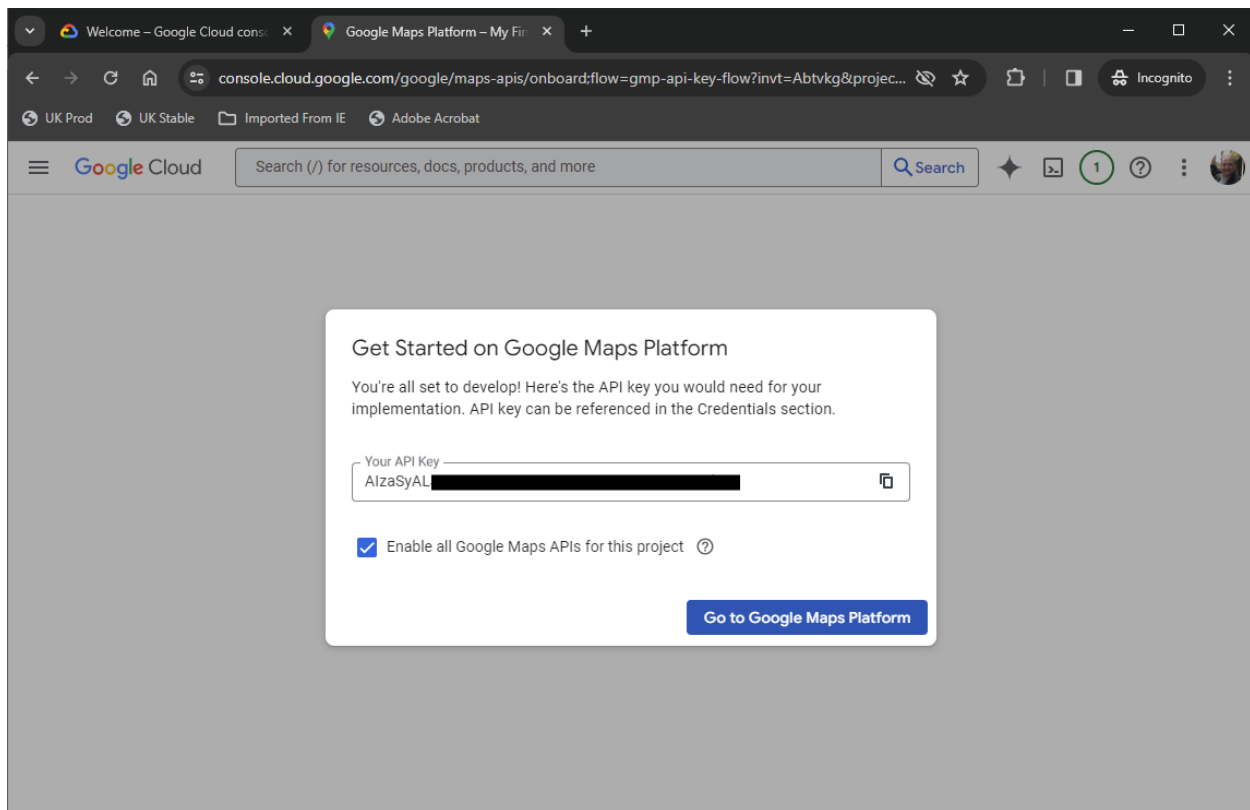


Figure 39: Obtaining a Google Maps API key

Now go to Visual Studio and open the **AndroidManifest.xml** file with the built-in XML editor, and then add the following line inside the **<application>** node:

```
<meta-data android:name="com.google.android.geo.API_KEY" android:value="your-google-maps-api-key" />
```

Enter the API key for Google Maps by replacing the content of the **android:value** tag. There is nothing else you have to do here, and you can ignore the Go to Google Maps Platform button in the Google Cloud Developer Console.

Implementing Maps

In Visual Studio, you need to install two NuGet packages:

- **Microsoft.Maui.Controls.Map**, which contains the **Map** control that you will use to display native maps.
- **Azure.Maps.Routing**, the client SDK library that allows interaction with the Azure Maps service from .NET. At the time of writing this ebook, the package is available as a prerelease version, so you need to select the **Include prerelease** option shown in Figure 40.

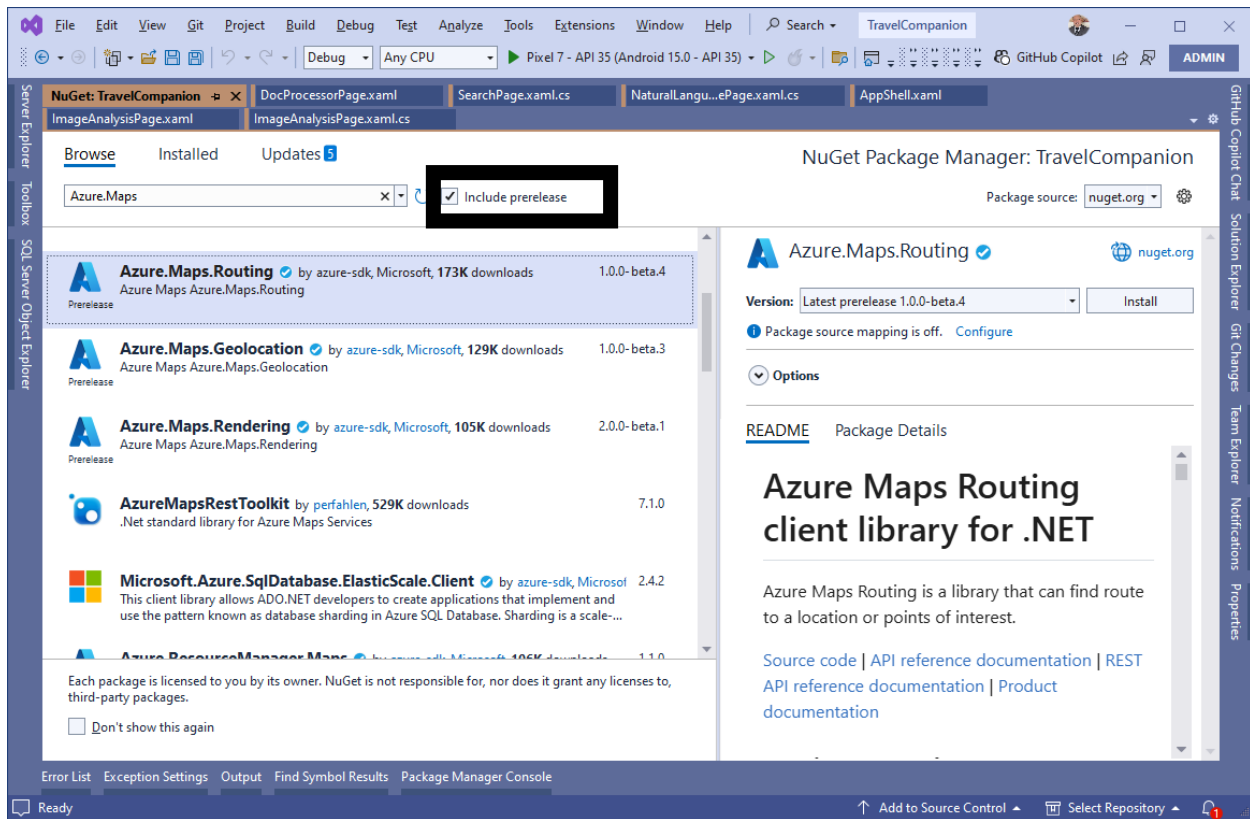


Figure 40: Installing the Azure.Maps.Routing NuGet package

Specifying permissions

Though displaying maps does not require user permissions, if you want to calculate a route from the user's current location, you need access to the GPS sensor, so you must ask for location permissions. This involves requesting permission to use the location only when the app is in use or in the background (if you really plan to use the location constantly). On iOS, you need to add the following nodes to the Info.plist file:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>We need your location to calculate the best route.</string>
<key>NSLocationAlwaysUsageDescription</key>
<string>This app requires location access to function properly.</string>
```

This ensures that both scenarios, while the app is in use and in the background, are covered. On Android, you might want to edit the **AndroidManifest.xml** file manually instead of using the user interface, because you need to specify hardware interaction as follows:

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
```

```

<uses-feature android:name="android.hardware.location"
android:required="true" />
<uses-feature android:name="android.hardware.location.gps"
android:required="true" />
<uses-feature android:name="android.hardware.location.network"
android:required="true" />

```

The first three lines ensure that the user grants permission to the app to use a low-accuracy location, a high-accuracy location, and the location while the app is in the background. The other lines specify which hardware features are required, namely the location service, the GPS sensor, and the location based on a network. For Android, there is an additional step; open the **MainActivity.cs** file in the **Platforms\Android** folder and add the following code:

```

protected override void OnCreate(Bundle? savedInstanceState)
{
    base.OnCreate(savedInstanceState);
    if (CheckSelfPermission(Manifest.Permission.AccessFineLocation)
        != Permission.Granted ||
        CheckSelfPermission(Manifest.Permission.AccessCoarseLocation)
        != Permission.Granted)
    {
        RequestPermissions(new string[] {
            Manifest.Permission.AccessFineLocation,
            Manifest.Permission.AccessCoarseLocation }, 0);
    }
}

```

When the main activity (that is, the main UI container of the app) is created, this code invokes the native **CheckSelfPermission** method to ensure that the user is prompted to grant location permissions.

Defining the user interface

Add a new page called **MapsPage.xaml** to the project. The flow of the user interface will be the following:

1. The user selects a destination on the map with a touch gesture on the target location.
2. The user then taps the only **Button** to enable the detection of the current location and to invoke the Azure Maps services to calculate an optimized route.
3. The results, including the detailed directions, will be displayed in a **Label1**.

Code Listing 14 declaratively defines this flow.

Code Listing 14

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"

```

```

        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps"
        x:Class="TravelCompanion.MapsPage"
        Title="MapsPage">
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="200" />
    </Grid.RowDefinitions>
    <Button Text="Get Current Location and Plan Route"
        Margin="5"
        Clicked="OnGetRouteClicked"/>
    <maps:Map x:Name="mapControl"
        IsTrafficEnabled="True" Grid.Row="1"
        IsShowingUser="True"
        MapType="Street" Margin="5"
        HeightRequest="500"
        WidthRequest="400"
        MapClicked="OnMapClicked"/>

    <ScrollView Grid.Row="2">
        <Label x:Name="DirectionsLabel"
            Margin="5"/>
    </ScrollView>
</Grid>
</ContentPage>

```

The XAML code declares a **Grid** layout with three rows. The first row contains a **Button** that triggers the **OnGetRouteClicked** event when pressed and that will be handled in the next section. The second row declares a **Map** control, configured with properties such as **IsTrafficEnabled** (to display traffic data), **IsShowingUser** (to show the user's current location), and **MapType** set to **Street**. Other supported values are **Satellite**, which displays satellite images for the area, and **Hybrid**, which combines satellite images with street images. When the user selects a destination on the map, the **MapClicked** event is fired (handled in the next section). The third row includes a scrollable view that displays route directions or other relevant information dynamically in a **Label**. Now it is time to write C# code that calculates the route.

Calculating routes

The goal of the C# code is to retrieve and display a route that is optimized based on the traffic conditions and user preferences. Code Listing 15 demonstrates this.



Tip: Notice how the code checks again for location permissions. Even if you implement a similar check at the Android app startup in the *MainActivity.cs* file, this is not redundant. Instead, it is best practice. If the user revokes permissions while the

app is running but before using the location service, your code is still safe because of this double-check.

Code Listing 15

```
using Azure;
using Microsoft.Maui.Controls.Maps;

namespace TravelCompanion;

public partial class MapsPage : ContentPage
{
    public MapsPage()
    {
        InitializeComponent();
    }

    private Location selectedDestination;

    private void OnMapClicked(object sender, MapClickedEventArgs e)
    {
        selectedDestination = new Location(e.Location.Latitude,
            e.Location.Longitude);

        Pin destinationPin = new Pin
        {
            Label = "Destination",
            Type = PinType.Place,
            Location = new Location(selectedDestination.Latitude,
                selectedDestination.Longitude)
        };
        mapControl.Pins.Clear();
        mapControl.Pins.Add(destinationPin);
    }

    private async void OnGetRouteClicked(object sender,
        EventArgs e)
    {
        if (selectedDestination != null)
        {
            CalculateRoute(selectedDestination.Latitude,
                selectedDestination.Longitude);
        }
        else
        {
            await DisplayAlert("Error",
                "Please select a destination on the map", "OK");
        }
    }
}
```

```

private async Task<bool> RequestLocationPermissionsAsync()
{
    var locationStatus = await Permissions.
        CheckStatusAsync<Permissions.LocationWhenInUse>();
    if (locationStatus != PermissionStatus.Granted)
    {
        locationStatus = await Permissions.
            RequestAsync<Permissions.LocationWhenInUse>();
    }

    return locationStatus == PermissionStatus.Granted;
}

public async void CalculateRoute(double destinationLat,
    double destinationLon)
{
    bool permissions = await RequestLocationPermissionsAsync();
    if (!permissions) return;
    try
    {
        var routeService = new RouteService("api-key");

        GeolocationRequest request =
            new GeolocationRequest(GeolocationAccuracy.Low,
                TimeSpan.FromSeconds(10));

        var _cancellationTokenSource = new CancellationTokenSource();

        Location currentLocation =
            await Geolocation.Default.
                GetLocationAsync(request, _cancellationTokenSource.Token);

        if (currentLocation == null)
        {
            await DisplayAlert("Error",
                "Unable to get current location", "OK");
            return;
        }

        var route = await routeService.GetOptimizedRouteAsync(
            currentLocation.Latitude, currentLocation.Longitude,
            destinationLat, destinationLon
        );

        if (route.Routes.Count > 0)
        {

```

```

        var singleRoute = route.Routes[0];

        var polyline = new Polyline
        {
            StrokeColor = Colors.Blue, // Set polyline color
            StrokeWidth = 5
        };

        foreach (var point in singleRoute.Legs[0].Points)
        {
            polyline.Geopath.Add(
                new Location(point.Latitude, point.Longitude));
        }

        mapControl.MapElements.Clear();
        mapControl.MapElements.Add(polyline);

        var directions = singleRoute.Guidance.
            Instructions.Select(i => i.Message).ToList();

        string formattedDirections = string.Join("\n", directions);
        DirectionsLabel.Text = formattedDirections;
    }
}
catch (RequestFailedException ex)
{
    await DisplayAlert("Error", ex.ErrorCode, "OK");
}
catch (Exception ex)
{
    await DisplayAlert("Error", ex.Message, "OK");
}
}
}

```

The following is an explanation of the flow and the relevant Azure Maps SDK objects used in the code. When the user clicks a location in the **Map** control, the app double-checks for location permissions and asks for them again if necessary. If permissions are granted (see the **RequestLocationPermissionsAsync** method), the current user location is retrieved via the **GetLocationAsync** method from the **Geolocation** class.



Tip: Make sure that the location sensor and services are enabled on the target device, otherwise the code will fail. In a real-world app, you should programmatically check if location is enabled and prompt the user accordingly.

The resulting **Location** instance represents the selected geographic coordinates. A pin, created using the **Pin** class, is placed at the selected point with properties like **Label** to name the destination and **Type** to specify its purpose as a place marker. The pin is added to the **MapControl.Pins** collection, ensuring it is dynamically displayed on the map. When the button is clicked, the **OnGetRouteClicked** method is invoked. If a destination is already selected, the **CalculateRoute** method retrieves the current location using geolocation APIs, ensuring location permissions are granted via the **RequestLocationPermissionsAsync** method.

The core routing logic relies on the **RouteService** class, which is initialized with an API key to authenticate with Azure Maps services. The **GetOptimizedRouteAsync** method is then called to fetch the most efficient route, taking the current and destination coordinates as parameters. The method returns a **RouteResponse** object containing route details, including a collection of route legs represented by the **RouteLeg** class. The route's path is visualized on the map using the **Polyline** class, which generates a collection of points. Properties like **StrokeColor** and **StrokeWidth** define the color and width of the polyline, respectively. Each point in the route leg's **Points** collection is added to the polyline's **Geopath**, representing the optimized route.

This polyline is then added to the **MapControl.MapElements** collection, ensuring it appears as an overlay on the map. Additionally, turn-by-turn instructions are extracted from the route's guidance messages using the **GuidanceInstructions** class and its **Instructions** property. These instructions are formatted into a readable string and displayed in the **DirectionsLabel** control, providing users with a clear set of navigation steps.

Adding the page to the navigation stack

In .NET MAUI, the **TabBar** of the shell can display up to five icons. This is to ensure consistent views across devices. Since you already have five icons, for the sake of simplicity, in the **AppShell.xaml** file, comment out the declaration of the **DocProcessorPage.xaml** and add the new page, as follows:

```
<!--<ShellContent Title="Docs" Icon="document.png"
    ContentTemplate="{DataTemplate local:DocProcessorPage}" />-->
<ShellContent Title="Maps" Icon="map.png"
    ContentTemplate="{DataTemplate local:MapsPage}" />
```

Once you have finished with this example, you can revert to the previous button implementation. You can also consider implementing a flyout menu, but this is not covered here.

Running the application

Press **F5** to run the application. At startup, you will be asked to grant location permissions, so give your consent. Open the new page, select the target city, and see how a pin is added to the map. When ready, click the button. After a few seconds, the AI-optimized route will be displayed, as shown in Figure 41.

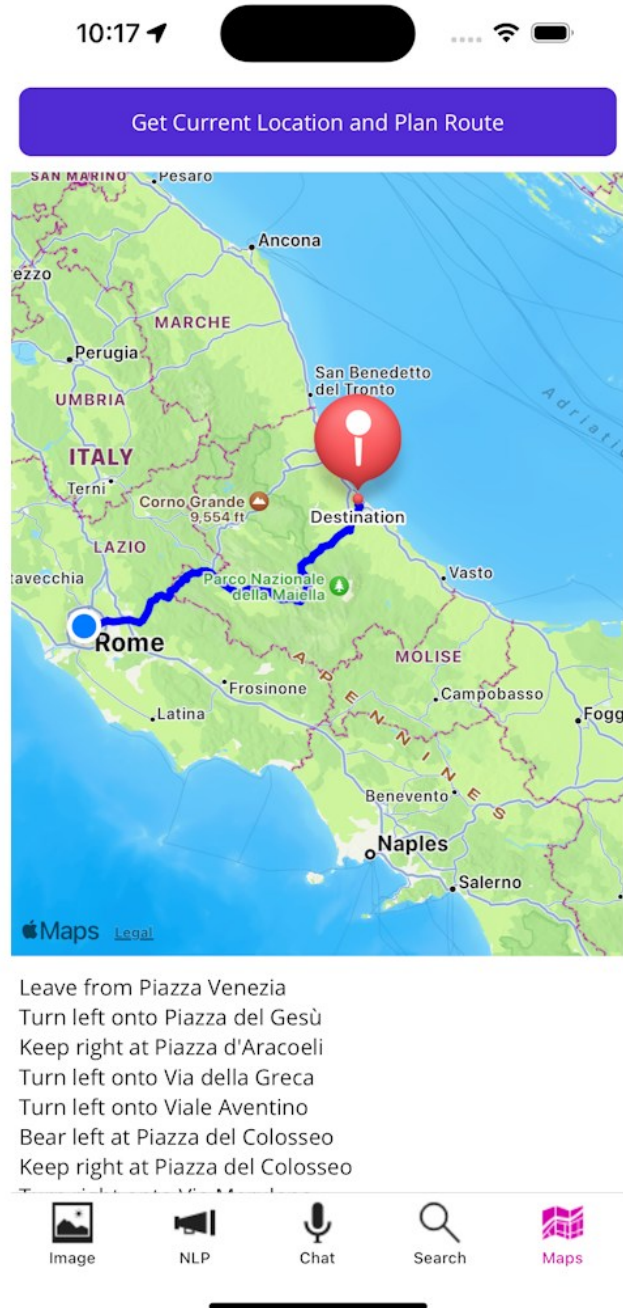
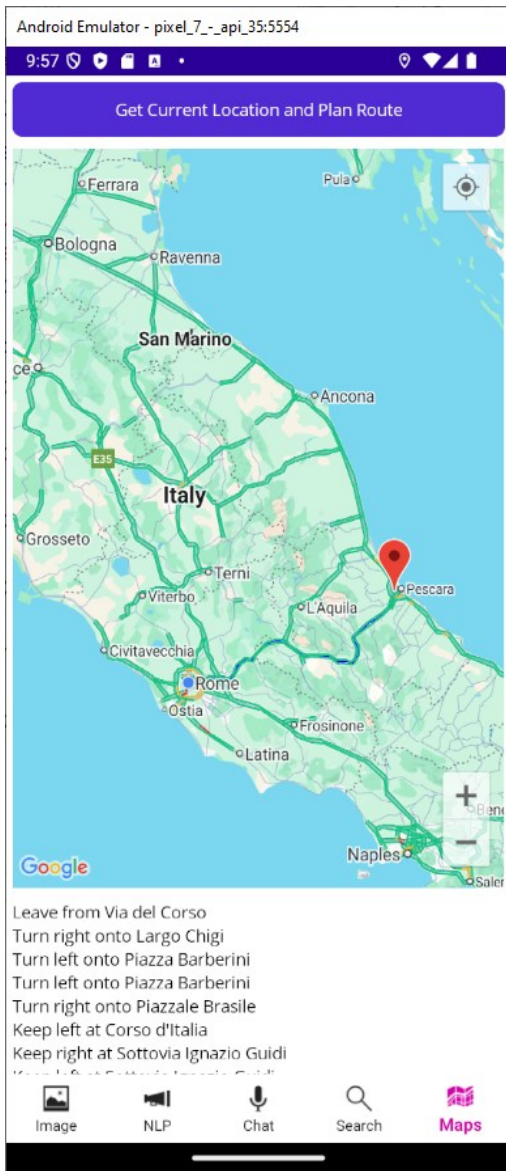


Figure 41: Calculating optimal routes with direction information

You can further refine the route optimization and also get the current location at startup, depending on your requirements.



Tip: On iOS, the native map view does not include zoom controls. If you use a simulator, it is recommended that you run it on the Mac, without remoting the simulator to Windows. In fact, on the Mac, you can press Option and click to zoom the map. On Windows, the Alt + click combination does not always work.

Chapter summary

In this chapter, you have seen how to leverage the Azure Maps service to provide route calculations that AI optimizes based on traffic and other conditions that can be relevant to the user of a mobile app. You have also seen how to set up a Google Maps account and how to implement permission requests.

Conclusion

AI is transforming mobile applications across industries. From chatbots that enhance customer service to real-time language translation and intelligent recommendations, AI-driven mobile experiences are becoming the norm. Integrating AI services in .NET MAUI applications allows developers to build cross-platform apps that are more intuitive, adaptive, and powerful. As mobile devices continue to evolve, leveraging AI will be crucial for delivering smarter applications that anticipate user needs, automate tasks, and provide enhanced personalization. Whether you are developing productivity tools, e-commerce platforms, or accessibility applications, AI services can give your app a competitive edge in today's digital landscape.

Throughout this ebook, you have explored how to integrate Microsoft's AI services into .NET MAUI applications, creating intelligent, feature-rich mobile experiences. From speech recognition and computer vision to search capabilities and document analysis, you have seen how AI can enhance app functionality, improve user interactions, and automate complex tasks. By mastering these AI integrations, you are well-equipped to create the next generation of intelligent mobile applications.